

[MS-MWBF]:

Microsoft Web Browser Federated Sign-On Protocol

Intellectual Property Rights Notice for Open Specifications Documentation

- **Technical Documentation.** Microsoft publishes Open Specifications documentation (“this documentation”) for protocols, file formats, data portability, computer languages, and standards support. Additionally, overview documents cover inter-protocol relationships and interactions.
- **Copyrights.** This documentation is covered by Microsoft copyrights. Regardless of any other terms that are contained in the terms of use for the Microsoft website that hosts this documentation, you can make copies of it in order to develop implementations of the technologies that are described in this documentation and can distribute portions of it in your implementations that use these technologies or in your documentation as necessary to properly document the implementation. You can also distribute in your implementation, with or without modification, any schemas, IDLs, or code samples that are included in the documentation. This permission also applies to any documents that are referenced in the Open Specifications documentation.
- **No Trade Secrets.** Microsoft does not claim any trade secret rights in this documentation.
- **Patents.** Microsoft has patents that might cover your implementations of the technologies described in the Open Specifications documentation. Neither this notice nor Microsoft's delivery of this documentation grants any licenses under those patents or any other Microsoft patents. However, a given Open Specifications document might be covered by the Microsoft [Open Specifications Promise](#) or the [Microsoft Community Promise](#). If you would prefer a written license, or if the technologies described in this documentation are not covered by the Open Specifications Promise or Community Promise, as applicable, patent licenses are available by contacting iplg@microsoft.com.
- **License Programs.** To see all of the protocols in scope under a specific license program and the associated patents, visit the [Patent Map](#).
- **Trademarks.** The names of companies and products contained in this documentation might be covered by trademarks or similar intellectual property rights. This notice does not grant any licenses under those rights. For a list of Microsoft trademarks, visit www.microsoft.com/trademarks.
- **Fictitious Names.** The example companies, organizations, products, domain names, email addresses, logos, people, places, and events that are depicted in this documentation are fictitious. No association with any real company, organization, product, domain name, email address, logo, person, place, or event is intended or should be inferred.

Reservation of Rights. All other rights are reserved, and this notice does not grant any rights other than as specifically described above, whether by implication, estoppel, or otherwise.

Tools. The Open Specifications documentation does not require the use of Microsoft programming tools or programming environments in order for you to develop an implementation. If you have access to Microsoft programming tools and environments, you are free to take advantage of them. Certain Open Specifications documents are intended for use in conjunction with publicly available standards specifications and network programming art and, as such, assume that the reader either is familiar with the aforementioned material or has immediate access to it.

Support. For questions and support, please contact dochelp@microsoft.com.

Revision Summary

Date	Revision History	Revision Class	Comments
10/22/2006	0.01	New	Version 0.01 release
1/19/2007	1.0	Major	Version 1.0 release
3/2/2007	1.1	Minor	Version 1.1 release
4/3/2007	1.2	Minor	Version 1.2 release
5/11/2007	1.3	Minor	Version 1.3 release
6/1/2007	1.3.1	Editorial	Changed language and formatting in the technical content.
7/3/2007	1.3.2	Editorial	Changed language and formatting in the technical content.
7/20/2007	1.3.3	Editorial	Changed language and formatting in the technical content.
8/10/2007	1.4	Minor	Clarified the meaning of the technical content.
9/28/2007	1.4.1	Editorial	Changed language and formatting in the technical content.
10/23/2007	1.5	Minor	Clarified the meaning of the technical content.
11/30/2007	1.6	Minor	Clarified the meaning of the technical content.
1/25/2008	1.6.1	Editorial	Changed language and formatting in the technical content.
3/14/2008	1.6.2	Editorial	Changed language and formatting in the technical content.
5/16/2008	1.6.3	Editorial	Changed language and formatting in the technical content.
6/20/2008	2.0	Major	Content changes for Release codenamed "Geneva".
7/25/2008	2.0.1	Editorial	Changed language and formatting in the technical content.
8/29/2008	3.0	Major	Removed "Geneva" content.
10/24/2008	4.0	Major	Updated and revised the technical content.
12/5/2008	4.0.1	Editorial	Changed language and formatting in the technical content.
1/16/2009	4.0.2	Editorial	Changed language and formatting in the technical content.
2/27/2009	4.0.3	Editorial	Changed language and formatting in the technical content.
4/10/2009	4.1	Minor	Clarified the meaning of the technical content.
5/22/2009	4.1.1	Editorial	Changed language and formatting in the technical content.
7/2/2009	5.0	Major	Updated and revised the technical content.
8/14/2009	6.0	Major	Updated and revised the technical content.
9/25/2009	6.1	Minor	Clarified the meaning of the technical content.
11/6/2009	6.1.1	Editorial	Changed language and formatting in the technical content.
12/18/2009	6.1.2	Editorial	Changed language and formatting in the technical content.
1/29/2010	6.2	Minor	Clarified the meaning of the technical content.

Date	Revision History	Revision Class	Comments
3/12/2010	6.2.1	Editorial	Changed language and formatting in the technical content.
4/23/2010	6.2.2	Editorial	Changed language and formatting in the technical content.
6/4/2010	6.2.3	Editorial	Changed language and formatting in the technical content.
7/16/2010	6.2.3	None	No changes to the meaning, language, or formatting of the technical content.
8/27/2010	6.2.3	None	No changes to the meaning, language, or formatting of the technical content.
10/8/2010	6.2.3	None	No changes to the meaning, language, or formatting of the technical content.
11/19/2010	6.2.3	None	No changes to the meaning, language, or formatting of the technical content.
1/7/2011	6.2.3	None	No changes to the meaning, language, or formatting of the technical content.
2/11/2011	6.2.3	None	No changes to the meaning, language, or formatting of the technical content.
3/25/2011	6.2.3	None	No changes to the meaning, language, or formatting of the technical content.
5/6/2011	6.2.3	None	No changes to the meaning, language, or formatting of the technical content.
6/17/2011	6.3	Minor	Clarified the meaning of the technical content.
9/23/2011	6.3	None	No changes to the meaning, language, or formatting of the technical content.
12/16/2011	7.0	Major	Updated and revised the technical content.
3/30/2012	7.0	None	No changes to the meaning, language, or formatting of the technical content.
7/12/2012	7.0	None	No changes to the meaning, language, or formatting of the technical content.
10/25/2012	7.0	None	No changes to the meaning, language, or formatting of the technical content.
1/31/2013	7.0	None	No changes to the meaning, language, or formatting of the technical content.
8/8/2013	8.0	Major	Updated and revised the technical content.
11/14/2013	9.0	Major	Updated and revised the technical content.
2/13/2014	9.0	None	No changes to the meaning, language, or formatting of the technical content.
5/15/2014	9.0	None	No changes to the meaning, language, or formatting of the technical content.
6/30/2015	10.0	Major	Significantly changed the technical content.
7/14/2016	11.0	Major	Significantly changed the technical content.

Date	Revision History	Revision Class	Comments
6/1/2017	12.0	Major	Significantly changed the technical content.
9/15/2017	13.0	Major	Significantly changed the technical content.
12/1/2017	13.0	None	No changes to the meaning, language, or formatting of the technical content.
9/12/2018	14.0	Major	Significantly changed the technical content.
4/7/2021	15.0	Major	Significantly changed the technical content.
4/23/2024	16.0	Major	Significantly changed the technical content.

Table of Contents

1	Introduction	8
1.1	Glossary	8
1.2	References	10
1.2.1	Normative References	10
1.2.2	Informative References	12
1.3	Overview	12
1.4	Relationship to Other Protocols	13
1.5	Prerequisites/Preconditions	14
1.6	Applicability Statement	15
1.7	Versioning and Capability Negotiation	15
1.7.1	Versioning	15
1.7.2	Capability Negotiation	15
1.8	Vendor-Extensible Fields	15
1.9	Standards Assignments.....	15
2	Messages.....	16
2.1	Transport	16
2.2	Message Syntax	16
2.2.1	Common Syntax for Request Messages	17
2.2.2	Common Syntax for Response Messages	17
2.2.3	wsignin1.0 Request Message.....	17
2.2.4	wsignin1.0 Response Message.....	19
2.2.4.1	High-Level Format of wresult Parameter	19
2.2.4.2	Security Token Format.....	19
2.2.4.2.1	Assertion Statements.....	20
2.2.4.2.1.1	Authentication Statements	20
2.2.4.2.1.2	Attribute Statements.....	20
2.2.4.2.1.3	Subject Element	21
2.2.4.2.2	Security Token Signature	21
2.2.5	wsignout1.0 Request Message.....	21
2.2.6	wsignoutcleanup1.0 Request Message	22
2.3	Directory Service Schema Elements	22
3	Protocol Details	23
3.1	Common Details for Requestor IP/STS and Relying Party Roles	23
3.1.1	Abstract Data Model.....	23
3.1.1.1	Security Token	23
3.1.1.2	User Authentication Context	23
3.1.1.3	Federation Partner	24
3.1.1.4	Claim	25
3.1.1.5	Federation Partner Session Lists for Web Browser Requestors.....	27
3.1.1.5.1	Requestor IP/STS Web Browser Requestor Sessions List.....	27
3.1.1.5.2	Relying Party Web Browser Requestor Sessions List.....	27
3.1.2	Timers	28
3.1.3	Initialization	28
3.1.4	Higher-Layer Triggered Events	28
3.1.5	Processing Events and Sequencing Rules	28
3.1.5.1	Determining Message Type	29
3.1.5.2	Error Handling	29
3.1.5.3	Requesting a Security Token by Issuing a wsignin1.0 Request Message	29
3.1.5.3.1	Protocol Activation.....	29
3.1.5.3.2	Parameter Marshaling	29
3.1.5.3.3	Requestor IP/STS Security Realm Discovery	30
3.1.5.3.4	Message Transmission	30
3.1.5.4	Issuing a Security Token by Responding to a wsignin1.0 Request Message...	30

3.1.5.4.1	Protocol Activation.....	31
3.1.5.4.2	Message Validation	31
3.1.5.4.3	User Identification and Authentication	31
3.1.5.4.4	User Attribute Retrieval.....	31
3.1.5.4.5	Claim Mapping	31
3.1.5.4.6	SAML Assertion Construction.....	32
3.1.5.4.7	Response Message Processing.....	32
3.1.6	Timer Events.....	32
3.1.7	Other Local Events.....	32
3.2	Requestor IP/STS Details	32
3.2.1	Abstract Data Model.....	32
3.2.2	Timers	32
3.2.3	Initialization	33
3.2.4	Higher-Layer Triggered Events	33
3.2.5	Processing Events and Sequencing Rules	33
3.2.5.1	Issuing a Security Token by Responding to a wsignin1.0 Request Message...	33
3.2.5.2	Inbound wsignout1.0 Request Message Processing.....	34
3.2.5.2.1	Protocol Activation.....	34
3.2.5.2.2	Clean-Up Processing	34
3.2.5.2.3	Response Message Processing.....	34
3.2.5.3	Outbound wsignoutcleanup1.0 Request Message Processing	34
3.2.5.3.1	Protocol Activation.....	34
3.2.5.3.2	Relying Party Security Realm Discovery.....	34
3.2.5.3.3	Clean-Up Processing	34
3.2.5.3.4	Message Transmission	35
3.2.6	Timer Events.....	35
3.2.7	Other Local Events.....	35
3.3	Relying Party Details	35
3.3.1	Abstract Data Model.....	35
3.3.1.1	Resource IP/STS Abstract Data Model Extensions.....	35
3.3.1.2	WS Resource Abstract Data Model Extensions.....	36
3.3.2	Timers	36
3.3.3	Initialization	36
3.3.4	Higher-Layer Triggered Events	36
3.3.5	Processing Events and Sequencing Rules	36
3.3.5.1	Requesting a Security Token by Sending a wsignin1.0 Request Message	37
3.3.5.1.1	Protocol Activation.....	37
3.3.5.1.2	Parameter Marshaling	37
3.3.5.2	Receiving a Security Token by Processing a wsignin1.0 Response Message ..	37
3.3.5.2.1	Protocol Activation.....	38
3.3.5.2.2	Message Validation	38
3.3.5.2.3	User Identification and Authentication	38
3.3.5.2.4	User Attribute Retrieval.....	38
3.3.5.2.5	Claim Mapping	38
3.3.5.2.6	Resource Access Control.....	38
3.3.5.3	Outbound wsignout1.0 Request Message Processing.....	38
3.3.5.3.1	Protocol Activation.....	38
3.3.5.3.2	Parameter Marshaling	38
3.3.5.3.3	Requestor IP/STS Security Realm Discovery	39
3.3.5.3.4	Message Transmission	39
3.3.5.4	Inbound wsignoutcleanup1.0 Request Message Processing	39
3.3.5.4.1	Protocol Activation.....	39
3.3.5.4.2	Clean-Up Processing	39
3.3.5.4.3	Relying Party Security Realm Discovery.....	39
3.3.5.4.4	Message Transmission	39
3.3.5.4.5	Response Message Processing.....	39
3.3.6	Timer Events.....	40
3.3.7	Other Local Events.....	40

3.4	Web Browser Requestor Details	40
3.4.1	Abstract Data Model.....	40
3.4.2	Timers	40
3.4.3	Initialization.....	40
3.4.4	Higher-Layer Triggered Events	40
3.4.5	Processing Events and Sequencing Rules	40
3.4.6	Timer Events.....	41
3.4.7	Other Local Events.....	41
4	Protocol Examples	42
4.1	Message Flows.....	42
4.2	XML Examples	48
4.2.1	Example RSTR.....	48
4.2.2	Example SAML Attribute Element.....	48
4.2.3	Using the X509Certificate Element	48
4.2.4	Using the X509SKI Element	49
4.3	Raw Message Examples	49
4.3.1	Original GET to WS Resource	49
4.3.2	HTTP Redirect to Resource IP/STS	49
4.3.3	HTTP GET To Resource IP/STS	49
4.3.4	HTTP Redirect to Requestor IP/STS	50
4.3.5	HTTP GET to Requestor IP/STS.....	50
4.3.6	Receive Security Token from Requestor IP/STS in HTML Form	50
4.3.7	HTTP POST Security Token to Resource IP/STS	52
4.3.8	Receive Security Token from Resource IP/STS in HTML Form	54
4.3.9	HTTP POST Security Token to WS Resource	55
4.3.10	Final HTTP 200 OK Response from WS Resource.....	57
5	Security	58
5.1	Security Considerations for Implementers	58
5.1.1	Security Token Integrity	58
5.1.2	Certificate Validation	58
5.1.3	Confidentiality	58
5.1.4	Replay Attack.....	58
5.1.5	Privacy	58
5.1.6	Identifiers.....	59
5.1.7	Cookies	59
5.2	Index of Security Parameters	59
6	Appendix A: Product Behavior	60
7	Change Tracking.....	69
8	Index.....	70

1 Introduction

The Microsoft Web Browser Federated Sign-On Protocol is primarily a restriction of the protocol specified in [\[WSFederation1.2\]](#) section 13. The restrictions are designed to enable greater interoperability by reducing the number of variations that have to be implemented. This document specifies minor additions to [\[WSFederation1.2\]](#) section 13 to handle common scenarios. This protocol is designed to enable the communication of a requestor's identity and attributes for the purpose of enabling access to a protected HTTP web application or its resources.

This protocol is based on the Web Service (WS) Federation Protocol described in [\[WSFederation\]](#) and [\[WSFederation1.2\]](#) section 13.

Sections 1.5, 1.8, 1.9, 2, and 3 of this specification are normative. All other sections and examples in this specification are informative.

1.1 Glossary

This document uses the following terms:

Active Directory Federation Services (AD FS): A Microsoft implementation of a federation services provider, which provides a security token service (STS) that can issue security tokens to a caller using various protocols such as WS-Trust, WS-Federation, and Security Assertion Markup Language (SAML) version 2.0.

AD FS behavior level: A specification of the functionality available in an AD FS server. Possible values such as AD_FS_BEHAVIOR_LEVEL_1 and AD_FS_BEHAVIOR_LEVEL_2 are described in [\[MS-OAPX\]](#).

AD FS server: See authorization server in [\[RFC6749\]](#).

claim: A declaration made by an entity (for example, name, identity, key, group, privilege, and capability). For more information, see [\[WSFederation1.2\]](#).

Coordinated Universal Time (UTC): A high-precision atomic time standard that approximately tracks Universal Time (UT). It is the basis for legal, civil time all over the Earth. Time zones around the world are expressed as positive and negative offsets from UTC. In this role, it is also referred to as Zulu time (Z) and Greenwich Mean Time (GMT). In these specifications, all references to UTC refer to the time at UTC-0 (or GMT).

digest: The fixed-length output string from a one-way hash function that takes a variable-length input string and is probabilistically unique for every different input string. Also, a cryptographic checksum of a data (octet) stream.

DNS name: A fully qualified domain name (FQDN).

domain naming service name: The fully qualified domain name (FQDN) as known by the Domain Name System (DNS), as specified in [\[RFC1035\]](#) and [\[RFC1123\]](#).

federation: A collection of **security realms** that have established **trust**.

identity provider/security token service (IP/STS): An **STS** that might also be an identity provider (IP). This term is used as shorthand to see both identity that verifies token services and general token services that do not verify identity. Note that the "/" symbol implies an "or" relationship.

relying party (RP): A web application or service that consumes **security tokens** issued by a **security token service (STS)**.

requestor IP/STS: An **IP/STS** in the same **security realms** as the **web browser requestor**. The **requestor IP/STS** has an existing relationship with the **user** that enables it to issue **security tokens** containing **user** information.

resource IP/STS: An **IP/STS** in the same **security realm** as the **web service (WS) resource**. The **resource IP/STS** has an existing relationship with the **WS resource** that enables it to issue **security tokens** that are trusted by the **WS resource**.

Rivest-Shamir-Adleman (RSA): A system for public key cryptography. **RSA** is specified in [\[RFC8017\]](#).

security realm or security domain: Represents a single unit of security administration or **trust**, for example, a Kerberos realm (for more information, see [\[RFC4120\]](#)) or a Windows Domain (for more information, see [\[MSFT-ADC\]](#)).

security token: A collection of one or more **claims**. Specifically in the case of mobile devices, a **security token** represents a previously authenticated user as defined in the Mobile Device Enrollment Protocol [\[MS-MDE\]](#).

security token service (STS): A web service that issues **security tokens**. That is, it makes assertions based on evidence that it **trusts**; these assertions are for consumption by whoever trusts it.

signature: A value computed with a cryptographic algorithm and bound to data in such a way that intended recipients of the data can use the **signature** to verify that the data has not been altered and/or has originated from the signer of the message, providing message integrity and authentication. The **signature** can be computed and verified either with symmetric key algorithms, where the same key is used for signing and verifying, or with asymmetric key algorithms, where different keys are used for signing and verifying (a private and public key pair are used). For more information, see [\[WSFederation1.2\]](#).

sign-out: The process by which a **user** (or an agent acting on the user's behalf) indicates that it will no longer be using its **security token**, and **relying parties** across **security realms** can destroy their **security token** caches for the **user**. For more information, see [\[WSFederation1.2\]](#). Note that the use of the term **sign-out** is based on [\[WSFederation1.2\]](#).

single sign-on (SSO): An authentication and authorization scheme in which a user needs only one set of credentials in order to access unrelated network resources.

subject key identifier (SKI): The **SKI** extension provides a means of identifying certificates that contain a particular public key. For more information, see [\[RFC3280\]](#) section 4.2.1.2.

trust: The characteristic that one entity is willing to rely on a second entity to execute a set of actions and/or to make a set of assertions about a set of subjects and/or scopes. For more information, see [\[WSFederation1.2\]](#).

user: A person who employs a **web browser requestor** to access a **WS resource**.

user principal name (UPN): A user account name (sometimes referred to as the user logon name) and a domain name that identifies the domain in which the user account is located. This is the standard usage for logging on to a Windows domain. The format is: someone@example.com (in the form of an email address). In Active Directory, the userPrincipalName attribute of the account object, as described in [\[MS-ADTS\]](#).

web browser requestor: An HTTP 1.1 web browser client that transmits protocol messages between an **IP/STS** and a **relying party**.

web service (WS) resource: A destination HTTP 1.1 web application or an HTTP 1.1 resource serviced by the application. In the context of this protocol, it refers to the application or manager of the resource that receives identity information and assertions issued by an **IP/STS**

using this protocol. The **WS resource** is a **relying party** in the context of this protocol. For more information, see [WSFederation1.2].

MAY, SHOULD, MUST, SHOULD NOT, MUST NOT: These terms (in all caps) are used as defined in [RFC2119]. All statements of optional behavior use either MAY, SHOULD, or SHOULD NOT.

1.2 References

Links to a document in the Microsoft Open Specifications library point to the correct section in the most recently published version of the referenced document. However, because individual documents in the library are not updated at the same time, the section numbers in the documents may not match. You can confirm the correct section numbering by checking the [Errata](#).

1.2.1 Normative References

We conduct frequent surveys of the normative references to assure their continued availability. If you have any issue with finding a normative reference, please contact dochelp@microsoft.com. We will assist you in finding the relevant information.

[Excl-C14N] Boyer, J., Eastlake 3rd, D. E., and Reagle, J., "Exclusive XML Canonicalization Version 1.0", July 2002, <http://www.w3.org/TR/xml-exc-c14n/>

[HTML] World Wide Web Consortium, "HTML 4.01 Specification", W3C Recommendation, December 1999, <http://www.w3.org/TR/html4/>

[MS-ADA1] Microsoft Corporation, "[Active Directory Schema Attributes A-L](#)".

[MS-ADA3] Microsoft Corporation, "[Active Directory Schema Attributes N-Z](#)".

[MS-ADFSP] Microsoft Corporation, "[Active Directory Federation Services and Proxy Integration Protocol](#)".

[MS-ADTS] Microsoft Corporation, "[Active Directory Technical Specification](#)".

[MS-DTYP] Microsoft Corporation, "[Windows Data Types](#)".

[MS-MWBE] Microsoft Corporation, "[Microsoft Web Browser Federated Sign-On Protocol Extensions](#)".

[MS-OAPX] Microsoft Corporation, "[OAuth 2.0 Protocol Extensions](#)".

[MSKB-3172614] Microsoft Corporation, "July 2016 update rollup for Windows RT 8.1, Windows 8.1, and Windows Server 2012 R2", <https://support.microsoft.com/en-us/kb/3172614>

[MSKB-4088889] Microsoft Corporation, "March 22, 2018 - KB4088889 (OS Build 14393.2155)", <https://support.microsoft.com/en-us/help/4088889>

[OIDCCore] Sakimura, N., Bradley, J., Jones, M., de Medeiros, B., and Mortimore, C., "OpenID Connect Core 1.0 incorporating errata set 1", November 2014, http://openid.net/specs/openid-connect-core-1_0.html

[RFC1738] Berners-Lee, T., Masinter, L., and McCahill, M., Eds., "Uniform Resource Locators (URL)", RFC 1738, December 1994, <https://www.rfc-editor.org/info/rfc1738>

[RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, March 1997, <https://www.rfc-editor.org/info/rfc2119>

[RFC2396] Berners-Lee, T., Fielding, R., and Masinter, L., "Uniform Resource Identifiers (URI): Generic Syntax", RFC 2396, August 1998, <https://www.rfc-editor.org/info/rfc2396>

[RFC2616] Fielding, R., Gettys, J., Mogul, J., et al., "Hypertext Transfer Protocol -- HTTP/1.1", RFC 2616, June 1999, <https://www.rfc-editor.org/info/rfc2616>

[RFC2822] Resnick, P., Ed., "Internet Message Format", RFC 2822, April 2001, <https://www.rfc-editor.org/info/rfc2822>

[RFC3280] Housley, R., Polk, W., Ford, W., and Solo, D., "Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile", RFC 3280, April 2002, <http://www.rfc-editor.org/info/rfc3280>

[RFC4648] Josefsson, S., "The Base16, Base32, and Base64 Data Encodings", RFC 4648, October 2006, <https://www.rfc-editor.org/info/rfc4648>

[RFC8017] Moriarty, K., Ed., Kaliski, B., Jonsson, J., and Rusch, A., "PKCS #1: RSA Cryptography Specifications Version 2.2", November 2016, <https://www.rfc-editor.org/info/rfc8017>

[SAMLASchema] OASIS Standard, "Security Assertion Markup Language (SAML) V1.1 XML Schema", September 2003, <http://www.oasis-open.org/committees/download.php/3408/oasis-sstc-saml-schema-assertion-1.1.xsd>

[SAMLCore] Maler, E., Mishra, P., Philpott, R., et al., "Assertions and Protocol for the OASIS Security Assertion Markup Language (SAML) V1.1", September 2003, <http://www.oasis-open.org/committees/download.php/3406/oasis-sstc-saml-core-1.1.pdf>

[WSA] Gudgin, M., Hadley, M., and Rogers, T., "Web Services Addressing 1.0 - Core", W3C Recommendation, May 2006, <http://www.w3.org/TR/2006/REC-ws-addr-core-20060509/>

[WSFederation1.2] Kaler, C., McIntosh, M., "Web Services Federation Language (WS-Federation)", Version 1.2, May 2009, <http://docs.oasis-open.org/wsfed/federation/v1.2/os/ws-federation-1.2-spec-os.html>

[WSFederation] Kaler, C., Nadalin, A., Bajaj, S., et al., "Web Services Federation Language (WS-Federation)", Version 1.1, December 2006, <http://specs.xmlsoap.org/ws/2006/12/federation/ws-federation.pdf>

[WSPolicyAtt] BEA Systems, IBM, Microsoft Corporation, SAP, Sonic Software, VeriSign, "Web Services Policy 1.2 - Attachment (WS-PolicyAttachment)", April 2006, <http://www.w3.org/Submission/WS-PolicyAttachment/>

[WSTrust] IBM, Microsoft, Nortel, VeriSign, "WS-Trust V1.0", February 2005, <http://specs.xmlsoap.org/ws/2005/02/trust/WS-Trust.pdf>

[X500] ITU-T, "Information Technology - Open Systems Interconnection - The Directory: Overview of Concepts, Models and Services", Recommendation X.500, August 2005, <http://www.itu.int/rec/T-REC-X.500-200508-S/en>

Note There is a charge to download the specification.

[X509] ITU-T, "Information Technology - Open Systems Interconnection - The Directory: Public-Key and Attribute Certificate Frameworks", Recommendation X.509, August 2005, <http://www.itu.int/rec/T-REC-X.509/en>

[XMLDSig] Bartel, M., Boyer, J., Fox, B., et al., "XML-Signature Syntax and Processing", W3C Recommendation, February 2002, <http://www.w3.org/TR/2002/REC-xmlsig-core-20020212/>

[XMLENC] Imamura, T., Dillaway, B., and Simon, E., "XML Encryption Syntax and Processing", W3C Recommendation, December 2002, <http://www.w3.org/TR/2002/REC-xmlenc-core-20021210/>

1.2.2 Informative References

- [FIPS180] FIPS PUBS, "Secure Hash Standard", FIPS PUB 180-1, April 1995, <https://www.niatec.iri.isu.edu/GetFile.aspx?pid=63>
- [IDFF] Liberty Alliance Project, "Liberty ID-FF Protocols and Schema Specification, Version: 1.2-errata-v2.0", 2004, <http://www.projectliberty.org/liberty/content/download/1992/13890/file/draft-liberty-idff-protocols-schema-1.2-errata-v2.0.pdf>
- [MS-PASS] Microsoft Corporation, "[Passport Server Side Include \(SSI\) Version 1.4 Protocol](#)".
- [RFC2246] Dierks, T., and Allen, C., "The TLS Protocol Version 1.0", RFC 2246, January 1999, <https://www.rfc-editor.org/info/rfc2246>
- [RFC2965] Kristol, D. and Montulli, L., "HTTP State Management Mechanism", RFC 2965, October 2000, <https://www.rfc-editor.org/info/rfc2965>
- [RFC3546] Blake-Wilson, S., Nystrom, M., Hopwood, D., Mikkelsen, J., and Wright, T., "Transport Layer Security (TLS) Extensions", RFC 3546, June 2003, <https://www.rfc-editor.org/info/rfc3546>
- [RFC4120] Neuman, C., Yu, T., Hartman, S., and Raeburn, K., "The Kerberos Network Authentication Service (V5)", RFC 4120, July 2005, <https://www.rfc-editor.org/rfc/rfc4120>
- [SAML20Prof] Hughes, J., Cantor, S., Hodges, J., et al., "Profiles for the OASIS Security Assertion Markup Language (SAML) V2.0", March 2005, <http://docs.oasis-open.org/security/saml/v2.0/saml-profiles-2.0-os.pdf>
- [XMLNS] Bray, T., Hollander, D., Layman, A., et al., Eds., "Namespaces in XML 1.0 (Third Edition)", W3C Recommendation, December 2009, <https://www.w3.org/TR/2009/REC-xml-names-20091208/>
- [XMLSCHEMA1] Thompson, H., Beech, D., Maloney, M., and Mendelsohn, N., Eds., "XML Schema Part 1: Structures", W3C Recommendation, May 2001, <https://www.w3.org/TR/2001/REC-xmlschema-1-20010502/>
- [XMLSCHEMA2] Biron, P.V., Ed. and Malhotra, A., Ed., "XML Schema Part 2: Datatypes", W3C Recommendation, May 2001, <https://www.w3.org/TR/2001/REC-xmlschema-2-20010502/>
- [XML] World Wide Web Consortium, "Extensible Markup Language (XML) 1.0 (Fourth Edition)", W3C Recommendation 16 August 2006, edited in place 29 September 2006, <http://www.w3.org/TR/2006/REC-xml-20060816/>

1.3 Overview

The requestor profile described in [WSFederation1.2] section 13 is designed to address two problems related to communicating **user** information to **web service (WS) resources**.

First, in order to properly control access to information in WS resources, those WS resources have to have information about the users who are accessing them. Previous solutions required the application to identify the user and to use that identification information to access information about the user. Second, users were forced to be prompted multiple times for user names and passwords to securely identify themselves across multiple WS resources.

The profile described in [WSFederation1.2] section 13 addresses these two problems by enabling the user to securely communicate account information across **security realms** to multiple WS resources without requiring multiple prompts for user names and passwords.

The profile described in [WSFederation1.2] section 13 solves these problems by moving the responsibility for authenticating the user away from the WS resource application to an **identity provider/security token service (IP/STS)** that already has an account for the user. This IP/STS

issues **security tokens** that contain information about the user. When accessing a WS resource, the user's **web browser requestor** presents a security token obtained from an IP/STS to the WS resource application. The **signature** of the security token allows the WS resource to verify its validity, and the content of the security token contains **claims** about the authentication with the IP/STS as well as the relevant user account information for the WS resource. These claims can then be used for authorization decisions by the WS resource. A sequence diagram describing the detailed exchange can be found in [WSFederation1.2] section 13.3.

The Microsoft Web Browser Federated Sign-On Protocol increases interoperability of the profile described in [WSFederation1.2] section 13 by restricting the protocol options and the variations of security tokens that can be included in [WSFederation1.2] section 13. The following list outlines restrictions to [WSFederation1.2] section 13:

- The HTTP verbs are restricted for different message types.
- The allowable message types are scoped down to message types directly related to sign-on or **sign-out** operations, as specified in [WSFederation1.2] sections 13.1.1 and 13.1.2.
- The parameters specified in [WSFederation1.2] section 13 are restricted for each message type.

This protocol also adds optional parameters to address existing limitations to the protocol.

In section [2.1](#), this document specifies restrictions on the choice of message transport allowed in [WSFederation1.2] section 13. The [WSFederation1.2] specification allows [wsignin1.0 \(section 2.2.4\)](#), [wsignout1.0 \(section 2.2.5\)](#), and [wsignoutcleanup1.0 \(section 2.2.6\)](#) operations to be transmitted using either GET or POST methods, as specified in [\[RFC2616\]](#). This specification restricts wsignin1.0, wsignout1.0, and wsignoutcleanup1.0 requests to use only the GET method. This protocol also restricts wsignin1.0 responses to be transmitted to **relying parties** using the POST method.

Parameter removals, restrictions, and additions are specified in section [2.2](#). The parameter restrictions and removals are designed to aid interoperability by reducing the possible variations in the protocol. The parameter additions address issues such as communicating the expected authentication method and communicating a user's **requestor IP/STS**. The Microsoft Web Browser Federated Sign-On Protocol restricts the content of the *wresult* parameter (using the standards as specified in [\[WSTrust\]](#) and [\[SAMLCore\]](#)) to enable interoperable communications of security tokens. The restrictions on the *wresult* parameter are specified in sections [2.2.4.1](#) and [2.2.4.2](#). The semantics and protocol details of these changes are addressed in section [3](#).

1.4 Relationship to Other Protocols

This protocol uses standard web protocols. To use this document effectively, the reader has to be familiar with the following IETF specifications:

- Uniform Resource Locators (URLs), as specified in [\[RFC1738\]](#).
- Uniform Resource Identifiers (URIs), as specified in [\[RFC2396\]](#).
- Hypertext Transfer Protocol (HTTP), as specified in [\[RFC2616\]](#).
- The HTTP State Management Mechanism (for more information, see [\[RFC2965\]](#)).

URLs and URIs are used to describe the data used in this protocol. This protocol depends on HTTP and its dependencies to communicate among **IP/STSS**, **relying parties**, and **web browser requestors**.

This protocol uses XML. To use this document effectively, the reader has to be familiar with the following W3C specifications:

- Extensible Markup Language (XML) 1.0 (Fourth Edition), as specified in [\[XML\]](#).
- Namespaces in XML, as specified in [\[XMLNS\]](#).

- XML Schema Part 1: Structures Second Edition, as specified in [\[XMLSCHEMA1\]](#).
- XML Schema Part 2: Datatypes Second Edition, as specified in [\[XMLSCHEMA2\]](#).
- XML-Signature Syntax and Processing, as specified in [\[XMLDSig\]](#).
- Exclusive XML Canonicalization Version 1.0, as specified in [\[Excl-C14N\]](#).

These specifications are used to describe the requirements for the XML syntax involved in the protocol.

This protocol uses the model specified in [\[WSFederation1.2\]](#) section 13.1, and restricts the message and parameters in [\[WSFederation1.2\]](#) section 13.2, to improve interoperability (that is, fewer protocol variations). To use this document effectively, the reader has to be familiar with the following specifications used in [\[WSFederation1.2\]](#):

- WS-Trust, as specified in [\[WSTrust\]](#).
- WS-Addressing, as specified in [\[WSA\]](#).
- WS-Policy, as specified in [\[WSPolicyAtt\]](#).
- WS-Federation, as specified in [\[WSFederation\]](#).

The Microsoft Web Browser Federated Sign-On Protocol uses Security Assertion Markup Language (SAML) 1.1 assertions to communicate **security tokens**. To use this document effectively, the reader has to be familiar with the OASIS specification for SAML 1.1 assertions (as specified in [\[SAMLCore\]](#)) and the SAML 1.1 XML Schema (as specified in [\[SAMLASchema\]](#)). These specifications build on XML syntax to communicate specific semantics for exchanging security tokens.

Currently, there are no known protocols that depend on the Microsoft Web Browser Federated Sign-On Protocol.

The Microsoft Web Browser Federated Sign-On Protocol is an alternative to the Web Browser SSO Protocol (for more information, see [\[SAML20Prof\]](#)), the Single Sign-On and Federation Protocol (for more information, see [\[IDFF\]](#)), or the Passport Protocol (for more information, see [\[MS-PASS\]](#)). While the Microsoft Web Browser Federated Sign-On Protocol and the Passport Protocol (for more information, see [\[MS-PASS\]](#)) are similar, there are two primary differences between them. The Microsoft Web Browser Federated Sign-On Protocol incorporates a rich abstract data model for communicating **user** information beyond a user name, while the Passport Protocol is primarily focused on communicating a user name across **security realms**. The Microsoft Web Browser Federated Sign-On Protocol is also intended to enable XML web service standards-based interoperability, while the Passport Protocol is not based on XML web service standards.

1.5 Prerequisites/Preconditions

For a **relying party** to verify information from an **IP/STS**, it must have the key material necessary to verify the digital **signature** on the message being communicated. The details of IP/STS configuration are vendor-specific.

For a requestor to initiate and engage in this protocol, a **web browser requestor** must be capable of both submitting HTML forms and following HTTP 302 redirects, as specified in [\[HTML\]](#) and [\[RFC2616\]](#).

The appropriate **trusts** must be in place to allow **security tokens** issued by the **requestor IP/STS** to be trusted by the **resource IP/STS**, and those issued by the resource IP/STS to be trusted by the **WS resource**. The trusts establishment methods are not addressed in this specification.

The web browser requestor is assumed to have knowledge of the URLs that correspond to WS resources that are protected using this protocol. The communication method of this information is not addressed in this specification.

1.6 Applicability Statement

The Microsoft Web Browser Federated Sign-On Protocol is used when a **web browser requestor** needs to communicate **user** information from one **security realm** to one or more HTTP **WS resources** in other security realms while avoiding repeated requests for authentication. This protocol is not applicable for non-HTTP-based applications.

1.7 Versioning and Capability Negotiation

1.7.1 Versioning

This specification defers all versioning issues to the specifications [\[WSFederation\]](#), [\[WSFederation1.2\]](#) section 13, and [\[SAMLCore\]](#). No additional versioning mechanisms are introduced in this specification.

1.7.2 Capability Negotiation

This specification defers all capability negotiation to the specifications [\[WSFederation\]](#), [\[WSFederation1.2\]](#) section 13, [\[SAMLCore\]](#), and [\[RFC2616\]](#). No additional capability negotiation mechanisms are introduced in this specification.

1.8 Vendor-Extensible Fields

As specified in section 2, the Microsoft Web Browser Federated Sign-On Protocol uses the SAML 1.1 token format, as specified in [\[SAMLCore\]](#), for **security tokens**. Vendors can extend the SAML Advice element to communicate extended data in the security token.<1> [\[MS-MWBE\]](#) specifies extensions to this protocol using the SAML Advice element. The XML elements placed under the SAML 1.1 Advice element can be guaranteed to be unique if the vendor registers the XML namespace URN with the Internet Assigned Numbers Authority (IANA).

Vendors can use the existing extensibility points, as specified in [\[WSFederation1.2\]](#) section 13.<2> As described in that specification, new URL parameters can be used to communicate extended information as part of the protocol. There is no process for guaranteeing that URL parameters added to the protocol by individual vendors are uniquely named across multiple vendors.

1.9 Standards Assignments

There are no standards assignments for this protocol beyond those specified in [\[WSFederation1.2\]](#) section 13, [\[WSTrust\]](#), and [\[SAMLCore\]](#).

2 Messages

This section specifies the transport and syntax of request and response messages in normative detail. References to section 3 are included when knowledge of the protocol details are necessary to understand the context of message transport or syntax.

This protocol references commonly used data types as defined in [\[MS-DTYP\]](#).

2.1 Transport

The [wsignin1.0 \(section 2.2.3\)](#), [wsignout1.0 \(section 2.2.5\)](#), and [wsignoutcleanup1.0 \(section 2.2.6\)](#) requests MUST be transmitted using the GET method; they MUST NOT be transmitted using the POST method. This protocol also restricts [wsignin1.0 \(section 2.2.4\)](#) responses; such responses SHOULD [<3>](#) be transmitted to **relying parties** using the POST method.

The **requestor IP/STS** and relying party SHOULD [<4>](#) use the HTTPS URL scheme to identify each **IP/STS** and **WS resource** for processing wsignin1.0 requests, wsignin1.0 responses, wsignout1.0 requests, and wsignoutcleanup1.0 requests. If the HTTPS URL scheme is not used, each IP/STS and WS resource is assumed to have other protection mechanisms or decided that protection is not necessary.

This protocol uses the redirection facilities of HTTP 1.1, as specified in [\[WSFederation1.2\]](#) section 13, to automate message exchange using the **web browser requestor**. Compliant web browser requestors MUST support HTTP 1.1 status code 302 redirection.

For wsignin1.0 requests, as depicted in the diagram in [\[WSFederation1.2\]](#) section 13.1.1, the web browser requestor MUST authenticate each IP/STS and the WS resource, using SSL/TLS transport security (for more information, see [\[RFC2246\]](#) and [\[RFC3546\]](#)) and X.509 certificates (as specified in [\[X509\]](#)). Before a **security token** is issued in response to a wsignin1.0 request message, the **user** MUST be authenticated to the IP/STS. The method by which the IP/STS verifies the user's identity is not addressed in this protocol. It is assumed here that the user's identity is verified in some way by the IP/STS. Requests for wsignout1.0 and wsignoutcleanup1.0, as depicted in the diagram of [\[WSFederation1.2\]](#) section 13.1.2, are not required [<5>](#) to be authenticated.

2.2 Message Syntax

Implementations conforming to this protocol MUST support the following messages from [\[WSFederation1.2\]](#).

Message	[WSFederation1.2]
wsignin1.0	Sections 13.2.1 and 13.2.2
wsignout1.0	Section 13.2.4
wsignoutcleanup1.0	Section 13.2.4

Implementations conforming to this protocol SHOULD NOT emit the following messages from [\[WSFederation1.2\]](#). Conforming implementations that receive one of these messages SHOULD [<6>](#) return an HTTP 403 error message if they receive such messages.

Message	[WSFederation1.2]
xml-attribute-request	Section 13.2.5
xml-pseudonym-request	Section 13.2.6

The following sections define the message syntax for the protocol, starting with common syntax for requests and responses. All parameters for protocol request and response messages MUST use HTTP encoding rules, as specified in [\[RFC2616\].<7>](#) For processing semantics on messages with parameters not specified in this document, see section [3.1.5.2](#).

2.2.1 Common Syntax for Request Messages

[\[WSFederation1.2\]](#) sections 13.2.1 and 13.2.2 specify the common syntax for requesting **security tokens**. For processing guidance on handling unsupported parameters, see section [3.1.5.2](#).

To simplify implementation and improve interoperability by restricting protocol variations, implementations conforming to this protocol SHOULD NOT [<8>](#) support the following parameters:

- *wres* (optional in [\[WSFederation1.2\]](#)): This URL parameter specifies the URL for the resource accessed. Note that this parameter is listed as a legacy parameter in [\[WSFederation1.2\]](#).
- *wp* (optional in [\[WSFederation1.2\]](#)): This optional parameter is a URL for policy that can be obtained using an HTTP GET.
- *wreq* (optional in [\[WSFederation1.2\]](#)): This optional parameter specifies a token request using either a `wsse:RequestSecurityToken` element or a full request message, as specified in [\[WSTrust\]](#).
- *wreqptr* (optional in [\[WSFederation1.2\]](#)): A URL where the **requestor IP/STS** can find (using an HTTP GET) the **relying party's** request.

If an implementation chooses to support these parameters (in addition to supporting the full protocol), it will still be compliant with the Microsoft Web Browser Federated Sign-On Protocol.

2.2.2 Common Syntax for Response Messages

[\[WSFederation1.2\]](#) section 13.2.3 specifies the common mechanisms for returning **security tokens**. For processing guidance on handling unsupported parameters, see section [3.1.5.2](#). To simplify implementation and improve interoperability by restricting protocol variations, the following parameters are restricted by this protocol:

- *wresult*: The issued security token that MUST be encoded as a `RequestSecurityTokenResponse` (RSTR) element, as specified in [\[WSTrust\]](#) section 6.2. This format is detailed in section [2.2.4.1](#).
- *wctx* (optional in [\[WSFederation1.2\]](#)): An opaque context value that MUST be returned with the response if it is passed in the request by the **relying party**. [<9>](#)

To simplify implementation and improve interoperability by restricting protocol variations, implementations in conformance to this protocol SHOULD NOT [<10>](#) support the following parameter:

- *wresultptr* (optional in [\[WSFederation1.2\]](#)): A URL where the relying party can find (using an HTTP GET) the **security token service's** response.

If an implementation chooses to support this parameter (in addition to supporting the full protocol), it will still be compliant with the Microsoft Web Browser Federated Sign-On Protocol.

2.2.3 wsignin1.0 Request Message

The `wsignin1.0` request message is sent to the **IP/STS** to request that a **security token** be issued for a specific **user** to allow access to resources managed by the **relying party**. For normative descriptions and details on this request message, see [\[WSFederation1.2\]](#) section 13.2.2. This message consists of an HTTP GET with the following query string parameters, formatted as specified in [\[WSFederation1.2\]](#) sections 13.2.1 and 13.2.2:

- *wa*: The value MUST be the literal string "wsignin1.0".

- *wtrealm*: This parameter MUST be included in a request message to a different **security realm** from the relying party. If present, this value MUST be a URI that the **requestor IP/STS** and the relying party have agreed to use to identify the security realm of the relying party in messages to the requestor IP/STS.
- *wreply* (optional): This parameter MAY be included in request messages to the same security realm as the relying party. If present, this value MUST be a URL to which responses MUST be directed. The requestor IP/STS MUST validate that this URL belongs to the relying party before directing responses to this URL. [<11>](#)
- *wctx* (optional): This value is an opaque context that MAY be passed in the request by the relying party. [<12>](#)
- *wct* (optional): This value is the current time at the relying party that MUST be the string encoding of time, using the XML schema `<datetime>` time with **Coordinated Universal Time (UTC)** notation. [<13>](#)
- *wauth* (optional): This value is a URI that indicates the method of authentication wanted. [<14>](#)
- *whr* (optional): This value is a URI that uniquely identifies the requestor IP/STS that SHOULD receive the wsignin1.0 request message. [<15>](#)
- *client-request-id* (optional): This value is a string that is used to specify a request identifier that is used when logging events, including errors or failures that occur while processing the request. [<16>](#)
- *login_hint* (optional): This value is a string that is used to provide a hint about the login identifier the end-user might use to log in. This value MAY be used to derive the IP/STS that SHOULD receive the wsignin1.0 request message. [<17>](#) Actual derivation is implementation specific.
- *username* (optional): This value is a string that is used to provide a hint about the login identifier the end-user might use to log in. This value MAY be used to derive the IP/STS that SHOULD receive the wsignin1.0 request message. [<18>](#) Actual derivation is implementation specific.
- *domain_hint* (optional): This value is a string that MAY be used to derive the IP/STS that SHOULD receive the wsignin1.0 request message. [<19>](#) Actual derivation is implementation specific.
- *prompt* (optional): This query parameter is used in the same way as the *prompt* parameter defined in [\[OIDCCore\]](#) section 3.1.2.1, but the only accepted value for this parameter is "login". [<20>](#) Any other values are ignored. This parameter is used to interactively prompt the end-user for re-authentication. Error handling for this parameter follows the specification of section [3.1.5.2](#).
- *mfa_max_age* (optional): This value is a string that is used to specify the allowable timespan, in seconds, within which the last multiple factor authentication of the user MUST have been performed by the IP/STS. The **AD FS server** ignores this parameter unless its `ad_fs_behavior_level` is `AD_FS_BEHAVIOR_LEVEL_3` or higher ([\[MS-OAPX\]](#) section 3.2.1.1). [<21>](#) The IP/STS SHOULD have a setting that configures it to issue the **claim** "http://schemas.microsoft.com/ws/2017/04/identity/claims/multifactorauthenticationinstant" in the security token to the relying party. The value of this claim SHOULD specify the time, in UTC, when the user last performed multiple factor authentication.

Note *login_hint* and *username* are aliases that signify the same query parameter and either of these query parameters can be used to provide a hint about the login identifier the end-user might use to log in.

2.2.4 wsignin1.0 Response Message

The wsignin1.0 response message is sent to the **relying party** that requested the **security token** to be issued. For normative descriptions and specifications on this response message, see [\[WSFederation1.2\]](#) section 13.2.3. This message consists of an HTTP POST with the following parameters encoded in the POST body, as specified in [\[WSFederation1.2\]](#) section 13.2.3:

- **wa**: This value MUST be the literal string "wsignin1.0".
- **wresult**: This value MUST be the issued security token encoded as a wst:RequestSecurityTokenResponse. The restrictions on the format of this element are specified further in section [2.2.4.1](#).
- **wctx** (optional): This value is an opaque context that MUST be returned with the response if it was included in the request by the relying party. [<22>](#)

2.2.4.1 High-Level Format of wresult Parameter

The syntax for successful [wsignin1.0](#) response message requires that the **wresult** parameter contain a **security token** that MUST be encoded as an RSTR element, as specified in [\[WSTrust\]](#) section 6.2. The RSTR MUST contain a RequestedSecurityToken element, as specified in [\[WSTrust\]](#) section 6.2. This child element MUST contain either a security token that is constructed as an Assertion element, as specified in [\[SAMLCore\]](#) section 2.3.2, or encrypted content in an **EncryptedData** element, as specified in [\[XMLENC\]](#) section 3.4.

If present, the syntax of the **Assertion** element MUST conform to a subset of the SAML assertion syntax, as specified in section [2.2.4.2](#).

The RSTR MAY [<23>](#) contain an AppliesTo element, as specified in [\[WSPolicyAtt\]](#) section 3.4. If present, this child element MUST contain an EndpointReference element, as specified in [\[WSA\]](#) section 2.2. The body of the Address element, also specified in [\[WSA\]](#) section 2.2, SHOULD specify the resource provider's **security realm** URI. Note that this data is redundant and MUST duplicate the information in the security token's Audience element, as specified in [\[SAMLCore\]](#) section 2.3.2.1.3.

Implementations that conform to this protocol MAY [<24>](#) include other optional elements or attributes in the RSTR. Such elements are informative to the requestor to indicate how the issuer processed the request. For further specification, see the RSTR example in section [4.2.1](#).

2.2.4.2 Security Token Format

As stated, the **security token** contained in the RequestedSecurityToken element, specified in section [2.2.4.1](#), MUST be formatted as an Assertion element, as specified in [\[SAMLCore\]](#) section 2.3.2, with the restrictions detailed in this section. For more specifications, [\[SAMLASchema\]](#) describes the full XML schema of a SAML assertion and describes all of the element names used in this section, unless otherwise specified. For processing semantics on SAML security tokens that do not conform to this message specification, see section [3.1.1.1](#).

The following restrictions are placed on the SAML assertion, as specified in [\[SAMLCore\]](#) section 2.3.2:

- The returned SAML assertion MUST use the schema specified in [\[SAMLASchema\]](#), corresponding to the namespace urn:oasis:names:tc:SAML:1.0:assertion.
- The **MajorVersion** attribute of the Assertion element MUST be 1, and the **MinorVersion** attribute of the Assertion element MUST be 1. These attributes are as specified in [\[SAMLCore\]](#) section 2.3.2.
- The returned SAML assertion MUST specify the **AssertionId**, **Issuer**, and **IssueInstant** attributes on the Assertion element. These attributes are as specified in [\[SAMLCore\]](#) section 2.3.2.

- The assertion MUST contain a Conditions element, which MUST specify the **NotBefore** and **NotOnOrAfter** attributes. These attributes are as specified in [SAMLCore] section 2.3.2.1.1.
- The Conditions element MUST contain an AudienceRestrictionCondition element that restricts the audience to the **resource IP/STS**. This element is specified in [SAMLCore] section 2.3.2.1.3.
- The AudienceRestrictionCondition element MUST contain one and only one Audience element that specifies the URI of the **relying party**.
- The Advice element, specified in [SAMLCore] section 2.3.2.2, MAY [<25>](#) be present in assertions conforming to this specification.

2.2.4.2.1 Assertion Statements

The following restriction is placed on the SAML statements used in the SAML assertion:

- Statements other than the AuthenticationStatement (specified in [\[SAMLCore\]](#) section 2.4.3) and the AttributeStatement (specified in [SAMLCore] section 2.4.4) MUST NOT be placed in the SAML assertion.

2.2.4.2.1.1 Authentication Statements

The following restrictions are placed on the SAML AuthenticationStatement used in the SAML assertion:

- The SAML assertion MUST contain one and only one AuthenticationStatement.
- An AuthenticationStatement MUST have a Subject element.
- The Subject element, as specified in [\[SAMLCore\]](#) section 2.4.2.1, MUST conform to the guidance of section [2.2.4.2.1.3](#).
- If an AttributeStatement is present, the Subject element in the AuthenticationStatement MUST match the Subject element in the AttributeStatement.
- The AuthenticationMethod and AuthenticationInstant attributes MUST be specified.
- The optional AuthenticationStatement elements SubjectLocality (specified in [SAMLCore] section 2.4.3.1) and AuthorityBinding (specified in [SAMLCore] section 2.4.3.2) MUST NOT be present in the **security token**.

2.2.4.2.1.2 Attribute Statements

The following restrictions are placed on a SAML AttributeStatement used in the SAML assertion:

- The SAML assertion MAY have one AttributeStatement.
- The SAML assertion MAY have no AttributeStatement.
- The SAML assertion MUST NOT have more than one AttributeStatement.
- The AttributeStatement, if present, MUST have a Subject element.
 - The Subject element MUST match the Subject element in the AuthenticationStatement.
 - The Subject element MUST conform to the guidance of section [2.2.4.2.1.3](#).
- The AttributeStatement, if present, MUST contain one or more Attribute elements, as specified in [\[SAMLCore\]](#) section 2.4.4.1. Each Attribute element encapsulates a name/value **claim**.

- The Attribute element MUST have **AttributeName** and the corresponding **AttributeNamespace** attributes specified. These attributes are specified in [SAMLCore] section 2.4.4.1. The **AttributeName** attribute specifies the name of the claim, and one or more AttributeValue elements (specified in [SAMLCore] section 2.4.4.1.1) specify the value (or values) of the claim. [<26>](#)
- All Attribute elements in the AttributeStatement SHOULD [<27>](#) have the namespace URL, <http://schemas.xmlsoap.org/claims/>, for the AttributeNamespace attribute value.

For more information, an example of a SAML attribute can be found in section [4.2.2](#). Values for the **AttributeName** attribute that correspond to claims are specified in the abstract data model in section [3](#).

2.2.4.2.1.3 Subject Element

The Subject element is used in both the AuthenticationStatement and the AttributeStatement. The Subject element MAY [<28>](#) specify the NameIdentifier element, as specified in [SAMLCore] section 2.4.2.2. The SubjectConfirmation element, as specified in [SAMLCore] section 2.4.2.3, MAY [<29>](#) be omitted.

For more details, the schema of a Subject element can be found in [\[SAMLASchema\]](#).

The **NameQualifier** attribute of the NameIdentifier element, as specified in [SAMLCore] section 2.4.2.2, MUST NOT be present. For more details, the schema of a NameIdentifier can be found in [\[SAMLASchema\]](#).

2.2.4.2.2 Security Token Signature

The **security token** MUST contain an enveloped XML digital **signature**, as specified in [\[XMLDSig\]](#). The signature MUST be performed over exclusively canonicalized XML, as specified in [\[Excl-C14N\]](#). All transforms performed on signed elements MUST be included in the Transforms element, as specified in [\[XMLDSig\]](#) section 4.3.3.4. The signature MUST be produced using the **requestor IP/STS** private key. The KeyInfo element, as specified in [\[XMLDSig\]](#) section 4.4, MUST either directly include an X.509 V.3 certificate (as specified in [\[X509\]](#)) or reference an X.509 V.3 certificate using the X.509 V.3 **subject key identifier (SKI)**, as specified in [\[RFC3280\]](#) section 4.2.1.2. For further specifications, see [\[XMLDSig\]](#) section 4.4.4. It is recommended [<30>](#) that the certificate be included directly in the KeyInfo element, using the X509Certificate element.

The X509SKI element contains the base64-encoded ([\[RFC4648\]](#) section 4) plain (that is, non-DER-encoded) value of an X.509 V.3 SKI extension. If the **SubjectKeyIdentifier** field is not present in the certificate, the certificate itself MUST be included directly in KeyInfo. Examples of these fields are found in sections [4.2.3](#) and [4.2.4](#).

Note that the message format of the security token does not incorporate encryption beyond the encryption provided by SSL/TLS.

2.2.5 wsignout1.0 Request Message

The wsignout1.0 request message is specified in [\[WSFederation1.2\]](#) section 13.2.4.1 and is a request to a **requestor IP/STS** to delete the cached session state for a specific **user**. The protocol does not specify a response and does not guarantee the operation will complete. The wsignout1.0 request message consists of an HTTP GET with the following query string parameters, as specified in [\[WSFederation1.2\]](#) section 13.2.4.1:

- **wa**: The value MUST be the literal string "wsignout1.0".
- **wreply** (optional): The value is a URL at the **relying party** to which the **web browser requestor** SHOULD [<31>](#) be redirected once **sign-out** processing is complete.

2.2.6 wsignoutcleanup1.0 Request Message

The wsignoutcleanup1.0 request message is specified in [\[WSFederation1.2\]](#) section 13.2.4.2 and is a request to a **relying party** to delete the cached session state for a specific **user**. The wsignoutcleanup1.0 request message consists of an HTTP GET with the following query string parameters, as specified in [\[WSFederation1.2\]](#) section 13.2.4.2 (for processing semantics of wsignoutcleanup1.0, see section [3.3.5.4.5](#)):

- **wa**: The value MUST be the literal string "wsignoutcleanup1.0".
- **wreply** (optional): This value is a URL at the **requestor IP/STS** to which the **web browser requestor** SHOULD [<32>](#) be redirected once clean-up processing is complete.

The wsignoutcleanup1.0 message is an instruction to relying parties by a requestor IP/STS to delete the cached session state for the specified user. The protocol does not specify a response back to the requestor IP/STS initiating the wsignoutcleanup1.0 message.

2.3 Directory Service Schema Elements

This protocol accesses the following Directory Service schema classes and attributes listed in the following table.

For the syntactic specifications of the following **<Class>** or **<Class><Attribute>** pairs, refer [\[MS-ADTS\]](#), [\[MS-ADA1\]](#), [\[MS-ADA3\]](#).

Class	Attribute
User	All

3 Protocol Details

This section addresses the message processing model for the protocol. It includes related information required by an implementation to successfully emit or consume protocol messages, such as an abstract data model for maintaining configuration or state information.

The protocol inherently defines three distinct classes of functionality (or roles) for the entities that emit, transport, and consume protocol messages. The **requestor IP/STS**, **relying party**, and **web browser requestor** roles are described in separate subsections. To improve readability, a fourth section is included at the beginning that describes common details for the requestor IP/STS and relying party roles.

3.1 Common Details for Requestor IP/STS and Relying Party Roles

The **requestor IP/STS** and **relying party** roles share some common message processing behavior, and they need the same kind of configuration data. This section describes that shared information and common processing semantics. As specified in section 3.3, it is possible to factor a relying party into separate components, which are defined in the [glossary \(section 1.1\)](#) as a **resource IP/STS** and a **WS resource**. Throughout this section, the generic term relying party is used unless it is necessary to distinguish that a topic applies only to a resource IP/STS or only to a WS resource.

3.1.1 Abstract Data Model

Proper operation of the protocol requires that a **requestor IP/STS** and a **relying party** maintain configuration information that describes the entities with which they exchange protocol messages. This section describes an abstract data model for maintaining that configuration information.

The following subsections describe a conceptual model of possible data organization that an implementation maintains to participate in this protocol. The described organization is provided to help explain how the protocol behaves. This specification does not mandate that implementations adhere to this model as long as their external behavior is consistent with that described in this document.

Note The conceptual data model can be implemented using a variety of techniques. Windows behavior is described for each data item at the end of the appropriate subsection.

3.1.1.1 Security Token

The primary protocol data unit transported by this protocol is a **security token** constructed as a subset of the SAML 1.1 assertion element syntax, as specified in section 2.2.4.2. Because this is the only security token format supported by this protocol, no abstract data model is introduced to represent a security token. Throughout section 3, wherever it is necessary to discuss internal constructs of a security token, the SAML terminology from section 2.2.4.2 will be used. For further specification, see [\[SAMLCore\]](#).

If a security token returned by this protocol is not formatted as required in section 2.2.4.2, the **relying party** MUST return an HTTP 1.1 status code 500 server error to the **web browser requestor**.

3.1.1.2 User Authentication Context

When a **requestor IP/STS** or a **resource IP/STS** issues a **security token**, it MUST authenticate the **user** to obtain the data required to construct a security token. How the user is authenticated is implementation-specific and not addressed in this protocol. Common practices are discussed in sections 3.2 and 3.3 under the topic of message processing. In addition, when a **relying party** accepts a security token to authenticate a user, it is necessary to map the security token AuthenticationStatement and AttributeStatement data into the structure that is used by the local

system to make access control decisions. This is implementation-specific and not addressed in this protocol. It is useful here to define an abstract data model to record the data returned from the user authentication process. The following is a potential representation to organize this data: [<33>](#33)

Authentication Context: This record contains data returned from user authentication. An **IP/STS** MUST maintain a separate record per user. The fields of this record are as follows:

AuthIdentity: A string field that uniquely and authoritatively identifies the user.

AuthMethod: A string field that identifies the authentication mechanism used, as required for the SAML assertion in a security token, as described in section [2.2.4.2](#2.2.4.2).

AuthTime: A date-time field that identifies the most recent time that the user was authenticated.

AuthStart: A date-time field used to hold the beginning of the validity interval for a security token that is specified by the **NotBefore** attribute, as discussed in section [2.2.4.2](#2.2.4.2).

AuthStop: A date-time field used to hold the end of the validity interval for a security token that is specified by the **NotOnOrAfter** attribute, as discussed in section [2.2.4.2](#2.2.4.2).

AuthGroups (optional): A string field used to contain a list of group names, if any exist for this user, that are returned by the **AuthMethod**.

AuthClaim (optional): This record holds a **claim** from a security token, as defined in section [3.1.1.4](#3.1.1.4). There MUST be one claim per **AuthClaim** record; thus multiple records can be present.

3.1.1.3 Federation Partner

A **federation** partner is a generic term used to refer to a **security realm** that operates either a **requestor IP/STS** or a **relying party** service (or both), capable of performing the mandatory operations defined in this specification.

A requestor IP/STS and a relying party MUST exchange configuration metadata before they begin exchanging protocol messages. How this metadata is exchanged is implementation-specific and not addressed in this protocol. Administrators MAY [<34>](#34) exchange files by an out-of-band process (such as email attachments) and enter the partner's metadata into a local configuration file or database.

A requestor IP/STS and a relying party MUST exchange at least the following metadata before they begin exchanging protocol messages:

- Unique identifiers that indicate the security realms that they represent, and distinguish them from other possible federation partners.
- URLs that indicate where protocol messages are to be sent.

Also, the relying party MUST obtain the certificate specified in [\[X509\]](#X509) that contains the public key that corresponds to the private key that the requestor IP/STS uses for signing **security tokens**.

The following is a potential representation for organizing this data. The data is organized as a series of records, each representing a federation partner. The fields of this record are as follows:

- **Identifier:** A string field that uniquely identifies the security realm of the partner. It must be the value that is used for the wtrealm parameter for [wsignin1.0](#wsignin1.0) request messages. For details, see section [2.2.3](#2.2.3).
- **Role:** A string field that identifies the role (or roles) played by the partner, and thus the types of protocol messages that it can send or receive. There are two possible values for this field, requestor IP/STS and relying party. This field must contain at least one value. If a federation partner is capable of playing both roles, this can be represented by a single record with both

requestor IP/STS and relying party values present, or it MAY be represented by two separate records with different values for the **Role** field.<35>

- **URL**: A field indicating the URL to which all protocol messages that are intended for the federation partner identified by this record must be sent.
- **Certificate** (optional): A field indicating an X.509 certificate that holds the public key that corresponds to the private key used by the partner for signing security tokens. If the **Role** field contains requestor IP/STS, a certificate must be present. Otherwise this field does not apply. Note that the field can contain multiple certificates. If the requestor IP/STS is a farm of servers, all of the servers can use the same private key, or each server can use a different private key.<36>

A security token, as described in section 2.2.4.2, MUST contain an AuthenticationStatement that specifies the identity of the **user** in the Subject element. The wsignin1.0 request message does not contain a provision to specify additional **claims** that the relying party requires to determine if a user is authorized to access a particular **WS resource**. If specific claims are to be included in a security token, federation partners MUST agree on them before protocol messages are exchanged. Specific claims can be requested as part of the metadata exchange and included in the federation partner record.

The following is a potential representation for organizing this data. Two fields are added to the abstract data model of a federation partner record. For further specifications on the AttributeStatement element of a security token mentioned in the following data definitions, see section 2.2.4.2:

- **ClaimsOut** (optional): A list of claims, as defined in section 3.1.1.4, that SHOULD be included in an AttributeStatement of a security token sent in a wsignin1.0 response message to the federation partner. This SHOULD match the corresponding **ClaimsIn** record at the federation partner.<37>
- **ClaimsIn** (optional): A list of claims, as specified in section 3.1.1.4, that MAY be included in the AttributeStatement of a security token received in a wsignin1.0 response message from the federation partner. This SHOULD match the corresponding **ClaimsOut** record at the federation partner.<38>

This protocol does not address whether these additional claims are treated as optional or mandatory by the requestor IP/STS. There is no guarantee that a particular user will have the attribute data necessary to satisfy every claim on the **ClaimsOut** list for every relying party that requests a security token for that user. Whether a requestor IP/STS fails a wsignin1.0 request message if it cannot set every claim is an implementation-specific detail that is not addressed in this protocol. It is recommended that a security token be issued and the relying party be allowed to decide if it has sufficient information about the user to grant access to the protected resource in question.<39>

3.1.1.4 Claim

A **security token** MAY<40> contain an AttributeStatement, with one or more Attribute elements, each of which contains a single **claim**, as specified in section 2.2.4.2. A claim is uniquely identified by its **AttributeName** attribute and <AttributeValue> element.

This protocol restricts the syntax and the interpretation of the semantics of these five claims to the following definitions. See section 2.2.4.2 for further specification on the AttributeStatement element of a security token and the usage of **AttributeName**, **AttributeNamespace**, and <AttributeValue> in the following claim definitions:

EmailAddress claim (optional): This claim is used to identify a Subject via an email address.

- The **AttributeName** attribute MUST be "EmailAddress".
- The **AttributeNamespace** attribute MUST be the URL <http://schemas.xmlsoap.org/claims>.

- The <AttributeValue> element content MUST conform to "addr-spec", as specified in [\[RFC2822\]](#). The value MUST be unique within the **security realm** of the **requestor IP/STS** that issued the security token such that a **relying party** could use it to make an access control decision.
- When this claim is used for the value of the Subject/NameIdentifier element of an AuthenticationStatement or AttributeStatement, the value of the **Format** attribute MUST be URI urn:oasis:names:tc:SAML:1.1:nameid-format:emailAddress.

UPN claim (optional): This claim is used to identify a Subject via a UPN.

- The **AttributeName** attribute MUST be "UPN".
- The **AttributeNamespace** attribute MUST be URL http://schemas.xmlsoap.org/claims.
- The <AttributeValue> element content MUST be a UPN. The value MUST be unique within the security realm of the requestor IP/STS that issued the security token such that a relying party could use it to make an access control decision.
- When this claim is used for the value of the Subject/NameIdentifier element of an AuthenticationStatement or AttributeStatement, the value of the Format attribute MUST be the URL http://schemas.xmlsoap.org/claims/UPN.

CommonName claim (optional): This claim is used to identify a Subject via a common name (CN) value consistent with X.500 naming conventions.

- The **AttributeName** attribute MUST be "CommonName".
- The **AttributeNamespace** attribute MUST be the URL http://schemas.xmlsoap.org/claims.
- The <AttributeValue> element content MUST conform to CommonName, as specified in [\[X500\]](#). The value of this claim is not necessarily unique and MUST NOT be used by a relying party to make an access control decision. It is suitable for displaying a friendly name for personalization.
- When this claim is used for the value of the Subject/NameIdentifier element of an AuthenticationStatement or AttributeStatement, the value of the Format attribute MUST be the URL http://schemas.xmlsoap.org/claims/CommonName.

Group claim (optional): This claim is used to indicate the association of the subject with other **users** that share a common characteristic. The semantic meaning of that association is application specific, but the common interpretation is group or role membership.

- The **AttributeName** attribute MUST be "Group".
- The **AttributeNamespace** attribute MUST be the URL http://schemas.xmlsoap.org/claims.
- The <AttributeValue> element content MUST be a string.

For more information about Group claims, see Appendix A: Windows Behavior. [<41>](#)

Implementations MAY define additional claims with prior agreement between **federation** partners and MUST conform to the following structure: [<42>](#)

Custom claim (optional): This claim is used to identify an application-specific attribute possessed by the subject.

- The **AttributeName** attribute MUST be a string constant, per agreement between federation partners.
- The **AttributeNamespace** attribute SHOULD be the URL http://schemas.xmlsoap.org/claims.

- The <AttributeValue> element content MAY be an arbitrary data type per agreement between federation partners.

3.1.1.5 Federation Partner Session Lists for Web Browser Requestors

The protocol [wsignout1.0](#) request message and [wsignoutcleanup1.0](#) messages do not explicitly identify the **user** who has triggered the **sign-out** operation. The **web browser requestor** that transports these messages is the only link to the user. Thus, to process the [wsignout1.0](#) request message and the [wsignoutcleanup1.0](#) request message, a user's activity MUST be tracked in terms of the activity of the user's web browser requestor. **Federation** partners MUST uniquely identify individual web browser requestors. This MAY [\[43\]](#) be done by setting an HTTP session cookie that contains a unique identifier. For more information, see [\[RFC2965\]](#).

web browser requestor session: This is a list of **security tokens** that was issued to (or received from) a specific instance of a web browser requestor in response to Microsoft Web Browser Federated Sign-On Protocol messages. A web browser requestor session is delimited by the user starting and stopping an instance of the software that implements a web browser requestor. For example, if a user was to start two instances of the same web browser requestor software in parallel to obtain security tokens, this would be treated as two sessions. A **requestor IP/STS** MUST maintain the list in terms of **relying parties** using the value of the Audience element from each security token issued during the web browser requestor session. A relying party MUST maintain the list in terms of requestor IP/STSs using the value of the Issuer element from each security token received during the web browser requestor session.

3.1.1.5.1 Requestor IP/STS Web Browser Requestor Sessions List

The following is a potential representation for a **requestor IP/STS** to organize the data for tracking **web browser requestor** sessions to support processing of a [wsignout1.0](#) request and a [wsignoutcleanup1.0](#) request message. The data is organized as a list of records, each representing a particular web browser requestor session.

The following is a potential representation for organizing the data record that represents a particular web browser requestor session on the list.

Outbound Sessions List: This data element marks the beginning of the list of web browser requestor sessions.

WebBrowserRequestorSession: This record holds the list of **security tokens** issued for a particular web browser requestor session. The fields of this record are as follows: [\[44\]](#)

- **Requestor Session Identifier:** This uniquely identifies a particular web browser requestor session. A WebBrowserRequestorSession record is added to the Outbound Sessions List when the first security token is issued for a particular web browser requestor session.
- **Session Entry:** This identifies a **relying party** based on the Audience element content from a security token. A Session Entry is added to the record when a security token is issued for the web browser requestor session.

3.1.1.5.2 Relying Party Web Browser Requestor Sessions List

The following is a potential representation for a **relying party** to organize the data for tracking **web browser requestor** sessions to support processing of [wsignout1.0](#) and [wsignoutcleanup1.0](#) messages. The data is organized as a list of records, each representing a particular web browser requestor session.

The following is a potential representation for organizing the data record that represents a particular web browser requestor session on the list.

Inbound Sessions List: This data element marks the beginning of the list of web browser requestor sessions.

WebBrowserRequestorSession: This record holds the list of **security tokens** received for a particular web browser requestor session. The fields of this record are as follows: [<45>](#)

- **Requestor Session Identifier:** This uniquely identifies a particular web browser requestor session. A WebBrowserRequestorSession record is added to the Inbound Sessions List when the first security token is received for a particular web browser requestor session.
- **Session Entry:** This identifies a **requestor IP/STS** based on the Issuer attribute value from a security token. A Session Entry is added to the record when a security token is received for the web browser requestor session.

3.1.2 Timers

This protocol does not require timers beyond those that might be used by the underlying transport to transmit and receive messages over HTTP and SSL/TLS.

The **security tokens** transported in protocol messages have a specific time interval during which they are considered to be valid. This MUST be set by the **security token service** that issues the security tokens using the **NotBefore** and **NotOnOrAfter** attributes of the Conditions element. For further specifications, see [\[SAMLCore\]](#) section 2.3.2.1.1. A **relying party** SHOULD NOT accept security tokens if the current time is equal to or greater than the value of the **NotOnOrAfter** attribute or equal to or less than the value of the **NotBefore** attribute.

When an Authentication Context is created from security tokens, the **AuthStart** and **AuthStop** fields MUST be set from the **NotBefore** and **NotOnOrAfter** attributes of the security tokens. A **resource IP/STS** SHOULD NOT use an Authentication Context to grant access to a **WS resource** if the current time falls outside the validity interval defined by the **AuthStart** and **AuthStop** values. [<46>](#)

Implementations MAY [<47>](#) use a timer to indicate when the validity interval of a security token or an Authentication Context expires to control maintenance operations (for example, flushing caches), but the protocol does not require the use of a timer.

3.1.3 Initialization

Requestor IP/STSS and **relying parties** MUST exchange metadata and initialize **federation** partner records in local configuration data, as specified in section [3.1.1.3](#). There is no protocol-specific initialization required for a **web browser requestor**.

3.1.4 Higher-Layer Triggered Events

Administrative actions starting or stopping the **requestor IP/STS** or **relying party** services, or changing service configuration data do not trigger protocol message exchanges. Protocol request messages are triggered by **user** action via a **web browser requestor**.

The [wsignin1.0](#) request message is triggered when a web browser requestor sends an HTTP GET to a **WS resource** at a WS resource that requires users to be authenticated using this protocol.

The [wsignout1.0](#) request message is triggered when a user explicitly requests to initiate the **sign-out** operation at a WS resource.

The [wsignoutcleanup1.0](#) request message is triggered when a user explicitly requests to initiate the sign-out operation at a requestor IP/STS, or in response to the [wsignout1.0](#) message discussed earlier.

3.1.5 Processing Events and Sequencing Rules

This section describes message processing functionality that is common to **federation** partners, regardless of the specific role they play.

3.1.5.1 Determining Message Type

A **requestor IP/STS** and a **relying party** process several different types of messages. Upon receipt of a message, they determine its type by inspection of the *wa* parameter. The *wa* parameter MUST be included in every message used by this protocol (for further specification, see section 2.2). The *wa* parameter, and all other protocol-specific parameters, MUST be transported in query string parameters when the protocol message is transported using an HTTP GET. These parameters MUST be transported in the body of the HTTP POST when the protocol message is transported using an HTTP POST. If protocol-specific parameters are included in query string parameters with an HTTP POST, they SHOULD [<48>](#) be ignored.

A **web browser requestor** is not required to understand the types or content of the protocol messages it transports between a requestor IP/STS and a relying party. [<49>](#)

3.1.5.2 Error Handling

If a protocol message is received that includes an optional parameter that is not supported by an implementation, or a parameter that is not specified by this protocol, a **requestor IP/STS** MAY [<50>](#) ignore the parameter and process the message as though it were not present.

Protocol messages are processed by a requestor IP/STS or a **relying party**. If a message cannot be processed successfully, the protocol allows for a SOAP fault to be returned to the appropriate **federation** partner via a **web browser requestor** using an HTTP POST method.

However, this protocol does not require end-to-end error message propagation. A requestor IP/STS or a relying party MAY return an HTTP 500 error response to the web browser requestor, or they MAY redirect it to an error page. [<51>](#)

This protocol does not include provisions for automated retry or recovery processing if there is no response to a protocol request message. This protocol is designed to rely on the intervention of the **user** to notice when a web browser requestor operation has failed and to manually retry the operation that triggered the protocol.

3.1.5.3 Requesting a Security Token by Issuing a wsignin1.0 Request Message

This section describes the generic message processing performed by a **relying party** when requesting a **security token**. The purpose is to authenticate a **user** by requesting a security token to be issued for a specific user with the relying party as the audience.

The generic processing model for sending a [wsignin1.0](#) message MUST consist of the following steps, as described in the following subsections:

- [Protocol Activation \(section 3.1.5.3.1\)](#)
- [Parameter Marshaling \(section 3.1.5.3.2\)](#)
- [Requestor IP/STS Security Realm Discovery \(section 3.1.5.3.3\)](#)
- [Message Transmission \(section 3.1.5.3.4\)](#)

3.1.5.3.1 Protocol Activation

For a **relying party**, the protocol is triggered when a **web browser requestor** attempts to access a **WS resource** that requires **users** to be authenticated, and an Authentication Context does not exist for the users.

3.1.5.3.2 Parameter Marshaling

The **relying party** MUST set all the required query string parameters for a [wsignin1.0](#) request message, as specified in section 2.2.3. It MUST set the *wrealm* value from the configuration data. It SHOULD [<52>](#) store the original application URL provided by **WS resource** in *wctx*.

3.1.5.3.3 Requestor IP/STS Security Realm Discovery

Because the **user** has not yet been authenticated, the **relying party** might not know where to send a [wsignin1.0](#) request message. The relying party MUST obtain the **security realm** identifier for the **requestor IP/STS** that issues **security tokens** for the user. When the relying party is factored into **WS resource** and **resource IP/STS** components, the WS resource relies on the resource IP/STS to discover the correct **federation** partner. The resource IP/STS MAY discover the security realm of the federation partner by interacting with the user via the user's **web browser requestor**. Or the information MAY be obtained or derived from a parameter (such as *whr*, *domain_hint*, *login_hint*, or *username*, specified in section 2.2.3) that was included on the original request to WS resource. [<53>](#)

The resource IP/STS MUST use the security realm identifier to look up the correct federation partner record in configuration data and obtain the requestor IP/STS URL for protocol messages. Once obtained, the security realm identifier MAY [<54>](#) be preserved for subsequent sessions by writing an HTTP persistent cookie (for more information, see [\[RFC2965\]](#)) to the web browser requestor.

3.1.5.3.4 Message Transmission

The **relying party** sends a [wsignin1.0](#) request message by returning an HTTP 302 response to the **web browser requestor** with the Location field set to the URL of the **requestor IP/STS**.

All the query string parameters required for the protocol MUST be set properly, as specified in section 2.2.3.

The **ad_fs_behavior_level** abstract data model (ADM) element is defined in [\[MS-OAPX\]](#) section 3.2.1.1 and is hereafter referred to simply as the **AD FS behavior level**. [\[MS-OAPX\]](#) section 3.2.1.1 also includes information about how the AD FS behavior level relates to product versions. The following are recommended best practices related to the AD FS behavior level:

- Upon forwarding the [wsignin1.0](#) request, the **resource IP/STS** SHOULD use only the parameters that are supported by the requestor IP/STS AD FS behavior level. The resource IP/STS can track the requestor IP/STS AD FS behavior level and choose the forwarding behavior accordingly. Behavior-level tracking is implementation specific.
- If a resource IP/STS that supports the *prompt* parameter receives the *prompt* parameter and knows that the requestor IP/STS does not support the *prompt* parameter, the resource IP/STS SHOULD send a [wsignin1.0](#) request using the protocol-specific parameters (for example, *wfresh* and *wauth*) to facilitate a fresh and interactive authentication.

Note Support for the *prompt* parameter depends on the AD FS behavior level and the product version. See section 2.2.3 for support information. If the parameter is not supported by the AD FS server, it is ignored.

3.1.5.4 Issuing a Security Token by Responding to a [wsignin1.0](#) Request Message

This section describes the generic message processing performed by either a **requestor IP/STS** or a **resource IP/STS** (a component of a **relying party** specified in section [3.3](#)) when issuing a **security**

token. **IP/STS** is used generically to see either a requestor IP/STS or a resource IP/STS in the following subsections.

The generic processing model for responding to a [wsignin1.0](#) message is specified to consist of the following steps, as described in the following subsections:

- [Protocol Activation \(section 3.1.5.4.1\)](#)
- [Message Validation \(section 3.1.5.4.2\)](#)
- [User Identification and Authentication \(section 3.1.5.4.3\)](#)
- [User Attribute Retrieval \(section 3.1.5.4.4\)](#)
- [Claim Mapping \(section 3.1.5.4.5\)](#)
- [SAML Assertion Construction \(section 3.1.5.4.6\)](#)
- [Response Message Processing \(section 3.1.5.4.7\)](#)

3.1.5.4.1 Protocol Activation

The protocol is triggered by receipt of a [wsignin1.0](#) request message. The syntax MUST be validated by ensuring that all required parameters are present and contain the correct type of data. For further specifications, see section 2.2.3.

3.1.5.4.2 Message Validation

The syntax MUST be validated by ensuring that all required parameters are present and contain the correct type of data. For further specifications, see section [2.2.3](#).

Before issuing a **security token** to protect the **user's** privacy, the **IP/STS** MUST verify that the entity that sent the [wsignin1.0](#) request message is a federated partner that holds the role of **relying party**, as described in the [Abstract Data Model](#). The relying party's identifier MUST be retrieved from the *wtrealm* parameter in the request (as specified in section [2.2.1](#)) and compared against the **federation** partner configuration data (as specified in section [3.1.1.2](#)). [<55>](#)

The *wauth* parameter, which is described in section 2.2.3, MAY [<56>](#) be used.

3.1.5.4.3 User Identification and Authentication

The **user's** identity is not conveyed explicitly in the [wsignin1.0](#) request message. The **IP/STS** MUST establish this by initiating a message exchange with the **web browser requestor** that will cause the users to identify themselves and prove their right to assert that identity.

The user authentication methods are implementation-specific and are not addressed in this protocol. It is recommended that the IP/STS employ a standard protocol to authenticate the user, such as Kerberos (for more information, see [\[RFC4120\]](#)). It MAY use an HTML form to collect credentials directly from the user (ideally using HTTPS) and compare them against a local database. Or it MAY authenticate the user by initiating the Microsoft Web Browser Federated Sign-On Protocol with another **STS**. [<57>](#)

Whatever method is used, the IP/STS MUST store the results in the Authentication Context, as defined in section [3.1.1.2](#).

If the user cannot successfully authenticate, the IP/STS MUST abort processing the request and return an error message to the user.

3.1.5.4.4 User Attribute Retrieval

If additional **claims** are required by a ClaimsOut entry for the **relying party**, as specified in section [3.1.1.2](#), the **IP/STS** MUST retrieve the correct values that correspond to the authenticated identity of the **user**. How this is performed is implementation-specific and not addressed in this protocol. The IP/STS SHOULD [<58>](#) retrieve the data from an authoritative user attribute authority based on the value of the AuthIdentity element from the user's Authentication Context.

3.1.5.4.5 Claim Mapping

If additional **claims** are required by a ClaimsOut entry for the **relying party**, the **IP/STS** MUST map **user** attributes retrieved in the previous step to their corresponding claims.

3.1.5.4.6 SAML Assertion Construction

Having assembled the required set of **claims**, the **IP/STS** constructs a **security token** according to the SAML 1.1 assertion syntax specified in section [2.2.4.2](#). The **relying party's** identity MUST be used to populate the Audience element of the required AuthenticationStatement. From the Authentication Context, AuthIdentity, AuthMethod, and AuthTime MUST be used to populate the Subject element, AuthenticationMethod attribute, and AuthenticationInstant, respectively, of the AuthenticationStatement. Additional claims required by the relying party MUST be placed in separate Attribute elements in the AttributeStatement.

As specified in section [2.2.4.1](#), the security token MUST be encoded as a wst:RequestSecurityTokenResponse and returned in the *wresult* parameter of the [wsignin1.0](#) response message. The IP/STS MAY encrypt the security token as described in section 2.2.4.1.

3.1.5.4.7 Response Message Processing

All required parameters for a [wsignin1.0](#) response message MUST be set correctly. The completed response message MUST be returned in an HTTP POST, as specified in section [2.1](#), to the URL designated for the **relying party** that sent the request. [<59>](#)

3.1.6 Timer Events

There are no protocol-specific timer events that MUST be serviced by an implementation. This protocol does not require timers beyond those that might be used by the underlying transport to transmit and receive messages over HTTP and SSL/TLS. The protocol does not include provisions for time-based retry for sending protocol messages.

Security tokens and Authentication Contexts do have validity intervals, as specified in section [3.1.2](#). Implementations MAY [<60>](#) use a timer to indicate when the validity interval of a security token or an Authentication Context expires, but the protocol does not require the use of a timer.

3.1.7 Other Local Events

This protocol does not have dependencies on other protocols other than HTTP 1.1 and SSL/TLS to transport protocol messages. This protocol relies on this transport mechanism for the correct and timely delivery of protocol messages. The protocol does not take action in response to any changes or failure in machine state or network communications.

3.2 Requestor IP/STS Details

This section describes details of protocol processing that must be understood, in addition to the information in section [3.1](#), to implement a **requestor IP/STS** that can correctly perform its role in the protocol message exchange.

3.2.1 Abstract Data Model

A **requestor IP/STS** does not require additions to the [abstract data model \(section 3.1.1\)](#).

3.2.2 Timers

A **requestor IP/STS** does not depend on timers beyond those that might be used by the underlying transport to transmit and receive messages over HTTP and SSL/TLS or those specified in section [3.1.2](#).

3.2.3 Initialization

Before any protocol messages can be exchanged, a **requestor IP/STS** MUST exchange metadata with **relying parties** and initialize **federation** partner records for them in local configuration data, as specified in section [3.1.1.2](#).

To service protocol messages, a requestor IP/STS MUST be listening for requests at the URL it has advertised to federation partners.

To service [wsignin1.0](#) request messages, a requestor IP/STS MUST be able to contact **user** authentication and account services in its local **security realm** to obtain the authenticated user identity and optional attributes necessary for constructing a **security token**.

The protocol does not require specific initialization on receipt of a protocol message.

3.2.4 Higher-Layer Triggered Events

In addition to user-triggered events (see section [3.1.4](#)), a **requestor IP/STS** is triggered on receipt of a protocol message to process that message and respond to the **federation** partner that sent it (see section [3.2.5](#)).

3.2.5 Processing Events and Sequencing Rules

This section describes the logical steps performed by a **requestor IP/STS** when processing the following protocol messages:

- wsignin1.0
- wsignout1.0
- wsignoutcleanup1.0

3.2.5.1 Issuing a Security Token by Responding to a wsignin1.0 Request Message

This message is received by a **requestor IP/STS** from a **relying party**. The generic model for responding to a [wsignin1.0](#) message is specified to consist of the following steps (from section [3.1.5.4](#)):

- [Protocol Activation \(section 3.1.5.4.1\)](#)
- [Message Validation \(section 3.1.5.4.2\)](#)
- [User Identification and Authentication \(section 3.1.5.4.3\)](#)
- [User Attribute Retrieval \(section 3.1.5.4.4\)](#)
- [Claim Mapping \(section 3.1.5.4.5\)](#)

- [SAML Assertion Construction \(section 3.1.5.4.6\)](#)
- [Response Message Processing \(section 3.1.5.4.7\)](#)

A requestor IP/STS MUST process a wsignin1.0 request message, as described in the preceding sections, with the following exception:

- User identification and authentication

It is not possible for a requestor IP/STS to authenticate the **user** by using this protocol with an **IP/STS** in another **security realm** because the user account is managed in the security realm of the requestor IP/STS.

3.2.5.2 Inbound wsignout1.0 Request Message Processing

This message is used to request that a **requestor IP/STS** initiate clean-up operations for cached session state, if any exists, for the **user** who triggered the request.

3.2.5.2.1 Protocol Activation

The protocol is triggered by receipt of a [wsignout1.0](#) request message. The request MUST be validated by ensuring that all required parameters are present and contain the correct type of data (as specified in section [2.2.3](#)).

3.2.5.2.2 Clean-Up Processing

A **requestor IP/STS** SHOULD delete any session state that has been locally cached for the **web browser requestor** that delivered the [wsignout1.0](#) request message. A requestor IP/STS SHOULD send a [wsignoutcleanup1.0](#) message to each **relying party** to which a **security token** has been issued for the web browser requestor that delivered the wsignout1.0 request message, as specified in section [3.2.5.3.<61>](#)

3.2.5.2.3 Response Message Processing

If a *wreply* parameter was included in the request, as specified in section [2.2.5](#), the **requestor IP/STS** SHOULD redirect the **web browser requestor** to the specified URL when it has completed clean-up operations.[<62>](#)

3.2.5.3 Outbound wsignoutcleanup1.0 Request Message Processing

This message is used to request that a **relying party** initiate clean-up operations for cached session state, if any exists, for the **user** who triggered the request.

3.2.5.3.1 Protocol Activation

The protocol is triggered either when a **user** requests that the user's session be terminated, possibly by clicking a **sign-out** button, or when a [wsignout1.0](#) request message is received from a **relying party**.

3.2.5.3.2 Relying Party Security Realm Discovery

As specified in section [3.1.1.5.1](#), a **requestor IP/STS** MAY store a session identifier for the **web browser requestor** in an HTTP session cookie (for more information, see [\[RFC2965\]](#)). It uses this identifier and its Outbound Sessions List to develop the list of **relying parties** to which it SHOULD [<63>](#) send [wsignoutcleanup1.0](#) request messages.

3.2.5.3.3 Clean-Up Processing

Because the protocol does not guarantee a response to a [wsignoutcleanup1.0](#) message, a **requestor IP/STS** SHOULD clean up the locally cached session state that it maintains before sending [wsignoutcleanup1.0](#) messages. For example, as specified in section [3.1.1.5.1](#), the `WebBrowserRequestorSession` record SHOULD be deleted from the Outbound Sessions List. As each [wsignoutcleanup1.0](#) message is sent, the requestor IP/STS SHOULD delete the corresponding Session Entry from the record. When the last Session Entry is deleted, the `WebBrowserRequestorSession` SHOULD be deleted from the Outbound Sessions List.[<64>](#)

3.2.5.3.4 Message Transmission

The **requestor IP/STS** SHOULD send a [wsignoutcleanup1.0](#) request message to each **relying party** using an explicit HTTP GET method because the protocol does not support chaining [wsignoutcleanup1.0](#) messages using the HTTP 1.1 redirection facilities. How these messages are sent is implementation-specific and not addressed in this protocol. The requestor IP/STS MAY walk the list of relying parties and issue the requests individually.[<65>](#)

3.2.6 Timer Events

A **requestor IP/STS** does not need to interact with any timers, or service any timer events, beyond those that might be used by the underlying transport to transmit and receive messages over HTTP and SSL/TLS, or those specified in section [3.1.6](#).[<66>](#)

3.2.7 Other Local Events

A **requestor IP/STS** does not have dependencies on local events beyond what is specified in section [3.1.7](#).

3.3 Relying Party Details

This section describes details of protocol processing that must be understood, in addition to the information from section [3.1](#), to implement a **relying party** that can correctly perform its role in the protocol message exchange.

3.3.1 Abstract Data Model

A **relying party** performs two distinct functions. It processes protocol messages and the **security tokens** it receives, and it controls **user** access to protected resources based on the contents of those security tokens. The latter function is implementation-specific and not addressed in this protocol. A relying party MAY[<67>](#) be factored into separate components, a **resource IP/STS**, and a **WS resource** as follows. The resource IP/STS component MUST be able to perform all of the metadata and protocol message exchanges required to obtain a security token from a **requestor IP/STS** in another **security realm**. The WS resource component SHOULD use the Authentication Context derived from that security token to control user access.

The abstract data model for a relying party MAY[<68>](#) be extended to enable this componentization without requiring a requestor IP/STS to have interior knowledge of the relying party structure. This implementation-approach is supported by the following extensions of the abstract data model for a **federation** partner, as specified in section [3.1.1.3](#).

3.3.1.1 Resource IP/STS Abstract Data Model Extensions

The following is a potential representation for a **resource IP/STS** to organize the data that represents its **federation** partners. The federation partner record is used, as specified in section [3.1.1.3](#), with the following extensions and dependencies for the possible range of values for fields:

Identifier: For a **requestor IP/STS** from another **security realm**, it MUST be the *wtrealm* value, as specified in section 3.1.1.3. For a **WS resource**, it MUST be an identifier that is unique within the security realm such as a web server or web application URL or URI.

Role: There are three possible values for this field: requestor IP/STS, **relying party**, and WS resource. For a partner from another security realm, this field MAY contain requestor IP/STS or relying party or both values, as specified in section 3.1.1.3. For a partner within the security realm, this field SHOULD only contain WS resource. [<69>](#)

URL: If the **Role** field contains requestor IP/STS or relying party, this field MUST contain a URL. If the **Role** field contains WS resource, a URL MAY be present but is not required because this information can be reliably passed in using the *wreply* parameter. [<70>](#)

Certificate (optional): If the **Role** field contains requestor IP/STS, the **Certificate** field MUST contain a certificate, as specified in section 3.1.1.3. Otherwise, this field does not apply.

3.3.1.2 WS Resource Abstract Data Model Extensions

Following is a potential representation for a **WS resource** to organize the data that represents its **federation** partners. The federation partner record is used, as specified in section [3.1.1.3](#), with the following restrictions on the range of values for the **Role** field:

- **Role:** A string field that identifies the role (or roles) played by the partner and thus the types of protocol messages that it can send or receive. The only possible value for this field is **resource IP/STS**.

3.3.2 Timers

A **relying party** does not depend on timers beyond those that might be used by the underlying transport to transmit and receive messages over HTTP and SSL/TLS, or those specified in section [3.1.2](#). [<71>](#)

3.3.3 Initialization

Before protocol messages can be exchanged, a **relying party** MUST exchange metadata with **requestor IP/STSs** and initialize **federation** partner records for them in local configuration data, as specified in section [3.1.1.2](#).

To service protocol messages, a relying party MUST be listening for requests at the URL it has advertised to federation partners.

To service [wsignin1.0](#) response messages, a relying party SHOULD [<72>](#) have network access to the certificate revocation list (CRL) distribution point (CDP) contained in X.509 certificates obtained from federation partners for the purpose of validating **security token signatures**, as specified in section 3.1.1.2.

The protocol does not require specific initialization upon receipt of a protocol message.

3.3.4 Higher-Layer Triggered Events

In addition to the user-triggered events discussed in section [3.1.4](#), a **relying party** is triggered upon receipt of a protocol message to process that message and respond to the **federation** partner that sent it, as specified in section [3.3.5](#).

3.3.5 Processing Events and Sequencing Rules

This section describes the logical steps performed by a **relying party** when processing the following protocol messages:

- wsignin1.0
- wsignout1.0
- wsignoutcleanup1.0

Message processing by the relying party is described separately for the **resource IP/STS** and **WS resource** components that were introduced in the revised [abstract data model \(section 3.3.1\)](#).

3.3.5.1 Requesting a Security Token by Sending a wsignin1.0 Request Message

This message is sent by a **relying party** to a **requestor IP/STS** to request that a **security token** be issued for a specific **user** with the relying party as the audience. The generic model for sending a [wsignin1.0](#) message is specified to consist of the following steps in Requesting a Security Token by Issuing a wsignin1.0 Request Message (section [3.1.5.3](#)):

- [Protocol Activation \(section 3.1.5.3.1\)](#)
- [Parameter Marshaling \(section 3.1.5.3.2\)](#)
- [Requestor IP/STS Security Realm Discovery \(section 3.1.5.3.3\)](#)
- [Message Transmission \(section 3.1.5.3.4\)](#)

A relying party MUST process a wsignin1.0 request message (section 2.2.3), as specified in the preceding sections, with the following exceptions:

- [Protocol Activation \(section 3.3.5.1.1\)](#)
- Parameter Marshaling ([Message Validation \(section 3.3.5.2.2\)](#))

3.3.5.1.1 Protocol Activation

When a **relying party** is factored into **resource IP/STS** and **WS resource** components, the protocol is triggered differently for the components. For the WS resource component, the protocol is triggered when a **web browser requestor** attempts to access a WS resource that requires **users** to be authenticated and an Authentication Context does not exist for the user. For the resource IP/STS component, the protocol is triggered by receipt of a user authentication request from the WS resource component. If the components are located on separate servers, the WS resource MUST redirect the web browser requestor to the resource IP/STS to deliver the **security token** request. How user authentication requests are communicated between the components of a relying party is implementation-specific and not addressed in this protocol. [<73>](#)

3.3.5.1.2 Parameter Marshaling

A **resource IP/STS** that sends a [wsignin1.0](#) request message to a **security token service** in a different **security realm** MUST set the *wtrealm* parameter, as specified in [Common Syntax for Request Messages \(section 2.2.1\)](#). A **WS resource** that sends a wsignin1.0 request message to a security token service in the same security realm cannot use the *wtrealm* parameter. It MAY use the *wreply* parameter to distinguish itself from other WS resources in the security realm. [<74>](#)

3.3.5.2 Receiving a Security Token by Processing a wsignin1.0 Response Message

This message is received by a **resource IP/STS** from a **requestor IP/STS**. The purpose is to accept a **security token** that was issued for a specific **user**, with the resource IP/STS as the audience, in response to a [wsignin1.0](#) request message. The generic model for **relying party** processing of a [wsignin1.0](#) response message is specified to consist of the following steps, specified in the following sections:

- [Protocol Activation \(section 3.3.5.2.1\)](#)
- [Message Validation \(section 3.3.5.2.2\)](#)
- [User Identification and Authentication \(section 3.3.5.2.3\)](#)
- [User Attribute Retrieval \(section 3.3.5.2.4\)](#)
- [Claim Mapping \(section 3.3.5.2.5\)](#)
- [Resource Access Control \(section 3.3.5.2.6\)](#)

3.3.5.2.1 Protocol Activation

The protocol is triggered by receipt of a [wsignin1.0](#) response messages.

3.3.5.2.2 Message Validation

The syntax MUST be validated by ensuring that all required parameters are present and contain the correct type of data. For further specifications, see section [wsignin1.0](#) response message. [<75>](#)

3.3.5.2.3 User Identification and Authentication

The **user's** identity and related authentication data from the **requestor IP/STS** are passed in a **security token**. The **resource IP/STS** MUST create an Authentication Context record using the appropriate data from the AuthenticationStatement in the security token.

3.3.5.2.4 User Attribute Retrieval

A **relying party** MAY [<76>](#) maintain local identities for all **users** to control access to **WS resources**. If so, the **requestor IP/STS** (or the WS resource) MUST retrieve the local identity and use it to replace the AuthIdentity in the user's Authentication Context.

3.3.5.2.5 Claim Mapping

If additional **claims** are required by a ClaimsIn entry for the **requestor IP/STS**, the **resource IP/STS** MUST retrieve them from the AttributeStatement in the **security token** and store them in the claims field of the **user's** Authentication Context.

3.3.5.2.6 Resource Access Control

The **user's** Authentication Context MUST be conveyed to the **WS resource** that the user originally tried to access. The location of the WS resource MUST be retrieved from the *wctx* parameter in the [wsignin1.0](#) response message. How this Authentication Context is passed to the WS resource is implementation-specific and not addressed in this protocol. Commonly used techniques when the **relying party** components are located on separate servers include an HTTP cookie, a query string parameter, or a POST body. [<77>](#)

3.3.5.3 Outbound wsignout1.0 Request Message Processing

This message is used to request that a **requestor IP/STS** initiate clean-up operations for cached session state, if any exist, for the **user** who triggered the request.

3.3.5.3.1 Protocol Activation

The protocol is triggered when a **user** requests that the user's session be terminated, possibly by clicking a **Sign-Out** button. The **WS resource** MUST notify the **resource IP/STS** to issue a [wsignout1.0](#) request. If the components are located on separate servers, this SHOULD be performed by sending a wsignout1.0 request message to the resource IP/STS using an HTTP 302 redirect.

3.3.5.3.2 Parameter Marshaling

For more information about parameter marshaling, see Appendix A: Windows Behavior. [<78>](#)

3.3.5.3.3 Requestor IP/STS Security Realm Discovery

As specified in [Relying Party Web Browser Requestor Sessions List \(section 3.1.1.5.2\)](#), **resource IP/STS** MAY store a session identifier for the **web browser requestor** in an HTTP session cookie (for more information, see [\[RFC2965\]](#)). It SHOULD use this identifier and its Inbound Sessions List to look up the correct **requestor IP/STS**. [<79>](#)

3.3.5.3.4 Message Transmission

The **resource IP/STS** sends a [wsignout1.0](#) Request Message by returning an HTTP 302 response to the **web browser requestor** with Location set to the URL of the **requestor IP/STS**. All the query string parameters required for the protocol MUST be set properly, as specified in wsignout1.0 Request Message.

3.3.5.4 Inbound wsignoutcleanup1.0 Request Message Processing

This message is used to request that a **resource IP/STS** initiate clean-up operations for cached session state, if any exist, for the **user** who triggered the request.

3.3.5.4.1 Protocol Activation

The protocol is triggered by receipt of a [wsignoutcleanup1.0](#) message from a **requestor IP/STS**.

3.3.5.4.2 Clean-Up Processing

A **resource IP/STS** SHOULD send [wsignoutcleanup1.0](#) messages to **WS resources** for which **security tokens** have been issued. Because the protocol does not guarantee a response to a wsignoutcleanup1.0 message, a resource IP/STS SHOULD delete any session state that has been locally cached for the **web browser requestor** before sending wsignoutcleanup1.0 messages to WS resources. [<80>](#)

3.3.5.4.3 Relying Party Security Realm Discovery

As specified in section [Relying Party Web Browser Requestor Sessions List \(section 3.1.1.5.2\)](#), a **resource IP/STS** MAY store a session identifier for the **web browser requestor** in an HTTP session cookie (for more information, see [\[RFC2965\]](#)). It SHOULD use this identifier and its Outbound Sessions List to develop the list of **WS resources** to which it SHOULD send [wsignoutcleanup1.0](#) request messages. [<81>](#)

3.3.5.4.4 Message Transmission

The **resource IP/STS** MUST send a [wsignoutcleanup1.0](#) request message to each **WS resource** using an explicit HTTP GET method because the protocol does not support chaining wsignoutcleanup1.0 messages using the HTTP 1.1 redirection facilities. How these messages are sent is implementation-specific and is not addressed in this protocol. The resource IP/STS MAY [<82>](#) walk the list of WS resources and issue the requests individually.

3.3.5.4.5 Response Message Processing

When clean-up processing is complete, the **relying party** SHOULD return any relying party specific data (such as a string indicating that clean up is complete) to the **web browser requestor**. If the *wreply* parameter, as specified in section [2.2.6](#), was included with the wsignoutcleanup1.0 message, the response SHOULD be sent to the URL specified by *wreply*. [<83>](#)

3.3.6 Timer Events

A **requestor IP/STS** does not need to interact with any timers, or service any timer events, beyond those that might be used by the underlying transport to transmit and receive messages over HTTP and SSL/TLS, or those specified in [Timer Events \(section 3.1.6\)](#). [<84>](#)

3.3.7 Other Local Events

A **relying party** does not have dependencies on local events beyond what is specified in section [3.1.7](#).

3.4 Web Browser Requestor Details

This section discusses how the **web browser requestor** is used to transport protocol messages.

3.4.1 Abstract Data Model

A **web browser requestor** does is not required to understand any protocol-specific data for the correct operation of the protocol. It MUST be able to support HTTP query string and POST body parameterization. To provide the best end-user experience, it SHOULD [<85>](#) be able to support HTTP cookies (for more information, see [\[RFC2965\]](#)).

3.4.2 Timers

A **web browser requestor** does not depend on timers beyond those that are used by the underlying transport to transmit and receive messages over HTTP and SSL/TLS, as specified in section [3.1.2](#).

A web browser requestor is not required to be aware of an implementation's use of timers to determine when the validity intervals of **security tokens** and Authentication Contexts expire.

3.4.3 Initialization

There is no protocol-specific initialization for a **web browser requestor**. It simply needs to be ready to perform the standard HTTP 1.1 methods required for accessing **WS resources**. Specifically, it MUST support HTTP GET and POST methods and properly respond to HTTP 1.1 redirection and error responses.

3.4.4 Higher-Layer Triggered Events

Protocol messages are exchanged between a **requestor IP/STS** and a **relying party**. The only function of the **web browser requestor** with respect to the protocol is to transport these messages. The web browser requestor can be triggered to begin protocol message exchange by receipt of an

HTTP/1.1 302 message found that includes a location directive, or an HTTP/1.1 200 OK that includes a form with method set to POST.

3.4.5 Processing Events and Sequencing Rules

A **web browser requestor** plays a passive role in the operation of the protocol. Its only function is to transport protocol message requests and responses between a **requestor IP/STS** and one or more **relying parties**. It is not required to understand the types or content of these protocol messages.

The web browser requestor SHOULD transport all protocol message requests and responses between a requestor IP/STS and a relying party without changing the messages. [<86>](#86)

3.4.6 Timer Events

A **web browser requestor** is not required to interact with any timers, or service any timer events, beyond those that are used by the underlying transport to transmit and receive messages over HTTP and SSL/TLS, or those specified in section [3.1.6](#3.1.6).

3.4.7 Other Local Events

A **web browser requestor** does not have dependencies on local events beyond what is specified in section [3.1.7](#3.1.7).

4 Protocol Examples

This section contains an example scenario for the protocol similar to the scenario specified in section 1.3. This section also includes sample XML sections for the [wsignin1.0](#) response parameters.

4.1 Message Flows

This section describes an example flow of messages among a **web browser requestor**, a **WS resource**, a **resource IP/STS**, and a **requestor IP/STS**, including example messages.

1. The web browser requestor sends a GET for the WS resource URL to the WS resource.
2. The WS resource returns a 302 Redirect to the resource IP/STS at the resource IP/STS URL.
 1. Query string parameters are appended to the **IP/STS** URL, that forms a [wsignin1.0](#) request message, as follows:
 1. A query string parameter is added, which indicates that WS resource is the resource wanting authentication (*wreply*).
 2. The original requested WS resource URL is saved as a context parameter in the sign-in message (*wctx*).

2. Example URL follows.

```
https://adfsresource1.treyresearch.net/adfs/ls/?wa=wsignin1.0&wreply=
https%3a%2f%2fadfsweb1.treyresearch.net%3a8081%2fclaimapp%2f&wct=
2006-07-11T03%3a26%3a39Z&wctx=https%3a%2f%2fadfsweb1.treyresearch.net
%3a8081%2fclaimapp%2fDefault.aspx
```

3. The web browser requestor sends a GET for the resource IP/STS URL.
4. The resource IP/STS returns a 302 Redirect to the requestor IP/STS URL:
 1. Query string parameters are appended to the requestor IP/STS URL, which form a *wsignin1.0* request message (section 2.2.3), as follows:
 1. A query string parameter is added, which indicates that the resource IP/STS is the partner wanting authentication (*wtrealm*).
 2. Any state required by the resource IP/STS to continue processing the request is saved as a context parameter in the sign-in message (*wctx*).
2. Example URL follows.

```
https://adfsaccount1.adatum.com/adfs/ls/auth/integrated/?wa=wsignin1.0
&wtrealm=urn%3afederation%3atreyCrazyResearch&wct=2006-07-11T03%3a28
%3a05Z&wctx=https%3a%2f%2fadfsweb1.treyresearch.net%3a8081%2fclaimapp
%2f%5chttps%3a%2f%2fadfsweb1.treyresearch.net%3a8081%2fclaimapp%2f
Default.aspx
```

5. The web browser requestor sends a GET for the requestor IP/STS URL.
 1. Authentication occurs.
6. The requestor IP/STS sends a 200 response to the client with a POST Redirect to the resource IP/STS URL:

1. The returned [\[HTML\]](#) contains a hidden form that contains a [wsignin1.0](#) response and JavaScript, which causes the form to POST immediately (optionally the form can have a visible Submit button). The form's target is the resource IP/STS URL.
2. The response contains a RequestSecurityTokenResponse message that includes a SAML token whose audience is the resource IP/STS. The token is signed by the requestor IP/STS X.509 certificate.
3. Example form follows.

```
<?xml version="1.0" encoding="utf-8" ?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html>
<head>
<title>Working...
</title>
</head>
<body>
<form method="POST" action=
"https://adfsresource1.treyresearch.net/adfs/ls/" />
<input type="hidden" name="wa" value="wsignin1.0" />
<input type="hidden" name="wresult" value="
<lt;wst:RequestSecurityTokenResponse xmlns:wst=&quot;
http://schemas.xmlsoap.org/ws/2005/02/trust&quot;>
<lt;wst:RequestedSecurityToken>
<lt;saml:Assertion AssertionID=&quot;
_784067ac-af2c-40b1-993a-cbb376597b6a&quot;
IssueInstant=&quot;2006-07-11T03:15:40Z&quot;
Issuer=&quot;urn:federation:apieceodata&quot;
MajorVersion=&quot;1&quot; MinorVersion=&quot;
1&quot; xmlns:saml=&quot;urn:oasis:names:tc:SAML:1.0:
assertion&quot;>
<lt;saml:Conditions NotBefore=&quot;2006-07-11T03:15:40Z&quot;
NotOnOrAfter=&quot;2006-07-11T04:15:40Z&quot;>
<lt;saml:AudienceRestrictionCondition>
<lt;saml:Audience>urn:federation:treyCrazyResearch
<lt;/saml:Audience>
<lt;/saml:AudienceRestrictionCondition>
<lt;/saml:Conditions>
<lt;saml:Advice>
<lt;adfs:CookieInfoHash xmlns:adfs=&quot;urn:microsoft:
federation&quot;>11AMDR+AihBUJMPNKS3N64ruuaY=
<lt;/adfs:CookieInfoHash>
<lt;/saml:Advice>
<lt;saml:AuthenticationStatement AuthenticationInstant=&
quot;2006-07-11T03:15:40Z&quot; AuthenticationMethod=
&quot;urn:federation:authentication:windows&quot;>
<lt;saml:Subject>
<lt;saml:NameIdentifier Format=&quot;
http://schemas.xmlsoap.org/claims/UPN&quot;>adamcar@adatum.com
<lt;/saml:NameIdentifier>
<lt;/saml:Subject>
<lt;/saml:AuthenticationStatement>
<lt;saml:AttributeStatement>
<lt;saml:Subject>
<lt;saml:NameIdentifier Format=&quot;
http://schemas.xmlsoap.org/claims/UPN&quot;>adamcar@adatum.com
<lt;/saml:NameIdentifier>
<lt;/saml:Subject>
<lt;saml:Attribute AttributeName=&quot;Group&quot;
AttributeNameNamespace=&quot;http://schemas.xmlsoap.org/claims&quot;>
<lt;saml:AttributeValue>ClaimAppMapping
<lt;/saml:AttributeValue>
<lt;/saml:Attribute>
<lt;saml:Attribute AttributeName=&quot;Group&quot;
AttributeNameNamespace=&quot;http://schemas.xmlsoap.org/claims&quot;>
<lt;saml:AttributeValue>TokenAppMapping
```

```

</saml:AttributeValue>
</saml:Attribute>
<saml:Attribute AttributeName="Group"
  AttributeNamespace="http://schemas.xmlsoap.org/claims";>
<saml:AttributeValue>ResearchPlatinum
</saml:AttributeValue>
</saml:Attribute>
<saml:Attribute AttributeName="Group"
  AttributeNamespace="http://schemas.xmlsoap.org/claims";>
<saml:AttributeValue>ResearchPurchaser
</saml:AttributeValue>
</saml:Attribute>
<saml:Attribute AttributeName="ResearchFirstName"
  AttributeNamespace="http://schemas.xmlsoap.org/claims";>
<saml:AttributeValue>Adam
</saml:AttributeValue>
</saml:Attribute>
</saml:AttributeStatement>
<Signature xmlns="http://www.w3.org/2000/09/xmldsig#";>
  <SignedInfo>
    <CanonicalizationMethod Algorithm="
      http://www.w3.org/2001/10/xml-exc-c14n#"; />
    <SignatureMethod Algorithm="
      http://www.w3.org/2000/09/xmldsig#rsa-sha1"; />
    <Reference URI="#_784067ac-af2c-40b1-993a-cbb376597b6a";>
    <Transforms>
      <Transform Algorithm="
        http://www.w3.org/2000/09/xmldsig#enveloped-signature"; />
      <Transform Algorithm="
        http://www.w3.org/2001/10/xml-exc-c14n#"; />
    </Transforms>
    <DigestMethod Algorithm="
      http://www.w3.org/2000/09/xmldsig#sha1"; />
    <DigestValue>Q4/7YEpc3fTVcZNPp6cyU+VAIE=
    </DigestValue>
  </Reference>
</SignedInfo>
  <SignatureValue>tHgfkQPiPTXnMWS3K1N6xkL5FOaRzoJicg0ZqV3CImj
gNtsAGhX8CKCJryYC3ARQdF6jCHGgthnPnyW0jPjA8/VPl1s+Nt5Pe9ODgwhJHx
3wk+gbQs8Ty0IY1+jftp4DM0WF6/LCvIxTmAZ4+GT40fCK9RxbgP4Wbr4cj6lE6
C6wwX4odK+lqxVwtR5qx64SUyzPq0zbKG8YX0fSIuSgBJYKLJt+CUK+6YjCeY8m
xH89iL2HWXEBTeMuLh32QrFV2+PFg3jeDcCxCIC9VjwmAyU6VZr3e1Spqp/RgtN
Dj8XckSZvVdOrVztd7sEnJmJmmoeaLpbYNUJWmlbvGsvQ==
  </SignatureValue>
  <KeyInfo>
    <X509Data>
      <X509Certificate>MIIC0jCCAb6gAwIBAgIQKVyguUrraIFEOd4S41AnOTAJB
gUrDgMCHQUAMCkxJzAlBgNVBAMThkZlZGVyYXRpb24gU2VydMvYIEFERlNBY2NvdW
50MTAeFw0wNjAxMzEwMjU1MTZaFw0wNzAxMzEwODU1MTZaMCKxJzAlBgNVBAMThkZl
ZGVyYXRpb24gU2VydMvYIEFERlNBY2NvdW50MTCCASIdDQYJKoZIhvcNAQEBBQAD
ggEPADCCAQoCggEBAL93fgaIJs+E+Y+eYhCrAVHYECQtE43ioEvv1jz/4c81eC6i2
Rpiwc+2N+AXWaHgae3vWoCkTCXWYomm9DXMlJpqtJq0pEWPLjkH7kMa34LUgaKxR
bbmw6Bus9SwGjhCQKdtAwX09wcp2gEW7FU+gkAtkcoQdvlbEOXw1CInWKX05jbybT
/18qGHsz+69fhJdDNoQtjwkIAXqQd9PNdp7r4POG4p5+6kQtkmX3xIWIB0rh0Bml0
9GBbWQUoYbvjx6GH7LB/XJhLIJ7RdIHTv4yS2sTHEgLnF3nx1hjW4cZlznDA7OywV
pZKerK8lm7CrHzd78dOJeZYJs2taHUEHY0CAWEAATAJBgUrDgMCHQUAA4IBAQBx9D
B1lt+3/efeQCQVuAwn9ypInC7CwWGeVW8ZyJe5V+3Lx9dQH101MpuXpOX+5tEdoDQ
fqjNNvwmA6UM9v1b70CmjWtI/b77scaBAq6B0iQqWlEcyiHKNKjIvF3/ME/SYOL7p
XmY1zvX9EgnOQVtNpLBueesUo86APAZoQWWWeH5qVwrz73sDoJCToGAtsrnN2b3c
9415u1KSyQNfVQVQ13vCdJXELjuTv0PcGcibgKKT4bEKzHcdkY038cuuNPgt1a7d0
QnrZ4ZgppjThLuBLVbzyMP3FiQgFC2hiZ0IKb0uYG5hZY7+wIRhbuYgyqLWsimRL/
aw4m7NdLORTBO
    </X509Certificate>
  </X509Data>
  </KeyInfo>
  </Signature>
</saml:Assertion>
</wst:RequestedSecurityToken>
<wsp:AppliesTo xmlns:wsp="

```

```

http://schemas.xmlsoap.org/ws/2004/09/policy&quot;>
&lt;wsa:EndpointReference xmlns:wsa=&quot;
http://schemas.xmlsoap.org/ws/2004/08/addressing&quot;>
&lt;wsa:Address>urn:federation:treyCrazyResearch
&lt;/wsa:Address>
&lt;/wsa:EndpointReference>
&lt;/wsp:AppliesTo>
&lt;/wst:RequestSecurityTokenResponse>" />
<input type="hidden" name="wctx" value=
"https://adfsweb1.treyresearch.net:8081/claimapp/\
https://adfsweb1.treyresearch.net:8081/claimapp/Default.aspx" />
<noscript>
<p>Script is disabled. Please click Submit to continue.
</p>
<input type="submit" value="Submit" />
</noscript>
</form>
<script language="javascript">
window.setTimeout('document.forms[0].submit()',0);
</script>
</body>
</html>

```

7. The web browser requestor sends a POST to the resource IP/STS URL.
8. The resource IP/STS sends a 200 response to the client with a POST Redirect to the WS resource URL:
 1. The returned HTML contains a hidden form that contains a wsignin1.0 response and JavaScript, which causes the form to POST immediately (optionally the form can have a visible Submit button). The form's target is the WS resource URL.
 2. The response contains a RequestSecurityTokenResponse message that includes a SAML token whose audience is the WS resource. The token is signed by the resource IP/STS X.509 certificate.
 3. Full-example HTML form follows.

```

<?xml version="1.0" encoding="utf-8" ?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html>
<head><title>Working...</title></head>
<body>
<form method="POST" action=
"https://adfsweb1.treyresearch.net:8081/claimapp/" />
<input type="hidden" name="wa" value="wsignin1.0" />
<input type="hidden" name="wresult" value="
&lt;wst:RequestSecurityTokenResponse xmlns:wst=&quot;
http://schemas.xmlsoap.org/ws/2005/02/trust&quot;>
&lt;wst:RequestedSecurityToken>
&lt;saml:Assertion AssertionID=&quot;
f81faa32-fc47-4ddb-98a2-0bda61b7ead2&quot;
IssueInstant=&quot;2006-07-11T03:19:05Z&quot;
Issuer=&quot;urn:federation:treyCrazyResearch&quot;
MajorVersion=&quot;1&quot; MinorVersion=&quot;1&quot;
xmlns:saml=&quot;urn:oasis:names:tc:SAML:1.0:assertion&quot;>
&lt;saml:Conditions NotBefore=&quot;
2006-07-11T03:19:05Z&quot;NotOnOrAfter=&quot;
2006-07-11T03:20:05Z&quot;>
&lt;saml:AudienceRestrictionCondition>
&lt;saml:Audience>
https://adfsweb1.treyresearch.net:8081/claimapp/
&lt;/saml:Audience>
&lt;/saml:AudienceRestrictionCondition>

```

```

        </saml:Conditions>
        <saml:Advice>
            <adfs:ClaimSource xmlns:adfs="urn:
microsoft:federation" >
                urn:federation:apieceodata
            </adfs:ClaimSource>
            <adfs:CookieInfoHash xmlns:adfs="urn:
microsoft:federation" >
                fA4h78BffP5tdaujuJ39y0b4qEo=
            </adfs:CookieInfoHash>
        </saml:Advice>
        <saml:AuthenticationStatement AuthenticationInstant="
2006-07-11T03:15:40Z" AuthenticationMethod="
urn:federation:authentication:windows" >
            <saml:Subject>
                <saml:NameIdentifier Format="
http://schemas.xmlsoap.org/claims/UPN" >
                    adamcar@adatum.com
                </saml:NameIdentifier>
            </saml:Subject>
            <saml:AuthenticationStatement>
            <saml:AttributeStatement>
                <saml:Subject>
                    <saml:NameIdentifier Format="
http://schemas.xmlsoap.org/claims/UPN" >
                        adamcar@adatum.com
                    </saml:NameIdentifier>
                </saml:Subject>
                <saml:Attribute AttributeName="Group"
AttributeNamespace="http://schemas.xmlsoap.org/claims" >
                    <saml:AttributeValue>
                        Adatum TokenApp Claim
                    </saml:AttributeValue>
                </saml:Attribute>
                <saml:Attribute AttributeName="Group"
AttributeNamespace="http://schemas.xmlsoap.org/claims" >
                    <saml:AttributeValue>
                        Adatum ClaimApp Claim
                    </saml:AttributeValue>
                </saml:Attribute>
                <saml:Attribute AttributeName="Group"
AttributeNamespace="http://schemas.xmlsoap.org/claims" >
                    <saml:AttributeValue>
                        Purchaser
                    </saml:AttributeValue>
                </saml:Attribute>
                <saml:Attribute AttributeName="FirstName"
AttributeNamespace="http://schemas.xmlsoap.org/claims" >
                    <saml:AttributeValue>
                        Adam
                    </saml:AttributeValue>
                </saml:Attribute>
            </saml:AttributeStatement>
            <Signature xmlns="
http://www.w3.org/2000/09/xmldsig#" >
                <SignedInfo>
                    <CanonicalizationMethod Algorithm="
http://www.w3.org/2001/10/xml-exc-c14n#" />
                    <SignatureMethod Algorithm="
http://www.w3.org/2000/09/xmldsig#rsa-sha1#" />
                    <Reference URI="
#_f81faa32-fc47-4ddb-98a2-0bda61b7ead2#" >
                        <Transforms>
                            <Transform Algorithm="
http://www.w3.org/2000/09/xmldsig#enveloped-signature" />
                            <Transform Algorithm="
http://www.w3.org/2001/10/xml-exc-c14n#" />
                        </Transforms>
                    <DigestMethod Algorithm="

```

```

http://www.w3.org/2000/09/xmldsig#sha1" />
    <DigestValue>
        iLxgLS5ZLZZePFwiGqrGBddqtUI=
    </DigestValue>
    </Reference>
    </SignedInfo>
    <SignatureValue>SeWQYd9ejm1KGmZoi3wWO3wrFGfvTufBus
7KtdVUovY1ha4ov7BVo3N081mou/Fd4+dEHbKmgAMWNegmGygr2bXfNxxJzHUvKf
YKCZoZu/T0tB1QK6mvRnMmcMPTLmmXY1Ckdk8Up2oVf7peFHDolpPlJPudloYtb
DwBn8Z2Z
        bJN/ktBH9bFRa7A17QM5RHc0/5HKU8n4fHQOZ3GhwXWtfiy
uTFYxjofG9nBmlehgKXPD3jftYrP/gCQf4QwCotQHdyatBDos/8gEhaTj049oN8
E8MuaoyGg8krV7/+K9H6jYD6N1N8e5mAoXW5x1lrTFM4yGkLFr8UfVo8PT3pUBg
g==
    </SignatureValue>
    <KeyInfo>
        <X509Data>
            <X509Certificate>MIIC1DCCACgAwIBAgIQ/EU1/PmUxKt
NHdsEKf/aODAJBgUrDgMCHQUAMCoxKDA MBgNVBAMTH0ZlZGVyYX Rpb242U2Vyd mVy
IEFERlNSZXNvdXJzZTEwHhcNM DYwMTMxMDI1OTAwWhcNMDCwMTMxMDg1O
TAWWjAqMSgwJgYDVQQDEx9GZW RlcmF0aW9uIFNlcnZlciBBREZTUwVz b3VyY2UxMI
IBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAsPXHOLNdt4DP7EOT27sQgiR
ILGW9Bsk+HwcGwiTWnedGeG6I JatjragHdW7MYnLrMyiHY9Jc0e4NwzHPDt4qyXlU
C5c8XuHbr74lc32JG6v1rRWYA/rxo+inRfxCrBkh5SOVRv4fiSyXya6jvballEhVc
sORV6g9iG3xw1Ja1GQofwXUkfPqgeXCJynG/NHe2HqNqPTFCHOoqciHBNHYXZPEo7
W0C8XBPavOEQ+lpVGAiubbcTsd3i15uuQGAR5Agce/FquUj4BCiQa8UkcTuuF7mmd
SOajnnNS784/5PK3Nc4CTNYiHPUy jg3r/O23z9BaOVixVgWuTbLr0i/RmtQIDAQAB
MAKGBSsOAwIdBADAQGEBAKLIc rdYRgYTYd/HGQCQDOoDrqAQFYD buU72hvPoc5Jkn1
wZu3Jcl18u5ZsZrStBenrnJmNWN TBmJrwZ6xR1leIYNfpCPY5o3dLK6JMSNgp/oT7
7pn6aYZ5/LpxhF3WGWWBG64/n4pOC5SSDteP1PsCm3LjW6Z9zYOL3mkqui3OLqMBK
cCo+JDMmNkgFDqcxutQ3YMPW DUyX+rvabhxMK2JZgWWhXyhWhn2qzi1z5cCFDA1hK
eMBcqDAMFlSsfbejL5tjapuUkiwAraa4BeTCJYz5/wlaeg2XGeTYOFawPoJG8vvno
n9R37QIBZ/Y8IKdDwZc6zAgTjLzFhDre2bZvM=
        </X509Certificate>
    </X509Data>
    </KeyInfo>
    </Signature>
    </saml:Assertion>
    </wst:RequestedSecurityToken>
    </wsp:AppliesTo xmlns:wsp="http://schemas.xmlsoap.org/
ws/2004/09/policy" />
    <wsa:EndpointReference xmlns:wsa="
http://schemas.xmlsoap.org/ws/2004/08/addressing" />
        <wsa:Address>
            https://adfsweb1.treyresearch.net:8081/claimapp/
        </wsa:Address>
        </wsa:EndpointReference>
    </wsp:AppliesTo>
    </wst:RequestSecurityTokenResponse>" />
<input type="hidden" name="wctx" value="
https://adfsweb1.treyresearch.net:8081/claimapp/Default.aspx" />
<noscript><p>Script is disabled.
Please click Submit to continue.</p><input type="submit"
value="Submit" /></noscript></form>
<script language="javascript">
window.setTimeout('document.forms[0].submit()',0);</script>
</body>
</html>

```

9. The web browser requestor sends a POST to the WS resource URL.
 1. The WS resource validates the SAML token.
10. The WS resource returns a 200 response from the application to the client. This message is an internal implementation detail of the WS-Resource and is mentioned here for completeness only. This message is not necessary for interoperability. The WS-Resource is not restricted by the protocol once it has received the token.

1. The WS resource authorizes a **user's** request based on attributes from the SAML token.
11. The web browser requestor continues to browse the application at the WS resource, which results in additional traffic.

4.2 XML Examples

This section describes some XML examples of data found in the [wsignin1.0](#) response *wresult* parameter.

4.2.1 Example RSTR

The following is an example of the shell of a properly constructed RSTR with the optional `wsp:AppliesTo` element containing sample content.

```
<wst:RequestSecurityTokenResponse>
  <wsp:AppliesTo>
    <wsa:EndpointReference>
      <wsa:Address>
        https://TreyResearch.net/Ordering
      </wsa:Address>
    </wsa:EndpointReference>
  </wsp:AppliesTo>
  <wst:RequestedSecurityToken>
    <saml:Assertion> ... </saml:Assertion>
  </wst:RequestedSecurityToken>
</wst:RequestSecurityTokenResponse>
```

4.2.2 Example SAML Attribute Element

The following is an example of the SAML Attribute element with sample data. Based on the content, this attribute contains a group **claim** of Purchasing Agent.

```
<saml:Attribute AttributeName="Group"
  AttributeNamespace="http://schemas.xmlsoap.org/claims">
  <saml:AttributeValue>
    Purchasing Agent
  </saml:AttributeValue>
</saml:Attribute>
```

4.2.3 Using the X509Certificate Element

The following is an example of the KeyInfo element, as specified in [\[XMLDSig\]](#). This KeyInfo element is using the X509Certificate element to directly include the X.509 certificate, as recommended. For further specifications on the relevant message syntax, see [Security Token Signature \(section 2.2.4.2.2\)](#).

```
<KeyInfo>
  <X509Data xmlns="http://www.w3.org/2000/09/xmldsig#">
    <X509Certificate>MIIEJDCCA42gAw ... </X509Certificate>
  </X509Data>
</KeyInfo>
```


4.2.4 Using the X509SKI Element

The following is an example of the KeyInfo element, as specified in [\[XMLDSig\]](#). This KeyInfo element is using the optional X509SKI element to reference the X.509 certificate without copying the entire certificate. For further specifications on the relevant message syntax, see [Security Token Signature \(section 2.2.4.2.2\)](#).

```
<KeyInfo>
  <X509Data xmlns="http://www.w3.org/2000/09/xmldsig#">
    <X509SKI> 31d97bd7 </X509SKI>
  </X509Data>
</KeyInfo>
```

4.3 Raw Message Examples

This section shows messages that represent sample HTTP encoded wsignin1.0 [requests](#) and [responses](#) involving a **WS resource**, **resource IP/STS**, and **requestor IP/STS** for one **user** accessing a WS resource.

4.3.1 Original GET to WS Resource

```
GET /claims/ HTTP/1.1
Accept: image/gif, image/x-xbitmap, image/jpeg, image/pjpeg,
application/vnd.ms-excel, application/vnd.ms-powerpoint,
application/msword, application/x-shockwave-flash, */*
Accept-Language: en-us
UA-CPU: x86
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.2; SV1;
.NET CLR 1.1.4322; .NET CLR 2.0.50727; InfoPath.1)
Host: treyws-test
Connection: Keep-Alive
```

4.3.2 HTTP Redirect to Resource IP/STS

```
HTTP/1.1 302 Found
Cache-Control: private
Content-Length: 298
Content-Type: text/html; charset=utf-8
Location: https://treyst-7/adfs/ls/?wa=wsignin1.0&
wreply=https%3a%2f%2ftreyws-test%2fclaims%2f&
wct=2006-07-13T07%3a13%3a22Z&wctx=
https%3a%2f%2ftreyws-test%2fclaims%2fDefault.aspx
Server: Microsoft-IIS/6.0
X-Powered-By: ASP.NET
X-AspNet-Version: 2.0.50727
Date: Thu, 13 Jul 2006 07:13:22 GMT

<html><head><title>Object moved</title></head><body>
<h2>Object moved to <a href="https://treyst-7/adfs/ls/?
wa=wsignin1.0&wreply=https%3a%2f%2ftreyws-test%2fclaims%2f&
wct=2006-07-13T07%3a13%3a22Z&wctx=
https%3a%2f%2ftreyws-test%2fclaims%2fDefault.aspx">here</a>.</h2>
</body></html>
```

4.3.3 HTTP GET To Resource IP/STS

```
GET /adfs/ls/?wa=wsignin1.0&wreply=
https%3a%2f%2ftreyws-test%2fclaims%2f&wct=2006-07-13T07%3a13%3a22Z&
wctx=https%3a%2f%2ftreyws-test%2fclaims%2fDefault.aspx HTTP/1.1
```

```
Accept: image/gif, image/x-xbitmap, image/jpeg, image/pjpeg,
application/vnd.ms-excel, application/vnd.ms-powerpoint,
application/msword, application/x-shockwave-flash, */*
Accept-Language: en-us
UA-CPU: x86
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.2; SV1;
.NET CLR 1.1.4322; .NET CLR 2.0.50727; InfoPath.1)
Connection: Keep-Alive
Host: treysts-7
```

4.3.4 HTTP Redirect to Requestor IP/STS

```
HTTP/1.1 302 Found
Date: Thu, 13 Jul 2006 07:13:22 GMT
Server: Microsoft-IIS/6.0
X-Powered-By: ASP.NET
X-AspNet-Version: 2.0.50727
Location: https://adatumsts-7/adfs/ls/?wa=
wsignin1.0&wrealm=urn%3afederation%3atrety+research&wct=
2006-07-13T07%3a13%3a22Z&wctx=https%3a%2f%2ftreyws-test%2fclaims%2f%
5chttps%3a%2f%2ftreyws-test%2fclai
ms%2fDefault.aspx
Cache-Control: private
Content-Type: text/html; charset=utf-8
Content-Length: 336

<html><head><title>Object moved</title></head><body>
<h2>Object moved to <a href="https://adatumsts-7/adfs/ls/?wa=
wsignin1.0&wrealm=urn%3afederation%3atrety+research&wct=
2006-07-13T07%3a13%3a22Z&wctx=https%3a%2f%2ftreyws-test%2fclaims
%2f%5chttps%3a%2f%2ftreyws-test%2fclaims%2fDefault.aspx">
here</a>.</h2>
</body></html>
```

4.3.5 HTTP GET to Requestor IP/STS

```
GET /adfs/ls/?wa=wsignin1.0&wrealm=urn%3afederation%3atrety+research&
wct=2006-07-13T07%3a13%3a22Z&wctx=https%3a%2f%2ftreyws-test%2fclaims
%2f%5chttps%3a%2f%2ftreyws-test%2fclaims%2fDefault.aspx HTTP/1.1
Accept: image/gif, image/x-xbitmap, image/jpeg, image/pjpeg,
application/vnd.ms-excel, application/vnd.ms-powerpoint,
application/msword, application/x-shockwave-flash, */*
Accept-Language: en-us
UA-CPU: x86
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.2; SV1;
.NET CLR 1.1.4322; .NET CLR 2.0.50727; InfoPath.1)
Connection: Keep-Alive
Host: adatumsts-7
```

4.3.6 Receive Security Token from Requestor IP/STS in HTML Form

```
HTTP/1.1 200 OK
Date: Thu, 13 Jul 2006 07:13:32 GMT
Server: Microsoft-IIS/6.0
X-Powered-By: ASP.NET
WWW-Authenticate: Negotiate
oYgGMIGdoAMKAQChCwYJKoZlgvcSAQICooGIBIGFYIGCBgkqhkiG9xIBAgICAG9zMHG
gAwIBBaEDAgEPomUwY6ADAgEXolwEWvlfVlnxDcKFUIUA+QeYjw1JgxS3Za+jWCdhQC
aUfc1DBEozTSd4QWIpu8DI6mTEX/R6nT8z9dE3g6vYJGlam7jdYuYxc1slacV884faP
u7LunNUYOd6U
```

lp+rw==
X-AspNet-Version: 2.0.50727
Cache-Control: no-cache
Pragma: no-cache
Expires: -1
Content-Type: text/html; charset=utf-8
Content-Length: 5660

```
<?xml version="1.0" encoding="utf-8" ?><!DOCTYPE html PUBLIC "-//W3C//
DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/
xhtml1-transitional.dtd"><html><head><title>Working...</title></head>
<body><form method="POST" action="https://treysts-7/adfs/ls/" />
<input type="hidden" name="wa" value="wsignin1.0" /><input type=
"hidden" name="wresult" value="&lt;wst:RequestSecurityTokenResponse
xmlns:wst=&quot;http://schemas.xmlsoap.org/ws/2005/02/trust&quot;
&lt;wst:RequestedSecurityToken&lt;saml:Assertion AssertionID=&quot;
efe97b43-7653-4914-ac6e-3058f9746200&quot; IssueInstant=&quot;
2006-07-13T07:13:32Z&quot; Issuer=&quot;urn:federation:adatum&quot;
MajorVersion=&quot;1&quot; MinorVersion=&quot;1&quot; xmlns:saml=
&quot;urn:oasis:names:tc:SAML:1.0:assertion&quot; >&lt;saml:
Conditions NotBefore=&quot;2006-07-13T07:13:32Z&quot;
NotOnOrAfter=&quot;2006-07-13T08:13:32Z&quot; >&lt;saml:
AudienceRestrictionCondition&lt;saml:Audience>urn:federation:
treyst-research&lt;/saml:Audience&lt;/saml:
AudienceRestrictionCondition&lt;/saml:Conditions&lt;saml:
Advice&lt;adfs:CookieInfoHash xmlns:adfs=&quot;urn:microsoft:
federation&quot; >zAlnNkoHU0Mug3rcyh8ScoWnsKE=&lt;/adfs:
CookieInfoHash&lt;/saml:Advice&lt;saml:AuthenticationStatement
AuthenticationInstant=&quot;2006-07-13T07:13:32Z&quot;
AuthenticationMethod=&quot;urn:federation:authentication:
windows&quot; >&lt;saml:Subject&lt;saml:NameIdentifier
Format=&quot;http://schemas.xmlsoap.org/claims/UPN&quot; >
Administrator@adatum.com&lt;/saml:NameIdentifier&lt;/saml:
Subject&lt;/saml:AuthenticationStatement&lt;saml:
AttributeStatement&lt;saml:Subject&lt;saml:NameIdentifier
Format=&quot;http://schemas.xmlsoap.org/claims/UPN&quot; >
Administrator@adatum.com&lt;/saml:NameIdentifier&lt;/saml:
Subject&lt;saml:Attribute AttributeName=&quot;EmailAddress&quot;
AttributeNamespace=&quot;http://schemas.xmlsoap.org/claims&quot; >
&lt;saml:AttributeValue>administrator@adatum.com&lt;/saml:
AttributeValue&lt;/saml:Attribute&lt;saml:Attribute AttributeName=
&quot;CommonName&quot; AttributeNamespace=&quot;
http://schemas.xmlsoap.org/claims&quot; >&lt;saml:AttributeValue>
Mister Admin&lt;/saml:AttributeValue&lt;/saml:Attribute&lt;
saml:Attribute AttributeName=&quot;Group&quot; AttributeNamespace=
&quot;http://schemas.xmlsoap.org/claims&quot; >
&lt;saml:AttributeValue>ClaimSubmitter&lt;/saml:AttributeValue>
&lt;/saml:Attribute&lt;saml:Attribute AttributeName=&quot;
Group&quot; AttributeNamespace=&quot;http://schemas.xmlsoap.org/
claims&quot; >&lt;saml:AttributeValue>ClaimApprover&lt;/saml:
AttributeValue&lt;/saml:Attribute&lt;/saml:
AttributeStatement&lt;Signature xmlns=&quot;
http://www.w3.org/2000/09/xmldsig#&quot; >&lt;SignedInfo>
&lt;CanonicalizationMethod Algorithm=&quot;
http://www.w3.org/2001/10/xml-exc-c14n#&quot; />&lt;
SignatureMethod Algorithm=&quot;
http://www.w3.org/2000/09/xmldsig#rsa-sha1&quot; />&lt;
Reference URI=&quot;#_efe97b43-7653-4914-ac6e-3058f9746200&
quot; >&lt;Transforms>&lt;Transform Algorithm=&quot;
http://www.w3.org/2000/09/xmldsig#enveloped-signature&quot; />
&lt;Transform Algorithm=&quot;
http://www.w3.org/2001/10/xml-exc-c14n#&quot; />&lt;/Transforms>
&lt;DigestMethod Algorithm=&quot;
http://www.w3.org/2000/09/xmldsig#sha1&quot; />&lt;
DigestValue>u1ZzBz7qhliMv9/sZ9w2xTlE2UA=&lt;/DigestValue&lt;
/Reference>&lt;/SignedInfo&lt;SignatureValue>eG/BzLV0Pm2XX9/
0vjWoDPj86NFCVb/BKkWBGkoCDaOAvBfdGJD4gD3kAJtE51fm0ddvy0T0W0KqnoId
MBxSQUc/z5a+UgdTqJ68pK/b+M8hrdXms/Tpdq/M+TMaL43sdIb9cGvrHLCCxqg/f
FvQKM1fbKrjju2u6yZdilVxaDU=&lt;/SignatureValue&lt;KeyInfo>&lt;X5
```

```

09Data>&lt;
X509Certificate>MIID1DCCARYgAwIBAgIKG3gmFQAAAAAExTANBgkqhkiG9w0BA
QUFADAZMRcwFQYDVQQDEw5UaGlyZFBhcnR5Um9vdAeFw0wNjA3MTIyMTAwMDNaFw
0wNzA3MTIyMTAwMDNaMBYxPDASBgNVBAMTC2FkYXR1bXN0cy03MIGfMA0GCSqGSIb
3DQEBAQUAA4GNADCBiQKBgQDSYqm/9eNOX4E72SSg0uqlJW3Mq9tuAKZ8aysPZhlz
MWwqK1951HDzDBtOUJaO+vBYADxJ/zFtT/4KxE3KiZ152PuAppzR8kK01KGLYhhdA
reFBC7mSbh9xSxYAg9yWJiYO+M8o3GZ02uBveAgimDHHJm6gCxcJlqkr91z8AKFzw
IDAQABo4IBozCAAZ8wDgYDVROPAQH/BAQDAgTwMEQGCCSgSIb3DQEQJdWQ3MDUwDgY
IKoZIHvCNawICAgCAMA4GCCqGSIb3DQMEAgIAgDAHBgUrDgMCBzAKBggqhkiG9w0D
BzAdBgNVHQ4EFgQUYcR0x1tdk4e01hJ/bYkb7z4b8hIwEwYDVR01BAAwCgYIKwYBB
QUHAIwIwYDVR01jBBgwFoAUF4n0HZrhd
VIA5STOYb4GSK5r3OUwVQYDVR0fBE4wTDBKOEigRoZEAHR0cDovL2N1cnRhdXR0Lm
Rucy5jb3JwLmlpY3Jvc29mdC5jb20vQ2VydEVucm9sbC9UaGlyZFBhcnR5Um9vdC5
jcmwwGZoGCCsGAQUFBwEBBIGNMIGKMEIGCCsGAQUFBzAChjZodHRwOi8vY2VydGF1
dGgvQ2VydEVucm9sbC9jZXJ0YXV0aF9UaGlyZFBhcnR5Um9vdC5jcnQwRAYIKwYBB
QUHMAKGOGZpbGU6Ly9cXGN1cnRhdXR0XEN1cnRFBnJvbGxcY2VydGF1dGhfVGhpcm
RQYXJ0eVJvb3QuY3J0MA0GCSqGSIb3DQEBBQUAA4IBAQB3qi+xNT/c6/mVxd3dCz1
ArkpxJ3bHIro/HAX7n7xGA6vqvTd49WdQt8fvKgEHfBcvI7maWetwFL4RcPm3UmVr
IV/+JkCiHdgf3eCPip36sZt94zVrGffZFa98/jxo4wlo+WGFUtrtqE6S/gi0LREiM
LCKRCZmh+UL7JHeJWHeY953K9cQLM2dGbZJZk/kula2CUI7GfPx4dKmlCGnboMcmk
3ifhCMmoRojesiAdCf+s89kbHtffYIITuFdb3SS9kwu2FsA8J3HHm20/vM8YyDq5x
VnZmP5sREumUFno/nq0502X/GzR63hn6nqoWWR4UcVkkq9FCo8ygEGm6yJefu&lt;
/X509Certificate>&lt;/X509Data>&lt;/KeyInfo>&lt;/Signature>&lt;
/saml:Assertion>&lt;/wst:RequestedSecurityToken>&lt;/wsp:AppliesTo
xmlns:wsp=&quot;http://schemas.xmlsoap.org/ws/2004/09/policy&quot;
>&lt;/wsp:AppliesTo>&lt;/wsp:AppliesTo>&lt;/wsp:AppliesTo>&lt;/wsp:AppliesTo>
http://schemas.xmlsoap.org/ws/2004/08/addressing&quot;>&lt;
wsa:Address>urn:federation:trey_research&lt;/wsa:Address>&lt;/
wsa:Address>&lt;/wsp:AppliesTo>&lt;/wsp:AppliesTo>&lt;/wsp:AppliesTo>
wst:RequestSecurityTokenResponse>" /><input type="hidden"
name="wctx" value="https://treyws-test/claims/
\https://treyws-test/claims/Default.aspx" /><noscript>
<p>Script is disabled. Please click Submit to continue.</p>
<input type="submit" value="Submit" /></noscript></form>
<script language="javascript">
window.setTimeout('document.forms[0].submit()',0);
</script></body></html>

```

4.3.7 HTTP POST Security Token to Resource IP/STS

```

POST /adfs/ls/ HTTP/1.1
Accept: image/gif, image/x-xbitmap, image/jpeg, image/pjpeg,
application/vnd.ms-excel, application/vnd.ms-powerpoint,
application/msword, application/x-shockwave-flash, */*
Referer: https://adatumsts-7/adfs/ls/auth/integrated/?
wa=wsignin1.0&wtrealm=urn%3afederation%3atrey+research&
wct=2006-07-13T07%3a13%3a22Z&wctx=https%3a%2f%2ftreyws-test%
2fclaims%2f%5chttps%3a%2f%2ftreyws-test%2fclaims%2fDefault.aspx
Accept-Language: en-us
Content-Type: application/x-www-form-urlencoded
UA-CPU: x86
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.2; SV1;
.NET CLR 1.1.4322; .NET CLR 2.0.50727; InfoPath.1)
Host: treysts-7
Content-Length: 5512
Connection: Keep-Alive
Cache-Control: no-cache

wa=wsignin1.0&wresult=%3Cwst%3ARequestSecurityTokenResponse+xmlns
%3Awst%3D%22http%3A%2F%2Fschemas.xmlsoap.org%2Fws%2F2005%2F02%2
Ftrust%22%3E%3Cwst%3ARequestedSecurityToken%3E%3Csaml%3AAssertion+
AssertionID%3D%22_efe97b43-7653-4914-ac6e-3058f9746200%22+
IssueInstant%3D%222006-07-13T07%3A13%3A32Z%22+Issuer%3D%22urn%3A
federation%3Aadatum%22+MajorVersion%3D%221%22+MinorVersion%3D%
221%22+xmlns%3Asaml%3D%22urn%3Aoasis%3Anames%3Aatc%3ASAML%3A1.0%3A
assertion%22%3E%3Csaml%3AConditions+NotBefore%3D%222006-07-13T07

```

[MS-MWBF] - v20240423
Microsoft Web Browser Federated Sign-On Protocol
Copyright © 2024 Microsoft Corporation
Release: April 23, 2024

```
K9cQLM2dGbZJZk%2Fkula2CUI7GfPx4dKm1CGnboMcmk3ifhCMmoRojeSiADcF%
2Bs89kbHtffYITuFdb3SS9kwu2FsA8J3HHm2O%2FvM8YyDg5xVnZmP5sREumUF
no%2FmqO502X%2FGzR63hn6nqoWwR4UcVkkq9FCo8ygEGm6yjefu%3C%2FX
509Certificate%3E%3C%2FX509Data%3E%3C%2FKeyInfo%3E%3C%2F
Signature%3E%3C%2Fsaml%3AAssertion%3E%3C%2Fwst%3A
RequestedSecurityToken%3E%3Cwsp%3AAppliesTo+xmlns%3Aawsp%3D%22
http%3A%2F%2Fschemas.xmlsoap.org%2Fws%2F2004%2F09%2Fpolicy%22%
3E%3Cwsa%3AEndpointReference+xmlns%3Aawsa%3D%22http%3A%2F%2F
schemas.xmlsoap.org%2Fws%2F2004%2F08%2Faddressing%22%3E%3C
wsa%3AAddress%3Eurn%3Afederation%3Atrey+research%3C%2F
wsa%3AAddress%3E%3C%2Fwsa%3AEndpointReference%3E%3C%2F
wsp%3AAppliesTo%3E%3C%2Fwst%3ARequestSecurityTokenResponse
%3E%3Cwctx=https%3A%2F%2Ftreyws-test%2Fclaims%2F%5Chttps
%3A%2F%2Ftreyws-test%2Fclaims%2FDefault.aspx
```

4.3.8 Receive Security Token from Resource IP/STS in HTML Form

```
HTTP/1.1 200 OK
Date: Thu, 13 Jul 2006 07:13:35 GMT
Server: Microsoft-IIS/6.0
X-Powered-By: ASP.NET
X-AspNet-Version: 2.0.50727
Cache-Control: private
Content-Type: text/html; charset=utf-8
Content-Length: 5750
```

```
<?xml version="1.0" encoding="utf-8" ?><!DOCTYPE html PUBLIC "-
//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/
DTD/xhtml1-transitional.dtd"><html><head><title>Working...</title>
</head><body><form method="POST" action="https://treyws-test/claims/"
/><input type="hidden" name="wa" value="wsignin1.0" /><input type=
"hidden" name="wresult" value="&lt;wst:RequestSecurityTokenResponse
xmlns:wst=&quot;http://schemas.xmlsoap.org/ws/2005/02/trust&quot;
&lt;wst:RequestedSecurityToken&lt;saml:Assertion AssertionID=&quot;
_cbeb6de4-8ddf-4365-8852-92fff3169910&quot; IssueInstant=&quot;
2006-07-13T07:13:35Z&quot; Issuer=&quot;urn:federation:trey research&
quot; MajorVersion=&quot;1&quot; MinorVersion=&quot;1&quot; xmlns:
saml=&quot;urn:oasis:names:tc:SAML:1.0:assertion&quot; &lt;saml:
Conditions NotBefore=&quot;2006-07-13T07:13:35Z&quot; NotOnOrAfter=&
quot;2006-07-13T08:13:35Z&quot; &lt;saml:AudienceRestrictionCondition>
&lt;saml:Audience>https://treyws-test/claims/&lt;/saml:Audience&lt;/
saml:AudienceRestrictionCondition&lt;/saml:
Conditions&lt;saml:Advice>
&lt;adfs:ClaimSource xmlns:adfs=&quot;urn:microsoft:federation&quot;
urn:federation:adatum&lt;/adfs:ClaimSource&lt;
adfs:CookieInfoHash xmlns:
adfs=&quot;urn:microsoft:federation&quot; >RcVgPMKduWJ0ISSZqegkFU+cl80=
&lt;/adfs:CookieInfoHash&lt;/saml:Advice&lt;saml:
AuthenticationStatement AuthenticationInstant=&quot;2006-07-13T07:
13:32Z&quot; AuthenticationMethod=&quot;urn:federation:authentication:
windows&quot; &lt;saml:Subject>&lt;saml:NameIdentifier Format=&quot;
http://schemas.xmlsoap.org/claims/UPN&quot; >Administrator@adatum.com
&lt;/saml:NameIdentifier&lt;/saml:Subject&lt;/saml:
AuthenticationStatement&lt;saml:AttributeStatement&lt;saml:
Subject&lt;saml:NameIdentifier Format=&quot;
http://schemas.xmlsoap.org/claims/UPN&quot; >Administrator@adatum.com
&lt;/saml:NameIdentifier&lt;/saml:Subject&lt;saml:Attribute
AttributeName=&quot;EmailAddress&quot; AttributeNamespace=&quot;
http://schemas.xmlsoap.org/claims&quot; &lt;saml:AttributeValue>
administrator@adatum.com&lt;/saml:AttributeValue&lt;/saml:Attribute>
&lt;saml:Attribute AttributeName=&quot;CommonName&quot;
AttributeNameNamespace=&quot;http://schemas.xmlsoap.org/claims&quot; >
&lt;saml:AttributeValue>Mister Admin&lt;/saml:AttributeValue&lt;/
saml:Attribute&lt;saml:Attribute AttributeName=&quot;Group&quot;
AttributeNameNamespace=&quot;http://schemas.xmlsoap.org/claims&quot; >
&lt;saml:AttributeValue>Form Approver&lt;/saml:AttributeValue&lt;
/saml:Attribute&lt;saml:Attribute AttributeName=&quot;Group&quot;
```



```
was=signin1.0&wresult=%3Cwst%3ARequestSecurityTokenResponse+xmlns%3A  
wst%3D%22http%3A%2F%2Fschemas.xmlsoap.org%2Fws%2F2005%2F02%2Ftrust%  
%22%3E%3Cwst%3ARequestedSecurityToken%3E%3Csaml%3AAssertion+  
AssertionID%3D%22_cbeb6de4-8ddf-4365-8852-92fff3169910%22+  
IssueInstant%3D%222006-07-13T07%3A13%3A35Z%22+Issuer%3D%22urn%3A  
federation%3Atrey+research%22+MajorVersion%3D%221%22+MinorVersion%  
%3D%221%22+xmlns%3Asaml%3D%22urn%3Aoaasis%3Anames%3Atc%3ASAML%3A  
1.0%3AAssertion%22%3E%3Csaml%3AConditions+NotBefore%3D%222006-07-  
13T07%3A13%3A35Z%22+NotOnOrAfter%3D%222006-07-13T08%3A13%3A35Z%  
%22%3E%3Csaml%3AAudienceRestrictionCondition%3E%3Csaml%3AAudience%  
%3EHttps%3A%2F%2Ftreyws-test%2Fclaims%2F%3C%2Fsaml%3AAudience%3E%  
3C%2Fsaml%3AAudienceRestrictionCondition%3E%3C%2Fsaml%3AConditions%  
3E%3Csaml%3AAadvice%3E%3Cadfs%3AClaimSource+xmlns%3Aadfs%3D%22urn%3A  
microsoft%3Afederation%22%3Eurn%3Afederation%3Adatum%3C%2Fadfs%3A  
ClaimSource%3E%3Cadfs%3ACookieInfoHash+xmlns%3Aadfs%3D%22urn%3A  
microsoft%3Afederation%22%3ERcvgPMKduWJ0ISSZqegkFU%2Bcl80%3D%3C%2F  
adfs%3ACookieInfoHash%3E%3C%2Fsaml%3AAadvice%3E%3Csaml%3A  
AuthenticationStatement+AuthenticationInstant%3D%  
222006-07-13T07%3A13%3A32Z%22+AuthenticationMethod%3D%22urn%3A  
federation%3Aauthentication%3Awindows%22%3E%3Csaml%3ASubject%3E%  
3Csaml%3ANameIdentifier+Format%3D%22http%3A%2F%2F  
schemas.xmlsoap.org%2Fclaims%2FUPN%22%3EAdministrator@datum.com  
%3C%2Fsaml%3ANameIdentifier%3E%3C%2Fsaml%3ASubject%3E%3C%2Fsaml%  
3AAuthenticationStatement%3E%3Csaml%3AAtributeStatement%3E%3C  
saml%3ASubject%3E%3Csaml%3ANameIdentifier+Format%3D%22http%3A%2F%  
2Fschemas.xmlsoap.org%2Fclaims%2FUPN%22%3E  
Administrator@datum.com%3C%2Fsaml%3ANameIdentifier%3E%3C%2F  
saml%3ASubject%3E%3Csaml%3AAtribute+AttributeName%3D%22  
EmailAddress%22+AttributeNamespace%3D%22http%3A%2F%2F  
schemas.xmlsoap.org%2Fclaims%22%3E%3Csaml%3AAtributeValue%  
%3E%3C%2Fsaml%3AAtribute%3E%3Csaml%3AAtribute+AttributeName%  
3D%22CommonName%22+AttributeNamespace%3D%22http%3A%2F%2F  
schemas.xmlsoap.org%2Fclaims%22%3E%3Csaml%3AAtributeValue%3E  
Mister+Admin%3C%2Fsaml%3AAtributeValue%3E%3C%2Fsaml%3A  
Attribute%3E%3Csaml%3AAtribute+AttributeName%3D%22Group  
%22+AttributeNamespace%3D%22http%3A%2F%2Fschemas.xmlsoap.org  
%2Fclaims%22%3E%3Csaml%3AAtributeValue%3EForm+Approver%3C%2F  
saml%3AAtributeValue%3E%3C%2Fsaml%3AAtribute%3E%3Csaml%3A  
Attribute+AttributeName%3D%22Group%22+AttributeNamespace%3D%  
22http%3A%2F%2Fschemas.xmlsoap.org%2Fclaims%22%3E%3Csaml%3A  
AttributeValue%3EForm+Submitter%3C%2Fsaml%3AAtributeValue%  
%3E%3C%2Fsaml%3AAtribute%3E%3C%2Fsaml%3AAtributeStatement%  
%3E%3CSignature+xmlns%3D%22http%3A%2F%2Fwww.w3.org%2F2000%2F  
09%2Fxmldsig%23%22%3E%3CSignedInfo%3E%3CCanonicalizationMethod  
+Algorithm%3D%22http%3A%2F%2Fwww.w3.org%2F2001%2F10%2Fxml-exc-  
c14n%23%22+%2F%3E%3CSignatureMethod+Algorithm%3D%22http%3A%2F%  
2Fwww.w3.org%2F2000%2F09%2Fxmldsig%23rsa-sha1%22+%2F%3E%3C  
Reference+URI%3D%22%23_cbeb6de4-8ddf-4365-8852-92fff3169910%  
22%3E%3CTransforms%3E%3CTransform+Algorithm%3D%22http%3A%2F%2  
Fwww.w3.org%2F2000%2F09%2Fxmldsig%23enveloped-signature%22+  
%2F%3E%3CTransform+Algorithm%3D%22http%3A%2F%2Fwww.w3.org%2F  
2001%2F10%2Fxml-exc-c14n%23%22+%2F%3E%3C%2FTransforms%3E%3C  
DigestMethod+Algorithm%3D%22http%3A%2F%2Fwww.w3.org%2F2000%  
%2F09%2Fxmldsig%23sha1%22+%2F%3E%3CDigestValue%3E%2F9ui0fsa  
61HZVJ15iLjstm48sJM%3D%3C%2FDigestValue%3E%3C%2FReference  
%3E%3C%2FSignedInfo%3E%3CSignatureValue%3EdJmJC1SMfVqTtEaf  
P0PRz%2BiOyTyCK6940jtBZICcaOMWxdUx%2FYmd0dGZPC2FMHRqKGCKta  
6JarXGWm0%2BYnL4ZhMvXDzisRO%2FFBApP6ra%3CYt91SSoMo%3D%3C%2F  
SignatureValue%3E%3CKeyInfo%3E%3CX509Data%3E%3CX509  
Certificate%3EMIIdojCCArqqAwIBAgIKG4FJ3gAAAAAEzxANBogkqhkiG9  
w0BAQUFADAZMRcwFQYDVQQDEw5UaGlzZFZhbnR5Um9vdDAEaW0wNjA3MTIy  
MTEwMDJaFw0wNjA3MTIyMTIwMDJAMBoxBjAQBGNVBMASTCXRyZXIzdhMTNzC
```


BnzANBqkqhkiG9w0BAQEFAAOBjQAwgYkCgYEAurI6nUCAigHhQ%2ByPwBP
kCeYTFUAsO%2FF%2BIyntOz%2F1HAqT%2BNuaG8v4oMn8ryMLTIZ9KUevLM
FV08azyl%2BtBeiOKMcWLV0JXH0t%2FnwWcjX5PFncdHqWJ%2B2HhMrIbf
RuUuzfb0uLTQKqveXYdfLJpKAay%2BxQqE6b5foxcVXu6NBjvUCAwEAAaOC
AaMwggGfMA4GA1UdDwEB%2FwQEAWIE8DBEBgkqhkiG9w0BCQ8ENzA1MA4GC
CqGS1b3QMCAGIAGDAOBgggqhkiG9w0DBAICAIaWbWYFKw4DAgcwCgYIKoZI
hvcNAwchQYDVR0OBBYEFNHRTdtbZ%2Bf6FcOfMpyeQ817ty3cMBMGA1UdJ
QQMMAoGCCSGAQUFBwMCMCB8GA1UdIwQYMBaAFBeJ9B2a4Q1SGuUkzmG%2BBk
iua9z1MFUGA1UdHwROMEwwSqBIOEaGRGH0dHA6Ly9jZXJ0YXV0aC5kb
nMuY29ycC5taWNyb3NvZnQuY29tL0N1cnRFbnJvbGwvVGHpcmRQYXJ0eVJv
b3QuY3JSMIGABggrBgEFBQcBAQSBjTCBiJBCBggrBgEFBQcwAoY2aHR0cDo
vL2N1cnRhdXR0L0N1cnRFbnJvbGwvY2VydGF1dGhfVGHpcmRQYXJ0eVJvb3
QuY3J0MEQGCCSGAQUFBzACHjhmaWx1Oi8vXFxjZXJ0YXV0aFxDZXJ0RW5yb
2xsXGN1cnRhdXR0X1RoaxJkUGFydH1Sb290LmNydDANBgkqhkiG9w0BAQUF
AAOCAQEAXtLcnh51HJ7XUNUFkr9BNAPy%2FqArolMojfXJRxdGuoFCmnRiV
%2FTYVFzSFCVOD9ChPnt4b0MPzXJLaAelXLhItiwt8gN4gQB1PhGnJ971J
RmUNgxLlueNM24qHJ3bRQBdKIXcaABnL0ICf76CEXUYBXg7mTiQz7rdOUKi
%2FDfweCeWoeFd7u%2BtJjrFX0W4fzdiASW9ymQp8jv%2BJPANCykOROyWx
CCvgGmBKRJ%2BaV%2FJ9zweaoo18BR11U7TDNfs9Acoi%2BTejlxFnZwf%2
B4rTXpxkXnIjWnMAcE6kcsqYHDTJPYc3p5RhCCeys9clyhkQznN62d7Qnu
HgTk9RwaPbpjlg%3D%3D%3C%2FX509Certificate%3E%3C%2FX509Data%
3E%3C%2FKeyInfo%3E%3C%2FSignature%3E%3C%2Fsaml%3AAssertion%
3E%3C%2Fwst%3ARequestedSecurityToken%3E%3Cwsp%3AAppliesTo+
xmlns%3Aawsp%3D%22http%3A%2F%2Fschemas.xmlsoap.org%2Fws%2F
2004%2F09%2Fpolicy%22%3E%3Cwsa%3AEndpointReference+xmlns
%3Awsa%3D%22http%3A%2F%2Fschemas.xmlsoap.org%2Fws%2F2004
%2F08%2Faddressing%22%3E%3Cwsa%3AAddress%3Ehttps%3A%2F
%2Ftreys-test%2Fclaims%2F%3C%2Fwsa%3AAddress%3E%3C%2F
wsa%3AEndpointReference%3E%3C%2Fwsp%3AAppliesTo%3E%3C
%2Fwst%3ARequestSecurityTokenResponse%3E%3Cwctx=https
%3A%2F%2Ftreys-test%2Fclaims%2FDefault.aspx

4.3.10 Final HTTP 200 OK Response from WS Resource

This message is an internal implementation detail of the WS-Resource and is shown here for completeness only. This message is not necessary for interoperability. The WS-Resource is not restricted by the protocol once it has received the token.

```
HTTP/1.1 200 OK
Cache-Control: no-cache
Pragma: no-cache
Content-Length: 10211
Content-Type: text/html; charset=utf-8
Expires: -1
Server: Microsoft-IIS/6.0
X-Powered-By: ASP.NET
X-AspNet-Version: 2.0.50727
Date: Thu, 13 Jul 2006 07:13:35 GMT

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
"http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" >
...
</html>
```

5 Security

5.1 Security Considerations for Implementers

The security considerations specified in [\[WSFederation1.2\]](#) section 16 that are relevant to passive requestors also apply to this protocol. Specific concerns are highlighted in this section.

5.1.1 Security Token Integrity

The integrity of a **security token** is compromised when the security token is modified. A digital **signature** on the **digest** of a security token enables the recipient of the security token to determine whether or not the security token has been modified since it was signed. The Microsoft Web Browser Federated Sign-On Protocol uses digital signatures to secure the integrity of the security token contained in the [wsignin1.0](#) response message while in transit. The **IP/STS** signs the security token that it issues, allowing the **relying party** to check for changes that might have occurred in transit. The strength of the digital signature depends on the signature algorithm used and the key sizes involved.[<87>](#)

5.1.2 Certificate Validation

When X.509 [\[X509\]](#) certificates are used, **relying parties** should validate the X.509 certificate that corresponds to the key used to sign the **security token**.[<88>](#) X.509 certificates might expire, might be revoked, or might not be issued by a **trusted** source. The steps required to validate X.509 certificates and to check the revocation status of an X.509 certificate are specified in [\[RFC3280\]](#). Implementers have to pay special attention to [\[RFC3280\]](#) section 9 for security considerations involving usage of X.509 certificates.

5.1.3 Confidentiality

Security tokens contain information about **users** and can contain sensitive data. This protocol requires the use of SSL/TLS, as specified in [Transport \(section 2.1\)](#). The use of SSL/TLS prevents the exposure of user information outside the services participating in the protocol, as well as helping to prevent replay attack.[<89>](#)

5.1.4 Replay Attack

[\[WSFederation1.2\]](#) section 16 specifies that **security tokens** can be replayed. SSL/TLS is the primary defense against replay, but implementers are to also understand that appropriate settings for the validity period of the token help to constrain the time that a security token can be replayed.[<90>](#)

5.1.5 Privacy

Privacy is a concern whenever the transmission of **user** information occurs. [\[WSFederation1.2\]](#) section 16 addresses some of the privacy issues, such as obtaining user permission for transmission of user data. The confidentiality measures specified in [Confidentiality \(section 5.1.3\)](#) also address some privacy concerns. Another privacy issue relevant to **single sign-on (SSO)** across **security realms** is the correlation of user information by **relying parties** using a common identifier. Because this protocol does not require the same identifier to be issued to every relying party, implementers have the option of implementing configurable behavior for the transmission of user identifiers to relying parties.[<91>](#)

Note Single sign-on is an optimization of user authentications that enables a user with a domain account to log on to a network once and gain access to all network resources. It removes the burden of repeating actions placed on the end user (for example, prompting for user names and passwords multiple times). To facilitate SSO, an IP/STS can provide evidence of authentication events and user

account information to third parties requesting information about the requestor (subject to policy and authorization restrictions). For more information, see [WSFederation1.2] sections 1.6 and 13. Note that the use of the term "sign on" or "sign-on" is based on [WSFederation1.2].

5.1.6 Identifiers

[Claim \(section 3.1.1.4\)](#) specifies the use of **UPN** and EmailAddress identifiers for **users**. The **relying party** will depend on the identifier being unique so collisions have to be avoided. Collisions can be avoided by configuring a relying party to only accept a specific set of suffix **domain naming service names** used in the UPN or EmailAddress claim of the **security token** issued by a **security realm's IP/STS**. This prevents a malicious IP/STS from enabling its users to impersonate users from another IP/STS. <92>

5.1.7 Cookies

For more information on the HTTP cookie state management mechanism, see [\[RFC2965\]](#). Cookies can be used to store state information about a **user** in the user's **web browser requestor** and to optimize the user experience. Because cookies can be used to identify a user's session and to store user information, special care has to be taken with the use of cookies. As specified in [\[WSFederation1.2\]](#) section 16, all cookies are to be set as secure. <93>

5.2 Index of Security Parameters

Security parameter	Section
<i>wauth</i>	wsignin1.0 request message
<i>wresult</i>	wsignin1.0 response message

6 Appendix A: Product Behavior

The information in this specification is applicable to the following Microsoft products or supplemental software. References to product versions include updates to those products.

- Windows Server 2003 R2 operating system
- Windows Server 2008 operating system
- Windows Server 2008 R2 operating system
- Windows Server 2012 operating system
- Windows Server 2012 R2 operating system
- Windows Server 2016 operating system
- Windows Server operating system
- Windows Server 2019 operating system
- Windows Server 2022 operating system
- Windows Server 2025 operating system

Exceptions, if any, are noted in this section. If an update version, service pack or Knowledge Base (KB) number appears with a product name, the behavior changed in that update. The new behavior also applies to subsequent updates unless otherwise specified. If a product edition appears with the product version, behavior is different in that product edition.

Unless otherwise specified, any statement of optional behavior in this specification that is prescribed using the terms "SHOULD" or "SHOULD NOT" implies product behavior in accordance with the SHOULD or SHOULD NOT prescription. Unless otherwise specified, the term "MAY" implies that the product does not follow the prescription.

[<1> Section 1.8](#): Windows uses the protocol extensions specified in [\[MS-MWBE\]](#). For further specifications on these extensions, see [\[MS-MWBE\]](#).

[<2> Section 1.8](#): Windows uses the existing extensibility points specified in [\[WSFederation1.2\]](#) as protocol extensions, as specified in [\[MS-MWBE\]](#). For details on these extensions, see [\[MS-MWBE\]](#).

[<3> Section 2.1](#): Except in the cases specified in [\[MS-MWBE\]](#) section 3.2.5.1.1, Windows uses POST methods to transmit wsignin1.0 responses to **relying parties**.

[<4> Section 2.1](#): Windows requires use of the HTTPS URL scheme to identify each **IP/STS** and **WS resource** participating in the protocol.

[<5> Section 2.1](#): Requests for wsignout1.0 and wsignoutcleanup1.0 are not authenticated by Windows.

[<6> Section 2.2](#): These messages are not supported. For the protocol behavior of Windows for such messages, see section [3.1.5.1](#).

[<7> Section 2.2](#): Parameters not specified in this document are not supported. For the protocol behavior of Windows on nonspecified parameters, see section [3.1.5.1](#).

[<8> Section 2.2.1](#): These parameters are not supported. For the protocol behavior of Windows on these parameters, see section [3.1.5.1](#).

[<9> Section 2.2.2](#): This parameter is supported. For the protocol behavior of Windows on this parameter, see sections [3.1.5.4.7](#) and [3.3.5.1.2](#).

<10> [Section 2.2.2](#): This parameter is not supported. For the protocol behavior of Windows for this parameter, see section 3.1.5.1.

<11> [Section 2.2.3](#): On Windows Server 2003 R2, Windows Server 2008, and Windows Server 2008 R2 operating system, this message contains either a *wtrealm* parameter or a *wreply* parameter. A *wtrealm* parameter is not mandatory if a *wreply* parameter is present.

<12> [Section 2.2.3](#): This parameter is supported by Windows. For the protocol behavior of Windows on this parameter, see sections 3.1.5.4.7 and 3.3.5.1.2.

<13> [Section 2.2.3](#): This parameter is supported by Windows. For the protocol behavior of Windows on this parameter, see section 3.3.5.1.2.

<14> [Section 2.2.3](#): The values supported for this parameter are specified in the following table. For the protocol behavior of Windows on this parameter, see sections [3.1.5.4.2](#) and 3.3.5.1.2

Method of authentication wanted	wauth URI
User name/password authentication	urn:oasis:names:tc:SAML:1.0:am:password
SSL client authentication	urn:ietf:rfc:2246
Windows integrated authentication	urn:federation:authentication:windows
Multiple factor authentication	http://schemas.microsoft.com/claims/multipleauthn
If the HTTP GET of the wsignin1.0 request message (section 2.2.3) contains an X-MS-Proxy HTTP header, then Windows integrated authentication; otherwise, multiple factor authentication. The X-MS-Proxy HTTP header is specified in [MS-ADFSPIP] section 2.2.1.1.	http://schemas.microsoft.com/claims/wiaormultiauthn

<15> [Section 2.2.3](#): This parameter is conditionally supported. For the supported conditions and processing semantics of Windows for this parameter, see sections 3.1.5.4.2 and 3.3.5.1.2.

<16> [Section 2.2.3](#): The *client-request-id* parameter is not supported on Windows Server 2003 R2, Windows Server 2008, Windows Server 2008 R2, or Windows Server 2012.

<17> [Section 2.2.3](#): The *login_hint* parameter is not supported on Windows Server 2003 R2, Windows Server 2008, Windows Server 2008 R2, or Windows Server 2012. Additionally, in **Active Directory Federation Services (AD FS)**, if the **ad_fs_behavior_level** ADM element, as defined in [\[MS-OAPX\]](#) section 3.2.1.1 (hereafter referred to simply as the **AD FS behavior level**), is set to AD_FS_BEHAVIOR_LEVEL_2 or higher, this parameter is used to derive the IP/STS that receives the wsignin1.0 request message.

<18> [Section 2.2.3](#): The *username* parameter is not supported on Windows Server 2003 R2, Windows Server 2008, Windows Server 2008 R2, or Windows Server 2012. Additionally, in AD FS, if the AD FS behavior level is set to AD_FS_BEHAVIOR_LEVEL_2 or higher, this parameter is used to derive the IP/STS that receives the wsignin1.0 request message.

<19> [Section 2.2.3](#): The *domain_hint* parameter is not supported on Windows Server 2003 R2, Windows Server 2008, Windows Server 2008 R2, Windows Server 2012, or Windows Server 2012 R2. Additionally, it is supported only in AD FS with AD FS behavior level set to AD_FS_BEHAVIOR_LEVEL_2 or higher, and ignored otherwise.

<20> [Section 2.2.3](#): The *prompt* parameter is not supported on Windows Server 2003 R2, Windows Server 2008, Windows Server 2008 R2, or Windows Server 2012. It is also not supported on Windows Server 2012 R2 unless [\[MSKB-3172614\]](#) is installed.

<21> [Section 2.2.3](#): Even though AD_FS_BEHAVIOR_LEVEL_3 is supported on Windows Server 2016, the *mfa_max_age* parameter is supported on Windows Server 2016 only if [\[MSKB-4088889\]](#) is installed.

<22> [Section 2.2.4](#): This parameter is supported. For the protocol behavior of Windows on this parameter, see section 3.1.5.4.7.

<23> [Section 2.2.4.1](#): The AppliesTo element is supported. For the protocol behavior of Windows for the AppliesTo element, see sections 3.1.5.4.7 and [3.3.5.2.2](#).

<24> [Section 2.2.4.1](#): Optional elements and attributes are not supported. For the protocol behavior of Windows for optional elements and attributes in the RSTR, see 3.1.5.4.7 and 3.3.5.2.2.

<25> [Section 2.2.4.2](#): The Advice element is used by Windows to extend the protocol. These extensions are as specified in [MS-MWBE].

<26> [Section 2.2.4.2.1.2](#): The AttributeStatement is conditionally supported. For more information on the protocol behavior of Windows for AttributeStatements, see sections 3.1.5.4.7 and 3.3.5.2.2.

<27> [Section 2.2.4.2.1.2](#): This namespace is supported. For the protocol behavior of Windows for AttributeNamespace values, see sections 3.1.5.4.7 and 3.3.5.2.2.

<28> [Section 2.2.4.2.1.3](#): On Active Directory Federation Services that shipped with Windows Server 2003 R2, Windows Server 2008, and Windows Server 2008 R2, Windows always specifies the NameIdentifier claim, and the value for the NameIdentifier element is the value of the EmailAddress, **user** principal name (**UPN**), or CommonName claim, as specified in the Abstract Data Model in section [3.1.1.4](#). The NameIdentifier element specifies the **Format** attribute, as specified in [\[SAMLCore\]](#) section 2.4.2.2. The corresponding value of the **Format** attribute is one of the following, as specified in the Abstract Data Model (see section 3.1.1.4).

Claim name	Format attribute URI
EmailAddress	urn:oasis:names:tc:SAML:1.1:nameid-format:emailAddress
UPN	http://schemas.xmlsoap.org/claims/UPN
CommonName	http://schemas.xmlsoap.org/claims/CommonName

<29> [Section 2.2.4.2.1.3](#): On Active Directory Federation Services that shipped with Windows Server 2003 R2, Windows Server 2008, and Windows Server 2008 R2, the SubjectConfirmation element is not supported. For the protocol behavior of Windows for the SubjectConfirmation element, see sections 3.1.5.4.7 and 3.3.5.2.2.

<30> [Section 2.2.4.2.2](#): Both direct inclusion and **SKI** are supported for X.509 certificates. For the protocol behavior of Windows for referencing X.509 certificates, see sections 3.1.5.4.7 and 3.3.5.2.2.

<31> [Section 2.2.5](#): The *wreply* parameter is not supported. For the protocol behavior of Windows for this parameter, see sections 3.1.5.4.7 and 3.3.5.1.2.

<32> [Section 2.2.6](#): The *wreply* parameter is not supported. For the protocol behavior of Windows for this parameter, see section [3.3.5.4.2](#).

<33> [Section 3.1.1.2](#): The **requestor IP/STS** and relying parties use internal data structures similar to Authentication Context and support all of the fields defined for the abstract data model.

<34> [Section 3.1.1.3](#): This information is exchanged using files that can be generated, or consumed, using export or import capabilities. Also, the information can be typed into a management console.

<35> [Section 3.1.1.3](#): Two separate records are used for a **federation** partner that is capable of playing both roles.

<36> [Section 3.1.1.3](#): Both configurations are supported. If a requestor IP/STS is configured as a farm of servers, each server can use a different private key, or the same private key can be installed on all servers.

<37> [Section 3.1.1.3](#): Metadata exchanged between federation partners includes **claims** requested by a relying party. A requestor IP/STS stores this information in local configuration data using the ClaimsOut concept.

<38> [Section 3.1.1.3](#): Metadata exchanged between federation partners includes claims requested by a relying party. A relying party stores this information in local configuration data using the **ClaimsIn** concepts.

<39> [Section 3.1.1.3](#): A requestor IP/STS includes as many claims as possible based on the attributes available for the user that is the subject of the **security token**. It sends the security token even if not all of the requested claims can be generated.

<40> [Section 3.1.1.4](#): A requestor IP/STS includes optional claims requested by a relying party in the AttributeStatement of the security token issued.

<41> [Section 3.1.1.4](#): Federation partners, performing either the requestor IP/STS or the relying party role, support the EmailAddress, UPN, CommonName, and Group claims, as defined in section 3.1.1.4. Based on local configuration, relying parties interpret Group claims as strings or map them to specific group objects defined in Active Directory.

<42> [Section 3.1.1.4](#): Federation partners, performing either the requestor IP/STS or the relying party role, support the Custom claim as defined. Windows rejects any request containing an **AttributeNamespace** value other than http://schemas.xmlsoap.org/claims with an HTTP 1.1 status code 500 server error.

<43> [Section 3.1.1.5](#): HTTP session cookies, as specified in [\[RFC2965\]](#), are used to identify individual **web browser requestors**.

<44> [Section 3.1.1.5.1](#): An HTTP session cookie, as specified in [\[RFC2965\]](#), is issued to a web browser requestor to manage the list of relying parties that are to be sent wsignoutcleanup1.0 messages when the user requests to **sign out**. The cookie is rewritten for every new security token that is issued for the web browser requestor session.

<45> [Section 3.1.1.5.2](#): An HTTP session cookie, as specified in [\[RFC2965\]](#), is issued to a web browser requestor to manage the list of requestor IP/STSs that are to be sent wsignout1.0 messages when the user requests to sign out. The cookie is rewritten for every new security token that is received for the web browser requestor session.

<46> [Section 3.1.2](#): A web browser requestor access request, such as an HTTP GET or POST, for a particular URL managed by a WS resource is rejected if the validity interval of the Authentication Context for the user has expired. Once an access request has been accepted, it will be processed even if the validity interval expires before the response is returned to the web browser requestor.

<47> [Section 3.1.2](#): Timers are not used to determine when validity intervals expire. The NotBefore and NotOnOrAfter values obtained from security tokens and recoded in Authentication Contexts are checked explicitly.

<48> [Section 3.1.5.1](#): Federation partners do not emit query string parameters when sending a protocol message using an HTTP POST. If any query string parameters are received with an HTTP POST, they are ignored.

<49> [Section 3.1.5.1](#): The xml-attribute-request and the xml-pseudonym-request messages described in section [2.2](#) are not emitted by Windows. Windows Server 2003 R2 returns an HTTP 403

error message. With the exception of Windows Server 2003 R2, all applicable Windows Server releases return an HTTP 500 error message if they receive such messages. Any parameters that are not specified in section 2.2 or in the protocol extension specified in [MS-MWBE] are ignored by Windows.

[<50> Section 3.1.5.2](#): An unsupported or unrecognized parameter received with a protocol message is ignored, and the message is processed as if it were not present.

[<51> Section 3.1.5.2](#): SOAP faults are not used. An HTTP 500 error response is returned to the web browser requestor.

[<52> Section 3.1.5.3.2](#): A WS resource stores the application URL in the *wctx* parameter. A **resource IP/STS** stores the WS resource URL in the *wctx* parameter by prepending it to the incoming value.

[<53> Section 3.1.5.3.3](#): A web browser requestor can send an HTTP GET with *whr*, *domain_hint*, *username*, or *login_hint* directly to WS resource URLs. A relying party uses the *whr* parameter for **security realm** discovery when present. If *whr* is not present, AD FS server with an AD FS behavior level of AD_FS_BEHAVIOR_LEVEL_2 or higher will additionally use *domain_hint*, *username*, or *login_hint* for security realm discovery. The resource IP/STS is expected to know how to translate the hint (or get it translated) to a URI that uniquely identifies the requestor IP/STS. See [\[OIDCCore\]](#) section 3.1.2.1 for recommendations on *login_hint*. If no realm can be identified, a resource IP/STS displays an HTML form to collect the information from the user.

If *whr*, *domain_hint*, *username*, and *login_hint* coexist in one request, the precedence, in descending order, for their use in identifying the requestor IP/STS is as follows:

- *whr*
- *domain_hint*
- *username*
- *login_hint*

[<54> Section 3.1.5.3.3](#): A resource IP/STS writes a persistent cookie (for more information, see [\[RFC2965\]](#)) to the web browser requestor to preserve the value of the security realm identifier.

[<55> Section 3.1.5.4.2](#): On Windows Server 2003 R2, Windows Server 2008, and Windows Server 2008 R2, if a resource IP/STS receives a request that has both the *wreply* and *wtrealm* parameters set, the resource IP/STS returns an HTTP 1.1 status code 500 server error. If a requestor IP/STS receives a request that has both the *wreply* and *wtrealm* parameters set, the requestor IP/STS ignores the *wreply*.

[<56> Section 3.1.5.4.2](#): If the *wauth* parameter, which is described in section 2.2.3, is present and is not set to one of the values from the table in that section, Windows returns an HTTP 1.1 status code 500 server error.

[<57> Section 3.1.5.4.3](#): An IP/STS can authenticate a user by means of the Windows implementation of Kerberos (for more information, see [\[RFC4120\]](#)) by collecting the user's Windows identifier and password in an HTML form (using HTTPS) and validating the credentials with Active Directory, or by client SSL authentication using an X.509 certificate [\[X509\]](#) issued to the web browser requestor. An IP/STS can also authenticate the user by initiating the Microsoft Web Browser Federated Sign-On Protocol with another **STS**.

[<58> Section 3.1.5.4.4](#): Attributes are retrieved from either Active Directory or Active Directory Application Mode using authenticated LDAP binds.

[<59> Section 3.1.5.4.7](#): The following Windows behaviors apply for response message processing:

- The *wctx* parameter specified in section [2.2.2](#) is always returned by Windows with the response if the parameter is present in the request.

- When emitting wsignin1.0 responses, Windows includes the wsp:AppliesTo element.
- When emitting wsignin1.0 response messages, Windows does not include any other optional elements or attributes in the RSTR.
- The presence of the SAML AttributeStatement in the SAML assertion in a wsignin1.0 response message emitted by Windows depends on configuration of the product. Windows supports emitting both no SAML AttributeStatement and one SAML AttributeStatement. Windows includes a SAML AttributeStatement when more than one claim is included in the token. For further specifications, see sections 3.1.1.4 and [3.1.5](#).
- When emitting wsignin1.0 response messages, Windows uses the namespace <http://schemas.xmlsoap.org/claims>.
- On Active Directory Federation Services that shipped with Windows Server 2003 R2, Windows Server 2008, and Windows Server 2008 R2, Windows does not specify the SubjectConfirmation element when emitting wsignin1.0 response messages.
- When emitting wsignin1.0 response messages, Windows includes the X.509 certificate directly in the KeyInfo element by default. Windows does not reference X.509 certificates [X509] using the X509SKI element by default when emitting wsignin1.0 response messages. Windows can be configured to use the X509SKI element when emitting wsignin1.0 response messages.

[<60> Section 3.1.6](#): Timers are not used to determine when validity intervals expire. The NotBefore and NotOnOrAfter values obtained from security tokens and recoded in Authentication Contexts are checked explicitly.

[<61> Section 3.2.5.2.2](#): An STS, regardless of role, sends wsignoutcleanup1.0 messages to relying parties. An HTTP session cookie (for more information, see [RFC2965]) is issued to a web browser requestor to manage the list of relying parties that are to be sent wsignoutcleanup1.0 messages when the user requests to sign out. A requestor IP/STS does not cache any user or web browser requestor session data on a server.

[<62> Section 3.2.5.2.3](#): The *wreply* parameter is not supported for wsignout1.0 request messages, so the web browser requestor is not redirected after wsignoutcleanup1.0 messages are sent.

[<63> Section 3.2.5.3.2](#): An HTTP session cookie (for more information, see [RFC2965]) is issued to a web browser requestor to manage the list of relying parties. A requestor IP/STS sends wsignoutcleanup1.0 messages to all relying parties identified in the session cookie when a user requests to sign-out.

[<64> Section 3.2.5.3.3](#): All of the web browser requestor session state that needs to be deleted is maintained in an HTTP session cookie (for more information, see [RFC2965]). This cookie is deleted as part of the process of sending wsignoutcleanup1.0 messages.

[<65> Section 3.2.5.3.4](#): When a wsignout1.0 message is received, Windows sends a wsignoutcleanup1.0 message to all relying parties identified by the Outbound Sessions List maintained for the web browser requestor. An HTML page that contains a set of IFrames, one for each WS resource, is constructed. In the process of returning this HTML page to the web browser requestor, the HTTP session cookie used to maintain the Outbound Sessions List is deleted. Processing of the iframes in this HTML page by the web browser requestor causes the wsignoutcleanup1.0 messages to be sent in parallel. Windows does not use the *wreply* parameter when emitting wsignoutcleanup1.0 messages.

[<66> Section 3.2.6](#): Timers are not used to determine when validity intervals expire. The NotBefore and NotOnOrAfter values obtained from security tokens (see section [2.2.4.2](#)) and recoded in Authentication Contexts are checked explicitly.

[<67> Section 3.3.1](#): A relying party is implemented as separate resource IP/STS and WS resource components. This factorization is used whether or not the components are deployed on the same server or on different servers.

<68> [Section 3.3.1](#): A relying party is factored into separate resource IP/STS and WS resource components even if the components are located on the same server. Interactions between the resource IP/STS and WS resource are not exposed to requestor IP/STSs in other security realms.

<69> [Section 3.3.1.1](#): The roles a federation partner can perform are indicated in configuration data, as described in [section 3.3.1.1](#). This information is obtained using out-of-band metadata exchange. Files can be exchanged using export or import capabilities, or the data can be typed into a management console.

<70> [Section 3.3.1.1](#): Federation partner URLs are maintained in configuration data. This configuration data is always treated as authoritative versus the value of a **wreply** parameter received in a wsignin1.0 message.

<71> [Section 3.3.2](#): Timers are not used to determine when validity intervals expire. The NotBefore and NotOnOrAfter values obtained from security tokens, as specified in [section 2.2.4.2](#), and recoded in Authentication Contexts are checked explicitly. A resource IP/STS or a WS resource will reject a security token for which the validity interval has expired. A WS resource maintains a user's Authentication Context in an HTTP session cookie (for more information, see [RFC2965]). The cookie is signed to prevent undetected manipulation. The WS resource will return an HTTP 500 error for any request that is accompanied by a session cookie for which the validity interval has expired.

<72> [Section 3.3.3](#): By default, relying parties support CRL checking by means of a CDP. However, if the CDP is not accessible, CRL checking can be disabled to prevent otherwise acceptable security tokens from being rejected.

<73> [Section 3.3.5.1.1](#): A dialect of the Microsoft Web Browser Federated Sign-On Protocol is used, with the WS resource acting as a relying party and the resource IP/STS acting as a requestor IP/STS. The WS resource sends a wsignin1.0 request message to the resource IP/STS to request a security token for the user.

<74> [Section 3.3.5.1.2](#): The following Windows behaviors apply when marshaling parameters for a request message:

- When a WS resource sends a wsignin1.0 request message, the *wrealm* parameter described in [Common Syntax for Request Messages \(section 2.2.1\)](#) cannot be used because the WS resource and the resource IP/STS are located in the same security realm. Because the *wrealm* parameter identifies a security realm, the *wrealm* parameter cannot be used to distinguish different federation partners in the same security realms. The *wreply* parameter is used instead, and it is set to the URL of the WS resource.
- Windows also includes the *wctx* parameter specified in Common Syntax for Response Messages ([section 2.2.2](#)) and wsignin1.0 request message ([section 2.2.3](#)) to preserve any *URL* parameters of the original HTTP request to the relying party.
- Windows always includes the *wct* parameter specified in the wsignin1.0 request message ([section 2.2.3](#)) in the request message.
- By default, Windows does not emit the *wauth* parameter specified in the wsignin1.0 request message ([section 2.2.3](#)). Windows can be configured to issue the *wauth* parameter and will issue a *wauth* parameter with one of the URIs specified in wsignin1.0 request message ([section 2.2.3](#)).
- The *whr* parameter specified in wsignin1.0 request message ([section 2.2.3](#)) is not emitted by a relying party unless the *whr* parameter is received on the original request to the Windows relying party and the destination IP/STS is in the same security realm. If a Windows resource IP/STS acting as a relying party receives the *whr* parameter specified in wsignin1.0 request message ([section 2.2.3](#)), the wsignin1.0 request message is forwarded to the requestor IP/STS corresponding to the URI value. The *whr* parameter is not included in the request forwarded to the Windows requestor IP/STS. If no requestor IP/STS corresponds to the URI value of the *whr* parameter, Windows ignores the parameter.

<75> [Section 3.3.5.2.2](#): The following Windows behaviors apply when validating wsignin1.0 response messages:

- Windows processes wsp:AppliesTo messages according to the message syntax requirements specified in section [2.2.4.1](#) when receiving wsignin1.0 responses.
- Windows ignores any other optional elements included in the RSTR when processing received wsignin1.0 response messages.
- Windows supports processing both no SAML AttributeStatement and one SAML AttributeStatement when receiving wsignin1.0 responses.
- Windows rejects the security token with an HTTP 1.1 500 error code if a wsignin1.0 response is received with an Attribute element that has an AttributeNamespace value other than <http://schemas.xmlsoap.org/claims>.
- When processing received wsignin1.0 messages, Windows ignores the SubjectConfirmation element, if present.
- When receiving wsignin1.0 responses, Windows uses the certificate included directly in the X509Certificate element, if present. When receiving wsignin1.0 responses, Windows uses the SKI, if present, to look up the corresponding certificate and continue to process the message. A failed look-up will result in the wsignin1.0 response messages being rejected with an HTTP 1.1 500 server error.
- When receiving wsignin1.0 responses, Windows rejects any messages with the **EncryptedData** element described in section 2.2.4.1 with an HTTP 1.1 500 server error.

<76> [Section 3.3.5.2.4](#): A user's identity from the Subject element of the security token is mapped to a local Windows security principal identity in Active Directory based on local configuration. In terms of the Authentication Context abstract data model, the local Windows security principal identity replaces the AuthIdentity value derived from the Subject of a security token.

<77> [Section 3.3.5.2.6](#): The Microsoft Web Browser Federated Sign-On Protocol is used with the WS resource acting as a relying party and the resource IP/STS acting as a requestor IP/STS. The resource IP/STS sends a wsignin1.0 response message to the WS resource to issue a security token for the user. There is no change to the protocol message processing, as specified in section Issuing a Security Token by responding to a wsignin1.0 request message (section [3.1.5.4](#)).

<78> [Section 3.3.5.3.2](#): The *wreply* parameter described in section wsignout1.0 Request Message (section [2.2.5](#)) is not supported when emitting wsignout1.0 Request Messages.

<79> [Section 3.3.5.3.3](#): Relying party components, both resource IP/STSs and WS resources, send wsignout1.0 messages to security token services. An HTTP session cookie (for more information, see [RFC2965]) is issued to a web browser requestor to manage the list of security token services that are to be sent wsignout1.0 messages when the user requests to sign-out.

<80> [Section 3.3.5.4.2](#): A security token service, regardless of role, sends wsignoutcleanup1.0 messages to relying parties. An HTTP session cookie (for more information, see [RFC2965]) is issued to a web browser requestor to manage the list of relying parties that are to be sent wsignoutcleanup1.0 messages when the user requests to sign-out. A resource IP/STS does not cache any user or web browser requestor session data on a server.

<81> [Section 3.3.5.4.3](#): A security token service, regardless of role, sends wsignoutcleanup1.0 request messages to relying parties. An HTTP session cookie (for more information, see [RFC2965]) is issued to a web browser requestor to manage the list of relying parties that are to be sent wsignoutcleanup1.0 messages when the user requests to sign-out.

<82> [Section 3.3.5.4.4](#): An HTML page that contains a set of iframes (as specified in [\[HTML\]](#) section 16.5), one for each WS resource, is returned to the web browser requestor. Processing of the iframes by the web browser requestor causes the wsignoutcleanup1.0 messages to be sent in parallel.

<83> [Section 3.3.5.4.5](#): When receiving a wsignoutcleanup1.0 request message (section [2.2.6](#)), the web browser requestor is not redirected to the *wreply* URL after sign-out processing is complete. Windows completes by returning a string indicating that clean up is complete for the security realm, regardless of the presence of *wreply*.

<84> [Section 3.3.6](#): Timers are not used to determine when validity intervals expire. The NotBefore and NotOnOrAfter values obtained from security tokens, as specified in section 2.2.4.2, and recoded in Authentication Contexts are checked explicitly.

<85> [Section 3.4.1](#): Windows Internet Explorer supports the use of session and persistent HTTP cookies (for more information, see [\[RFC2965\]](#)). The Microsoft implementation of this protocol requires that web browser requestors support at least session cookies. It uses persistent cookies to preserve security realm identifiers if they are supported by the web browser requestor.

<86> [Section 3.4.5](#): The RMS 2.0 client in Windows Vista operating system with Service Pack 1 (SP1), Windows 7 operating system, Windows 8 operating system, Windows 8.1 operating system, and Windows 10 operating system, and the RMS 2.0 client in all applicable Windows Server releases with the exception of Windows Server 2003 R2, adds a *whr* parameter to the wsignin 1.0 Request Message (section 2.2.3) if the wsignin 1.0 Request Message does not already contain a *whr* parameter.

<87> [Section 5.1.1](#): The Windows IP/STS only supports the **Rivest-Shamir-Adleman (RSA)** algorithm ([\[RFC8017\]](#)) for **signatures**, and supports the SHA-1 algorithm (see [\[FIPS180\]](#)) for calculating **digests**. The default key length of the Windows RSA keys used for signing security tokens is 2,048 bits.

<88> [Section 5.1.2](#): The certificate validation behavior of Windows is configurable. By default, for certificates used to sign security tokens, Windows ensures that the current time is in the certificate's validity interval, that the certificate has a valid signature, that the certificate is issued by a **trusted** authority, and that the certificate serial number is not present in the CRL of the issuing authority.

<89> [Section 5.1.3](#): The use of SSL/TLS is required for each Windows IP/STS and WS resource.

<90> [Section 5.1.4](#): The validity period of security tokens issued by the Windows IP/STS is limited to 8 hours by default.

<91> [Section 5.1.5](#): The Windows requestor IP/STS can be configured to issue a different identifier to each resource IP/STS for each user to prevent correlation of user information across multiple relying parties. This behavior is turned off by default.

<92> [Section 5.1.6](#): The Windows resource IP/STS requires that messages from each requestor IP/STS be restricted to only using a specific set of suffix **DNS names** when UPN or EmailAddress is used as the unique identifier for the user by that requestor IP/STS.

<93> [Section 5.1.7](#): Windows sets cookies as secure.

7 Change Tracking

This section identifies changes that were made to this document since the last release. Changes are classified as Major, Minor, or None.

The revision class **Major** means that the technical content in the document was significantly revised. Major changes affect protocol interoperability or implementation. Examples of major changes are:

- A document revision that incorporates changes to interoperability requirements.
- A document revision that captures changes to protocol functionality.

The revision class **Minor** means that the meaning of the technical content was clarified. Minor changes do not affect protocol interoperability or implementation. Examples of minor changes are updates to clarify ambiguity at the sentence, paragraph, or table level.

The revision class **None** means that no new technical changes were introduced. Minor editorial and formatting changes may have been made, but the relevant technical content is identical to the last released version.

The changes made to this document are listed in the following table. For more information, please contact dochelp@microsoft.com.

Section	Description	Revision class
5.1.1 Security Token Integrity	In behavior note, replaced one instance of obsoleted Normative reference [RFC3447] with [RFC8017] and added glossary term 'Rivest-Shamir-Adleman (RSA)'.	Major
6 Appendix A: Product Behavior	Added Windows Server 2025 to the list of applicable products.	Major

8 Index

A

Abstract data model
 relying party ([section 3.1.1](#) 23, [section 3.3.1](#) 35)
 requestor IP/STS ([section 3.1.1](#) 23, [section 3.2.1](#) 32)
 [web browser requestor](#) 40
[Applicability](#) 15
[Assertion statements](#) 20
[Attribute element example](#) 48
[Attribute statements](#) 20
[Authentication](#) 31
[Authentication statements](#) 20

C

Capability negotiation ([section 1.7](#) 15, [section 1.7.2](#) 15)
[Certificate validation](#) 58
[Change tracking](#) 69
Claim
 [IP/STS](#) 25
 [relying party](#) 25
Claim mapping ([section 3.1.5.4.5](#) 31, [section 3.3.5.2.5](#) 38)
Cleanup processing ([section 3.2.5.2.2](#) 34, [section 3.2.5.3.3](#) 34, [section 3.3.5.4.2](#) 39)
[Common Syntax for Request Messages message](#) 17
[Common Syntax for Response Messages message](#) 17
[Confidentiality](#) 58
[Cookies](#) 59

D

Data model - abstract
 relying party ([section 3.1.1](#) 23, [section 3.3.1](#) 35)
 requestor IP/STS ([section 3.1.1](#) 23, [section 3.2.1](#) 32)
 [web browser requestor](#) 40
[Directory service schema elements](#) 22

E

[Elements - directory service schema](#) 22
[Error handling](#) 29
Examples
 [final HTTP 200 OK response from WS resource example](#) 57
 [HTTP Get to Requestor IP/STS example](#) 50
 [HTTP Get To Resource IP/STS example](#) 49
 [HTTP post security token to resource IP/STS example](#) 52
 [HTTP post security token to WS resource example](#) 55
 [HTTP Redirect to Requestor IP/STS example](#) 50
 [HTTP Redirect to Resource IP/STS example](#) 49
 [message flows](#) 42
 [original get to WS resource example](#) 49
 [overview](#) 42
 [raw messages examples](#) 49

[receive security token from requestor IP/STS in HTML form example](#) 50
 [receive security token from resource IP/STS in HTML form example](#) 54
 [RSTR](#) 48
 [SAML attribute element example](#) 48
 [X509Certificate element example](#) 48
 [X509SKI element example](#) 49
 [XML examples](#) 48

F

Federation partner
 [IP/STS](#) 24
 [relying party](#) 24
 [web browser requestor](#) 27
[Fields - vendor-extensible](#) 15
[Final HTTP 200 OK response from WS resource example](#) 57

G

[Glossary](#) 8

H

Higher-layer triggered events
 relying party ([section 3.1.4](#) 28, [section 3.3.4](#) 36)
 requestor IP/STS ([section 3.1.4](#) 28, [section 3.2.4](#) 33)
 [web browser requestor](#) 40
 [High-level result - wresult](#) 19
 [HTTP Get to Requestor IP/STS example](#) 50
 [HTTP Get To Resource IP/STS example](#) 49
 [HTTP post security token to resource IP/STS example](#) 52
 [HTTP post security token to WS resource example](#) 55
 [HTTP Redirect to Requestor IP/STS example](#) 50
 [HTTP Redirect to Resource IP/STS example](#) 49

I

[Identification](#) 31
[Identifiers](#) 59
[Implementer - security considerations](#) 58
[Inbound wsignout1.0 request message processing](#) 34
[Inbound wsignoutcleanup1.0 request message processing](#) 39
[Index of security parameters](#) 59
[Informative references](#) 12
Initialization
 relying party ([section 3.1.3](#) 28, [section 3.3.3](#) 36)
 requestor IP/STS ([section 3.1.3](#) 28, [section 3.2.3](#) 33)
 [web browser requestor](#) 40
[Introduction](#) 8
IP/STS
 [claim](#) 25
 [federation partner](#) 24
 [user authentication context](#) 23
 [web browser requestor sessions list](#) 27

L

Local events

relying party ([section 3.1.7](#) 32, [section 3.3.7](#) 40)
requestor IP/STS ([section 3.1.7](#) 32, [section 3.2.7](#) 35)
[web browser requestor](#) 41

M

[Message flow example](#) 42

Message processing

relying party ([section 3.1.5](#) 28, [section 3.3.5](#) 36)
requestor IP/STS ([section 3.1.5](#) 28, [section 3.2.5](#) 33)
web browser requestor ([section 3.1.5](#) 28, [section 3.4.5](#) 40)

Message transmission ([section 3.2.5.3.4](#) 35, [section 3.3.5.3.4](#) 39, [section 3.3.5.4.4](#) 39)

[Message type - determining](#) 29

Message validation ([section 3.1.5.4.2](#) 31, [section 3.3.5.2.2](#) 38)

Messages

[Common Syntax for Request Messages](#) 17
[Common Syntax for Response Messages](#) 17
[determining type](#) 29
[overview](#) 16
[syntax](#) 16
[transmission](#) 30
[transport](#) 16
[wsignin1.0 Request Message](#) 17
[wsignin1.0 Response Message](#) 19
[wsignout1.0 Request Message](#) 21
[wsignoutcleanup1.0 Request Message](#) 22

N

[Normative references](#) 10

O

[Original get to WS resource example](#) 49

[Outbound wsignout1.0 request message processing](#) 38

[Outbound wsignoutcleanup1.0 request message processing](#) 34

[Overview](#) 23

[Overview \(synopsis\)](#) 12

P

Parameter marshaling ([section 3.1.5.3.2](#) 29, [section 3.3.5.1.2](#) 37, [section 3.3.5.3.2](#) 38)

[Parameters - security](#) 59

[Parameters - security index](#) 59

[Preconditions](#) 14

[Prerequisites](#) 14

[Privacy](#) 58

[Product behavior](#) 60

Protocol activation ([section 3.1.5.3.1](#) 29, [section 3.1.5.4.1](#) 31, [section 3.2.5.2.1](#) 34, [section 3.2.5.3.1](#) 34, [section 3.3.5.1.1](#) 37, [section](#)

[3.3.5.2.1](#) 38, [section 3.3.5.3.1](#) 38, [section 3.3.5.4.1](#) 39)

[Protocol Details](#) 23
[overview](#) 23

R

[Raw messages examples](#) 49

[Receive security token from requestor IP/STS in HTML form example](#) 50

[Receive security token from resource IP/STS in HTML form example](#) 54

[References](#) 10

[informative](#) 12

[normative](#) 10

[Relationship to other protocols](#) 13

Relying Party

[abstract data model](#) 23

[claim](#) 25

[federation partner](#) 24

higher-layer triggered events ([section 3.1.4](#) 28, [section 3.3.4](#) 36)

initialization ([section 3.1.3](#) 28, [section 3.3.3](#) 36)

local events ([section 3.1.7](#) 32, [section 3.3.7](#) 40)

message processing ([section 3.1.5](#) 28, [section 3.3.5](#) 36)

overview ([section 3.1](#) 23, [section 3.3](#) 35, [section 3.3.1](#) 35)

security token ([section 3.1.1.1](#) 23, [section 3.1.5.3](#) 29, [section 3.1.5.4](#) 30, [section 3.3.5.1](#) 37, [section 3.3.5.2](#) 37)

sequencing rules ([section 3.1.5](#) 28, [section 3.3.5](#) 36)

timer events ([section 3.1.6](#) 32, [section 3.3.6](#) 40)

timers ([section 3.1.2](#) 28, [section 3.3.2](#) 36)

[user authentication context](#) 23

[web browser requestor sessions list](#) 27

Relying party security realm ([section 3.2.5.3.2](#) 34, [section 3.3.5.4.3](#) 39)

[Replay attack](#) 58

Request messages

[syntax](#) 17

[wsignin1.0](#) 17

[wsignout1.0](#) 21

[wsignoutcleanup1.0](#) 22

Requestor IP/STS

abstract data model ([section 3.1.1](#) 23, [section 3.2.1](#) 32)

higher-layer triggered events ([section 3.1.4](#) 28, [section 3.2.4](#) 33)

initialization ([section 3.1.3](#) 28, [section 3.2.3](#) 33)

local events ([section 3.1.7](#) 32, [section 3.2.7](#) 35)

message processing ([section 3.1.5](#) 28, [section 3.2.5](#) 33)

overview ([section 3.1](#) 23, [section 3.2](#) 32)

security token ([section 3.1.1.1](#) 23, [section 3.1.5.3](#) 29, [section 3.1.5.4](#) 30, [section 3.2.5.1](#) 33)

sequencing rules ([section 3.1.5](#) 28, [section 3.2.5](#) 33)

timer events ([section 3.1.6](#) 32, [section 3.2.6](#) 35)

timers ([section 3.1.2](#) 28, [section 3.2.2](#) 32)

Requestor IP/STS security realm ([section 3.1.5.3.3](#) 30, [section 3.3.5.3.3](#) 39)

[Resource access control](#) 38

[Resource IP/STS abstract data model extensions](#) 35

Response message ([section 3.1.5.4.7](#) 32, [section 3.2.5.2.3](#) 34, [section 3.3.5.4.5](#) 39)
Response messages
[syntax](#) 17
[wsignin1.0](#) 19
[RSTR example](#) 48

S

[SAML assertion](#) 32
[SAML attribute element example](#) 48
[Schema elements - directory service](#) 22
Security
[certificate validation](#) 58
[confidentiality](#) 58
[cookies](#) 59
[identifiers](#) 59
[implementer considerations](#) 58
[parameter index](#) 59
[privacy](#) 58
[replay attack](#) 58
[token integrity](#) 58
Security token
relying party ([section 3.1.1.1](#) 23, [section 3.1.5.3](#) 29, [section 3.1.5.4](#) 30, [section 3.3.5.1](#) 37, [section 3.3.5.2](#) 37)
requestor IP/STS ([section 3.1.1.1](#) 23, [section 3.1.5.3](#) 29, [section 3.1.5.4](#) 30, [section 3.2.5.1](#) 33)
[Security token format](#) 19
[Security token integrity](#) 58
[Security token signature](#) 21
Sequencing rules
relying party ([section 3.1.5](#) 28, [section 3.3.5](#) 36)
requestor IP/STS ([section 3.1.5](#) 28, [section 3.2.5](#) 33)
web browser requestor ([section 3.1.5](#) 28, [section 3.4.5](#) 40)
[Signature - security token](#) 21
[Standards assignments](#) 15
Statements
[Assertion](#) 20
[Attribute](#) 20
[Authentication](#) 20
[Subject element](#) 21
Syntax
[overview](#) 16
[request messages](#) 17
[response messages](#) 17

T

Timer events
relying party ([section 3.1.6](#) 32, [section 3.3.6](#) 40)
requestor IP/STS ([section 3.1.6](#) 32, [section 3.2.6](#) 35)
[web browser requestor](#) 41
Timers
relying party ([section 3.1.2](#) 28, [section 3.3.2](#) 36)
requestor IP/STS ([section 3.1.2](#) 28, [section 3.2.2](#) 32)
[web browser requestor](#) 40
[Tracking changes](#) 69
[Transmitting messages](#) 30
[Transport](#) 16

Triggered events - higher-layer
relying party ([section 3.1.4](#) 28, [section 3.3.4](#) 36)
requestor IP/STS ([section 3.1.4](#) 28, [section 3.2.4](#) 33)
[web browser requestor](#) 40

U

User attributes ([section 3.1.5.4.4](#) 31, [section 3.3.5.2.4](#) 38)
User authentication context
[IP/STS](#) 23
[relying party](#) 23
User identification and authentication ([section 3.1.5.4.3](#) 31, [section 3.3.5.2.3](#) 38)

V

[Vendor-extensible fields](#) 15
Versioning ([section 1.7](#) 15, [section 1.7.1](#) 15)

W

Web browser requestor
[abstract data model](#) 40
[federation partner](#) 27
[higher-layer triggered events](#) 40
[initialization](#) 40
[IP/STS - sessions list](#) 27
[local events](#) 41
message processing ([section 3.1.5](#) 28, [section 3.4.5](#) 40)
[overview](#) 40
[relying party - sessions list](#) 27
sequencing rules ([section 3.1.5](#) 28, [section 3.4.5](#) 40)
[timer events](#) 41
[timers](#) 40
[wresult](#) 19
[WS resource abstract data model extensions](#) 36
[wsignin1.0](#) ([section 2.2.3](#) 17, [section 2.2.4](#) 19)
[wsignin1.0 Request Message message](#) 17
[wsignin1.0 Response Message message](#) 19
[wsignout1.0](#) 21
[wsignout1.0 Request Message message](#) 21
[wsignoutcleanup1.0](#) 22
[wsignoutcleanup1.0 Request Message message](#) 22

X

[X509Certificate element example](#) 48
[X509SKI element example](#) 49
[XML examples](#) 48